

Required Readings

Required Readings for this program are open access, which means you should be able to access them at no cost. The link provided in the citation will take you directly to the reading or to a page where you can download it.

1. Yu, Tong, and Hong Zhu. "Hyper-Parameter Optimization: A Review of Algorithms and Applications." *arXiv*, 2020, <https://arxiv.org/abs/2003.05689>.

- **Required pages:** Sections 2 and 3
- **Estimated reading time:** 90 minutes

MACHINE LEARNING IN FINANCE

MODULE 7 | LESSON 1

MODEL SELECTION AND HYPERPARAMETER TUNING

Reading Time	50 minutes
Prior Knowledge	Machine learning
Keywords	Hyperparameter, grid search, random search, Cross validation

In the previous modules, we have trained and evaluated the performance of machine learning models. Machine learning models have a number of parameters that the data learn; model parameters are found by fitting our model on the training dataset. Hyperparameters on the other hand cannot be learned by our normal training process and are fixed before we begin the training process. In this module, we will learn more about hyperparameters. In this lesson, we will begin by discussing how to select the best performing model and the various hyperparameter tuning strategies.

1. Improving Model Performance

Before putting our machine learning model(s) to production, we should make sure that the model gives the best possible solution devoid of bias and can generalize well on new unseen data.

We will cover the following topics in this section.

- Overfitting and underfitting.
- Train, validation, and test sets.
- Cross-validation.

We begin with data splitting into train, test, and validation sets.

1.1 Train, validation and tests sets

It is advisable to split the data into training, testing, and validation sets especially if we want to tune our hyperparameters. The validation set helps us get an unbiased evaluation of our model performance and can therefore tune our model hyperparameters before passing the model to the test dataset.

Given a dataset, we can get the train, validation, and test set by using one of the following:

- Random Split
- Stratified Split

Random split involves splitting the original data into pre-defined proportions representing the train, validation, and test sets by first shuffling the data so as to overcome the challenge of there existing a pattern based on the data indices.

Stratified split splits the train, validation, and test sets according to the proportion of samples in the target variable. Stratified split is often used when our data is imbalanced.

Let us see an example of this using the California housing dataset whose target variable is the house price median.

```
In [1]: from sklearn.datasets import fetch_california_housing

housing = fetch_california_housing(as_frame=True)
X, y = housing.data, housing.target * 10
```

```
In [2]: X.head()
```

```
Out[2]:
```

	MedIn c	House Age	AveRo oms	AveBe drms	Popula tion	AveOc cup	Latitu de	Longit ude
0	8.3252	41.0	6.9841 27	1.0238 10	322.0	2.5555 56	37.88	-122.23
1	8.3014	21.0	6.2381 37	0.9718 80	2401.0	2.1098 42	37.86	-122.22
2	7.2574	52.0	8.2881 36	1.0734 46	496.0	2.8022 60	37.85	-122.24
3	5.6431	52.0	5.8173 52	1.0730 59	558.0	2.5479 45	37.85	-122.25
4	3.8462	52.0	6.2818 53	1.0810 81	565.0	2.1814 67	37.85	-122.25

```
In [3]: y.head()
```

```
Out[3]: 0    45.26
1    35.85
2    35.21
3    34.13
4    34.22
Name: MedHouseVal, dtype: float64
```

We will use the decision tree regressor to predict the median house price with an interest in getting the generalization of our model. So, we start by training the model on the entire dataset.

```
In [4]: from sklearn.tree import DecisionTreeRegressor

regressor = DecisionTreeRegressor(random_state=42)
regressor.fit(X, y)
```

```
Out[4]: ▼ DecisionTreeRegressor ⓘ ?
        ► Parameters
        ► Fitted attributes
```

We also evaluate the model on the dataset to see how it performs.

```
In [5]: from sklearn.metrics import mean_absolute_error

target_predicted = regressor.predict(X)
score = mean_absolute_error(y, target_predicted)
print(f"On average, our regressor makes an error of {score:.3f} ")
```

On average, our regressor makes an error of 0.000

The resulting mean absolute error is actually very impressive: it says that our model actually got the median price of each house accurately. So, the question is should we pick this model? The simple answer is not so fast! Remember we trained and evaluated our model on the same dataset and the decision tree regressor could have memorized the dataset and would therefore give us the wrong notion about its ability to generalize on unseen data.

When training our model, we are interested in it minimizing the error on unseen data, and this is the idea behind us splitting the data into train and test sets.

So, we can begin by splitting our dataset as shown below. Again, we train our model on the training set and evaluate its performance on the same data and record the error.

```
In [6]: from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=0)
```

We again train the model on our training data.

```
In [7]: regressor.fit(X_train, y_train)
```

```
Out[7]: ▼ DecisionTreeRegressor ⓘ ?
        ► Parameters
        ► Fitted attributes
```

We predict the data on our training data and look at the resulting training error.

```
In [8]: y_predicted = regressor.predict(X_train)
score = mean_absolute_error(y_train, y_predicted)
print(f"The training error of our model is {score:.3f}")
```

The training error of our model is 0.000

The output tells us that the model was able to accurately predict the actual house value. To check the validity of the model's strength, we use the test data, which has not been 'seen' by the model, to predict the house median price. All rights reserved WQU WorldQuant University QQQQ

```
In [9]: y_predicted = regressor.predict(X_test)
score = mean_absolute_error(y_test, y_predicted)
print(f"The testing error of our model is {score:.3f} ")
```

The testing error of our model is 4.756

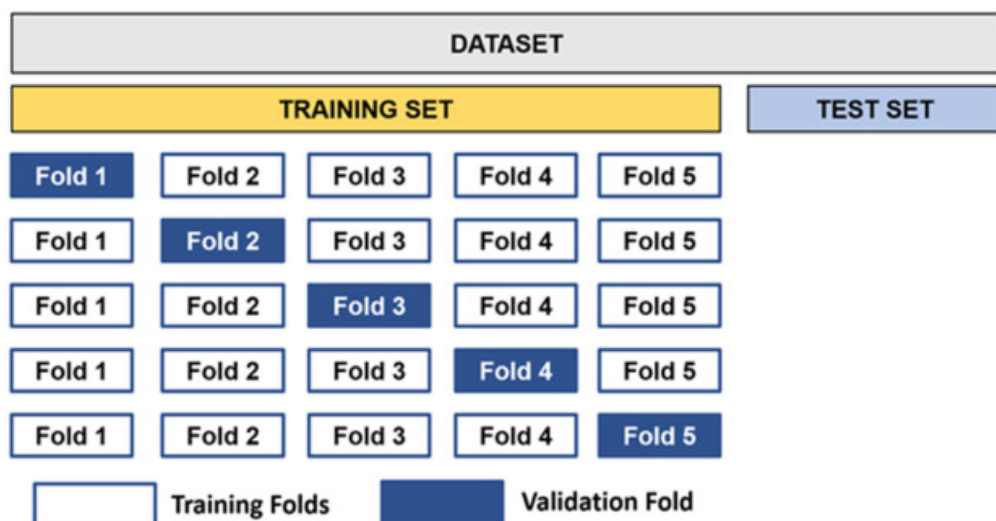
We now see that there is a bigger error on the testing dataset as compared to the training set and hence signs that our model is *overfitting*. To avoid issues with overfitting, we can use cross-validation that splits the data into smaller chunks iteratively so that it covers the entire train data, which is alternatively split into train and validation sets.

1.2 Cross-Validation

In this technique, the train data is split into train and validation sets multiple times, and each split is called a **Fold**. In this subsection, we will discuss the k-fold cross-validation.

K-fold cross-validation splits the data into k segments where $k - 1$ segments are used for training and the remaining segment used for validation as shown in the diagram below.

Fig 1: K-Fold Cross Validation



Source: Srivastava, Amiy, et al. "Ensemble Prediction of Mean Bubble Size in a Continuous Casting Mold Using Data Driven Modeling Techniques." *Machine Learning with Applications*, vol. 6, 2021.

Below are the steps involved in performing a cross-validation:

1. Shuffle the dataset.
2. Hold out the test data.
3. Perform k-fold cross validation on the train dataset.
4. Get the best evaluation score, which is the mean of all the folds.
5. Perform model evaluation on the test data.

Cross-validation allows us to estimate the robustness of our model by splitting the dataset repetitively a given number of times.

The next cells demonstrate how to apply cross-validation on the dataset.

```
In [10]: from sklearn.model_selection import ShuffleSplit, cross_validate

cv = ShuffleSplit(n_splits=40, test_size=0.3, random_state=0)
cv_results = cross_validate(regressor, X, y, cv=cv,
scoring="neg_mean_absolute_error")
```

We convert the `cv_results` Python dictionary into a dataframe for easier visualization.

```
In [11]: import pandas as pd

cv_results = pd.DataFrame(cv_results)
cv_results.head()
```

```
Out[11]:
```

	fit_time	score_time	test_score
0	0.216833	0.006194	-4.691438
1	0.241928	0.010464	-4.703234
2	0.397350	0.006691	-4.721598
3	0.280365	0.006664	-4.466483
4	0.396568	0.018495	-4.767493

We negate the `test_score` to get the actual test errors because the parameter `scoring="neg_mean_absolute_error"` gives negative values of the mean absolute error.

```
In [12]: cv_results["test_error"] = -cv_results["test_score"]
```

```
In [13]: cv_results.head(10)
```

Out[13]:

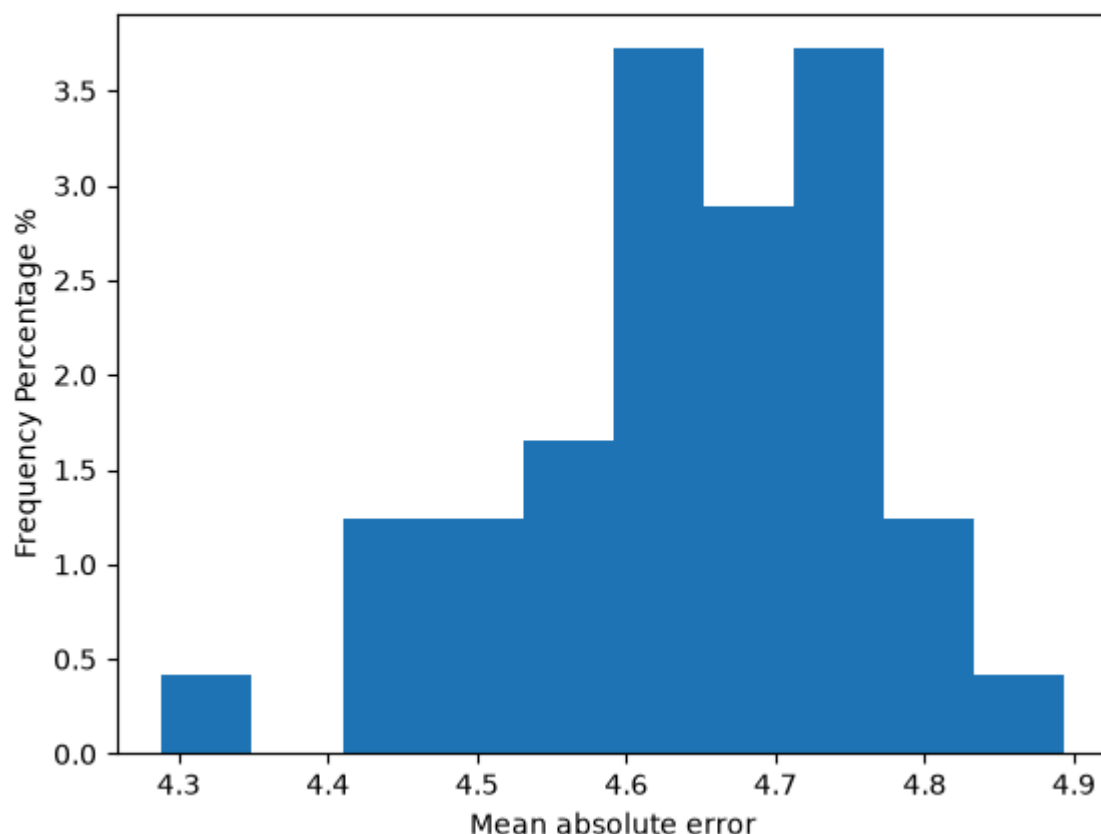
	fit_time	score_time	test_score	test_error
0	0.216833	0.006194	-4.691438	4.691438
1	0.241928	0.010464	-4.703234	4.703234
2	0.397350	0.006691	-4.721598	4.721598
3	0.280365	0.006664	-4.466483	4.466483
4	0.396568	0.018495	-4.767493	4.767493
5	0.520694	0.034121	-4.585270	4.585270
6	0.627572	0.005991	-4.694785	4.694785
7	0.788481	0.023091	-4.634316	4.634316
8	0.688639	0.030022	-4.546915	4.546915
9	0.657277	0.015577	-4.538927	4.538927

The plot below shows how the testing errors are distributed.

```
In [14]: import matplotlib.pyplot as plt

plt.hist(cv_results["test_error"], density=True)
# cv_results["test_error"].plot.hist(bins=10, edgecolor="black")
plt.xlabel("Mean absolute error")
plt.ylabel("Frequency Percentage %")
plt.suptitle(
    "Fig. 2: Test Errors Distribution.", fontweight="bold",
    horizontalalignment="right"
)
plt.show()
```

Fig. 2: Test Errors Distribution.



```
In [15]: print(  
    f"The mean cross-validated testing error is: "  
    f"{cv_results['test_error'].mean():.3f}"  
)
```

The mean cross-validated testing error is: 4.643

The error has reduced when compared to the initial error without using cross-validation.

```
In [16]: print(  
    f"The standard deviation of the testing error is: "  
    f"{cv_results['test_error'].std():.3f}"  
)
```

The standard deviation of the testing error is: 0.118

Cross-validation reduces the error on our model albeit at a smaller fraction as compared to the case when we did not use cross-validation.

A problem tends to occur in our bid to improve model performance: we can find ourselves solving one problem (underfitting) while introducing another problem (overfitting). In the next subsection, let's discuss the overfitting and underfitting concepts.

1.3 Overfitting and Underfitting

When the model memorizes the train data, then we say overfitting has occurred. In such a case, we get a high evaluation score on our train dataset but a not-so-impressive score on the test data set; therefore, the model fails to generalize well on unseen data.

Models that overfit respond to random noise in the training set and thus become sensitive to small fluctuations in the data. Overfitting is a combination of low bias and high variance.

Underfitting on the other hand is characterized by a low performance score on both the training dataset and the validation dataset. Underfitting is caused by simple algorithms that are unable to learn complex patterns existing in our data. Underfitting is also referred to as high bias and often low variance.

A right fit model neither underfits nor overfits, and the model performance in the training set is comparable to the test set. Linear machine learning algorithms such as logistic regression tend to underfit on the dataset while non-linear models like polynomial regression tend to overfit a dataset.

In case our model underfits, to improve its performance, we can

1. Use non-linear algorithms.
2. Increase the complexity of the model by tweaking its parameters.
3. Use non-parameterized algorithms.

To curb overfitting of our models, we

1. Increase the data size used for training to improve the generalizing ability of the model.
2. Use regularization techniques like the L1 and L2 norms.
3. Tune the hyperparameters of our model.
4. Reduce the number of features in our dataset.
5. Reduce the complexity of the model.
6. Apply cross-validation techniques.

Fig 3: Overfitting and Underfitting Examples



Source: [Badillo, Solveig, et al. "An Introduction to Machine Learning." *Clinical Pharmacology & Therapeutics*, vol. 107, no. 4, 2020, pp. 871-885.](#)

In the next cells, we evaluate our model and determine whether it is a good fit, underfitting or overfitting on our dataset.

```
In [17]: import pandas as pd
from sklearn.model_selection import ShuffleSplit, cross_validate

cv = ShuffleSplit(n_splits=30, test_size=0.2)
cv_results = cross_validate(
    regressor,
    X,
    y,
    cv=cv,
    scoring="neg_mean_absolute_error",
    return_train_score=True,
    n_jobs=2,
)
cv_results = pd.DataFrame(cv_results)
```

As discussed in the previous subsection, we negate the scores to get the actual train and test errors.

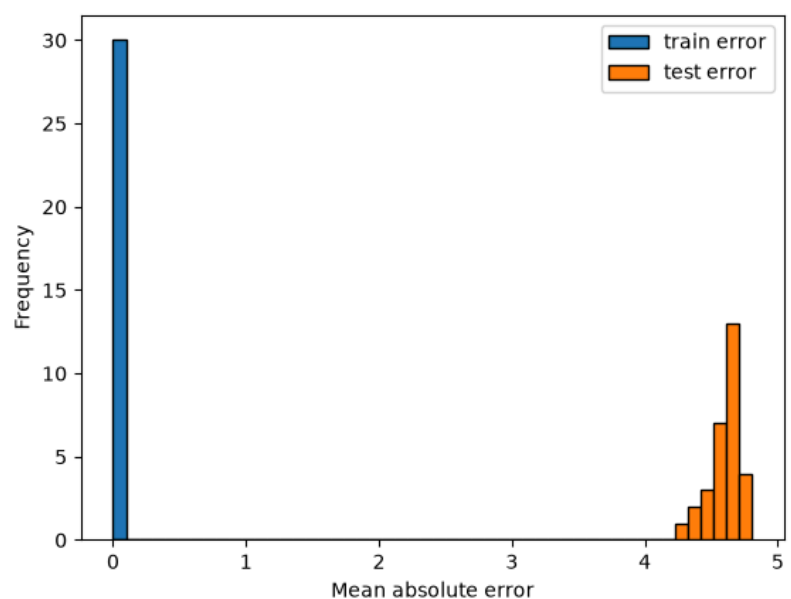
```
In [18]: scores = pd.DataFrame()
scores[["train error", "test error"]] = -cv_results[["train_score",
"test_score"]]
```

The next cell plots the distribution of the errors.

```
In [19]: import matplotlib.pyplot as plt

scores.plot.hist(bins=50, edgecolor="black")
plt.xlabel("Mean absolute error")
plt.suptitle(
    "Fig. 4: Train and Test Errors Distribution via Cross-validation.",
    fontweight="bold",
    horizontalalignment="right",
)
plt.show()
```

Fig. 4: Train and Test Errors Distribution via Cross-validation.



We can observe that the training error is zero, implying that our model is not underfitting. The test error is larger, and therefore, our model seems to have

memorized the noise in our dataset. Hence, its generalization abilities have been compromised.

One of the challenges with hyperparameter tuning is that we can easily shift from a case of a model underfitting into another case of the model overfitting on our dataset. To overcome such challenges, we need to consider **validation curves**.

1.4 Validation Curves

The validation curve varies the values of the hyperparameter(s) and gives the intuition of how the model behaves given different hyperparameter values.

```
In [20]: from sklearn.model_selection import validation_curve

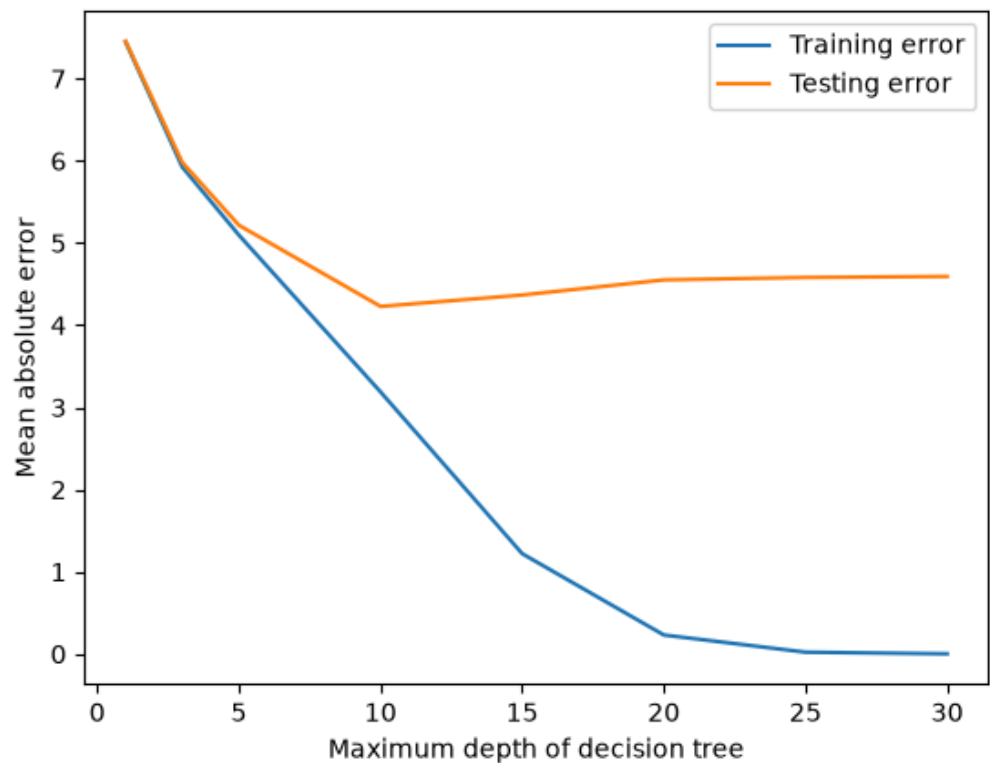
max_depth = [1, 3, 5, 10, 15, 20, 25, 30]
train_scores, test_scores = validation_curve(
    regressor,
    X,
    y,
    param_name="max_depth",
    param_range=max_depth,
    cv=cv,
    scoring="neg_mean_absolute_error",
    n_jobs=2,
)
train_errors, test_errors = -train_scores, -test_scores
```

In the example, we will restrict ourselves to one of the hyperparameters, the `max_depth`.

```
In [21]: plt.plot(max_depth, train_errors.mean(axis=1), label="Training error")
plt.plot(max_depth, test_errors.mean(axis=1), label="Testing error")
plt.legend()

plt.xlabel("Maximum depth of decision tree")
plt.ylabel("Mean absolute error")
plt.suptitle(
    "Fig. 5: Validation Curve for Decision Tree",
    fontweight="bold",
    horizontalalignment="right",
)
plt.show()
```

Fig. 5: Validation Curve for Decision Tree

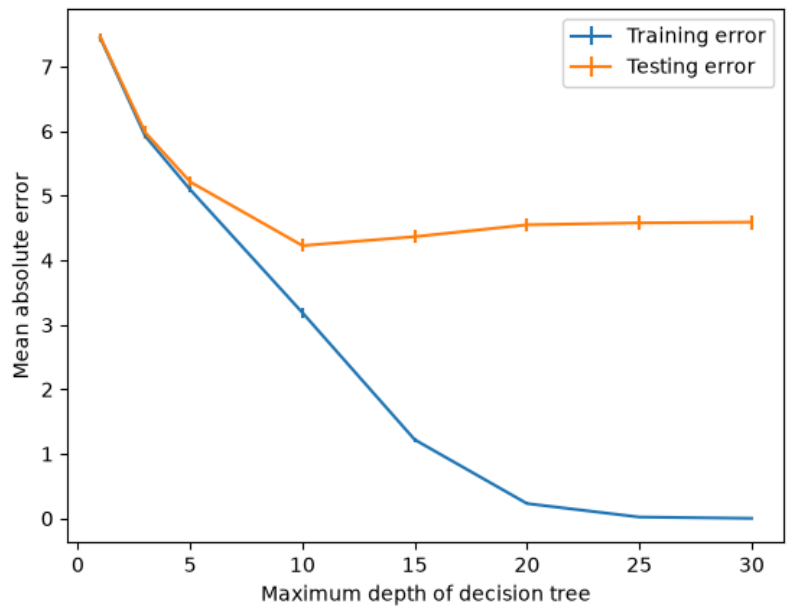


From the above plot, we see that for lower values of maximum depth, the model seems to underfit while as the maximum depth of trees increases, the model overfits on our dataset. So, for the values of 10, we can say that our model will provide a good fit and hence better generalization on an unseen dataset. The plot below repeats the above plot but on training errors.

```
In [22]: plt.errorbar(
    max_depth,
    train_errors.mean(axis=1),
    yerr=train_errors.std(axis=1),
    label="Training error",
)
plt.errorbar(
    max_depth,
    test_errors.mean(axis=1),
    yerr=test_errors.std(axis=1),
    label="Testing error",
)
plt.legend()

plt.xlabel("Maximum depth of decision tree")
plt.ylabel("Mean absolute error")
plt.suptitle(
    "Fig. 6: Validation Curve for Decision Tree using Train Errors",
    fontweight="bold",
    horizontalalignment="right",
)
plt.show()
```

Fig. 6: Validation Curve for Decision Tree using Train Errors



Our target when running machine learning models is to get an unbiased estimate of our errors. This is possible when we understand how the concept of bias-variance tradeoff relates to underfitting and overfitting.

1.5 Bias-Variance Tradeoff

We can divide the prediction errors from machine learning algorithms into two:

- Reducible parts
- Irreducible parts.

The irreducible parts are caused by stochastic noise as a result of missing important features or measurement errors and can't be reduced even with very good models. The reducible part, on the other hand, constitutes errors due to:

- Bias
- Variance

Error due to Bias - Bias is the difference between our predicted values and the actual values we are trying to predict. These errors occur when the algorithm is too simple (less complex) to capture the functional relationship between our features. Models having high bias tend to oversimplify the model by not learning a lot from the training data. This causes the model to make some systematic mistakes, and therefore, the predictions will be biased. This is also referred to as underfitting.

Error due to Variance - We define variance as the measure of spread in our dataset. These errors occur when the algorithm is so complex that it extracts patterns from the noise in the dataset. Models with high variance perform very well on training data but do not generalize well on unseen data. This is what we will refer to as overfitting, as discussed above.

So, we see two issues here:

1. If our model is too simple and has few parameters, then there is a tendency that we risk having a model with high bias and low variance.
2. If our model is too complex with many parameters, then it is highly likely the model will overfit on the dataset due to high variance and low bias on the dataset.

This is what introduces the bias-variance tradeoff since it is impossible to have a model that will be both complex and simple at the same time.

Building accurate models requires understanding these errors so as to avoid our model underfitting or overfitting.

In the next few cells, we will demonstrate the concept of bias and variance tradeoff.

```
In [23]: !pip install mlxtend
```

Requirement already satisfied: mlxtend in /usr/local/lib/python3.12/dist-packages (0.25.0)

Requirement already satisfied: scipy>=1.16.3 in /usr/local/lib/python3.12/dist-packages (from mlxtend) (1.16.3)

Requirement already satisfied: numpy>=2.3.5 in /usr/local/lib/python3.12/dist-packages (from mlxtend) (2.5.1)

Requirement already satisfied: pandas>=2.3.3 in /usr/local/lib/python3.12/dist-packages (from mlxtend) (3.0.3)

Requirement already satisfied: scikit-learn>=1.8.0 in /usr/local/lib/python3.12/dist-packages (from mlxtend) (1.9.0)

Requirement already satisfied: matplotlib>=3.10.8 in /usr/local/lib/python3.12/dist-packages (from mlxtend) (3.11.0)

Requirement already satisfied: joblib>=1.5.2 in /usr/local/lib/python3.12/dist-packages (from mlxtend) (1.5.3)

Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.10.8->mlxtend) (1.3.3)

Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.10.8->mlxtend) (0.12.1)

Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.10.8->mlxtend) (4.63.0)

Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.10.8->mlxtend) (1.5.0)

Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.10.8->mlxtend) (26.2)

Requirement already satisfied: pillow>=9 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.10.8->mlxtend) (11.3.0)

Requirement already satisfied: pyparsing>=3 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.10.8->mlxtend) (3.3.2)

Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.10.8->mlxtend) (2.9.0.post0)

Requirement already satisfied: narwhals>=2.0.1 in /usr/local/lib/python3.12/dist-packages (from scikit-learn>=1.8.0->mlxtend) (2.22.1)

Requirement already satisfied: threadpoolctl>=3.5.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn>=1.8.0->mlxtend) (3.6.0)

Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.7->matplotlib>=3.10.8->mlxtend) (1.17.0)

In [24]: `from mlxtend.evaluate import bias_variance_decomp`

We use the data as above and the same algorithm to demonstrate the concept of bias-variance tradeoff.

In [25]: `from sklearn.datasets import fetch_california_housing`

```
housing = fetch_california_housing(as_frame=True)
X, y = housing.data.values, housing.target.values * 10
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.33, random_state=1
)
```

In the next cell, we will define the hyperparameter that will define an increase in model complexity.

```
In [26]: max_levels = list(range(1, 50))
levels = []
squared_bias_plus_variance = []
for level in max_levels:
    model = DecisionTreeRegressor(max_depth=level)
    model.fit(X_train, y_train)
    mse, bias, var = bias_variance_decomp(
        model,
        X_train,
        y_train,
        X_test,
        y_test,
        loss="mse",
        num_rounds=200,
        random_seed=1,
    )
    score = model.score(X_test, y_test)
    squared_bias_plus_variance.append(bias**2 + var)
    levels.append(level)
```

In the next cell, we plot the bias-variance tradeoff chart that shows how an increase in model complexity affects bias and variance.

```
In [27]: import plotly.graph_objects as go

scatter = go.Scatter(x=levels, y=squared_bias_plus_variance)
layout = go.Layout(
    title="Fig. 7. Bias variance tradeoff",
    xaxis=dict(title="levels"),
    yaxis=dict(title="bias^2+variance"),
)
go.Figure(data=[scatter], layout=layout)
```


It can be seen that as the model complexity increases, the error reduces.

Now that we have a good understanding of how to improve our model performance, we can go ahead and learn how to tweak the hyperparameters of our models.

2. Hyperparameter Tuning

Hyperparameter tuning involves searching over a given set of values and finding the optimal values of a hyperparameter in a machine learning model. We expect that after performing hyperparameter tuning, the result of the model training will give us a better score without overfitting on the dataset.

Now let's highlight the difference between a parameter and a hyperparameter.

While a parameter is learned from the data during model training, a hyperparameter is set by the user before we begin training our model.

An example of a parameter is the coefficients we obtain from training a linear regression model while the number of epochs and batch size are examples of hyperparameters. We should note that some models like linear regression only have parameters while others like the K-Nearest Neighbors (KNN) contain only hyperparameters.

Before commencing on to hyperparameter tuning, we first define the **hyperparameter space**, which is the set of all possible hyperparameter values. Hyperparameters can either be discrete or continuous values. When defining the hyperparameter space, we also define their underlying distribution.

In the next subsections of this lesson, we intend to discuss the various types of hyperparameter tuning.

2.1 Manual Search

The manual search method entails defining the search space and looping all the possible hyperparameters to get a combination that will give us the best outcome. We follow these steps when performing manual search:

1. Split the train data into train and test splits.
2. Define the initial hyperparameter values.
3. Perform k-fold cross validation on the training data.
4. Get the validation score.
5. Choose another set of hyperparameter values.
6. Repeat steps 3-5 until we get satisfactory performance.
7. Train the model on the full train set with the chosen hyperparameter value.
8. Evaluate this performance on the test set.

In the example below, we show how we can get and set the value(s) of model hyperparameters. We will consider the [blood transfusion data](#).

```
In [28]: from sklearn import set_config  
  
set_config(display="diagram")  
  
import pandas as pd
```

We load the data and display the first few rows.

```
In [29]: url = "https://archive.ics.uci.edu/ml/machine-learning-databases/blood-  
transfusion/transfusion.data"  
data_df = pd.read_csv(url)  
data_df.head()
```

Out[29]:

	Recency (months)	Frequency (times)	Monetary (c.c. blood)	Time (months)	whether he/she donated blood in March 2007
0	2	50	12500	98	1
1	0	13	3250	28	1
2	1	16	4000	35	1
3	2	20	5000	45	1
4	1	24	6000	77	0

The next step involves renaming the columns for easier handling.

```
In [30]: cols = [c.lower().split()[0] for c in data_df.columns]
cols[-1] = "class_name"
data_df.columns = cols
```

```
In [31]: data_df.head()
```

Out[31]:

	recency	frequency	monetary	time	class_name
0	2	50	12500	98	1
1	0	13	3250	28	1
2	1	16	4000	35	1
3	2	20	5000	45	1
4	1	24	6000	77	0

We then define the target class that will set the stage for the type of machine learning problem we are supposed to solve. In our case, we are presented with a binary classification problem that we will not delve into too much as we are only interested in learning the hyperparameter tuning techniques.

```
In [32]: target_name = "class_name"
feature_columns = ["recency", "frequency", "monetary"]

target = data_df[target_name]
data = data_df[feature_columns]
```

We transform the data by rescaling it using `StandardScaler` that removes the mean of the feature and scales to a unit variance.

```
In [33]: from sklearn.ensemble import HistGradientBoostingClassifier
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler

model = Pipeline(
    steps=[
        ("preprocessor", StandardScaler()),
        ("classifier", HistGradientBoostingClassifier(random_state=42)),
    ]
)
```

As discussed in the previous section, we evaluate the performance of the model using cross-validation.

```
In [34]: from sklearn.model_selection import cross_validate

cv_results = cross_validate(model, data, target)
scores = cv_results["test_score"]
print(
    f"Accuracy score via cross-validation:\n"
    f"{scores.mean():.3f} ± {scores.std():.3f}"
)
```

```
Accuracy score via cross-validation:
0.738 ± 0.039
```

In the demonstrations that will follow in this lesson, we will try to tune the hyperparameters `learning_rate` and `max_leaf_nodes` of the `HistGradientBoostingClassifier` model. We start first by displaying their default values below.

```
In [35]: print("learning rate default value", model.get_params()
["classifier__learning_rate"])
print("max_leaf_nodes default value", model.get_params()
["classifier__max_leaf_nodes"])
```

```
learning rate default value 0.1
max_leaf_nodes default value 31
```

Without specifying the value of the hyperparameters, the model uses the default parameters, for example, the default value for `C` is 1. We can manually tune the hyperparameter, in this case, change the value of `C` as shown in the cell below.

```
In [36]: model.set_params(classifier__learning_rate=1e-3)
model.set_params(classifier__max_leaf_nodes=20)
cv_results = cross_validate(model, data, target)
scores = cv_results["test_score"]
print(
    f"Model accuracy score with cross-validation:\n"
    f"{scores.mean():.3f} ± {scores.std():.3f}"
)
```

```
Model accuracy score with cross-validation:
0.762 ± 0.002
```

The hyperparameters are stored in `model.get_params` and we can explore them one by one. For demonstration purposes we will consider tuning the

hyperparameter `C` in the cells below.

The cell below displays the values of the hyperparameters we have set above.

```
In [37]: print("learning rate default value", model.get_params()  
["classifier__learning_rate"])  
print("max_leaf_nodes default value", model.get_params()  
["classifier__max_leaf_nodes"])
```

```
learning rate default value 0.001  
max_leaf_nodes default value 20
```

In the next cell, we manually set a couple of values for `C` and choose the one that gives us better performance.

```
In [38]: for learning_rate in [1e-3, 1e-2, 1e-1, 1, 10]:  
        for max_leaf_nodes in [3, 5, 10, 15, 20, 30]:  
            model.set_params(classifier__learning_rate=learning_rate)  
            model.set_params(classifier__max_leaf_nodes=max_leaf_nodes)  
            cv_results = cross_validate(model, data, target)  
            scores = cv_results["test_score"]  
            print(  
                f"Model accuracy score with cross-validation:\n learning_rate=  
{learning_rate} and max_leaf_nodes={max_leaf_nodes}:\n"  
                f"{scores.mean():.3f} ± {scores.std():.3f}"  
            )
```

Model accuracy score with cross-validation:
learning_rate=0.001 and max_leaf_nodes=3:
0.762 $\pm$ 0.002

Model accuracy score with cross-validation:
learning_rate=0.001 and max_leaf_nodes=5:
0.762 $\pm$ 0.002

Model accuracy score with cross-validation:
learning_rate=0.001 and max_leaf_nodes=10:
0.762 $\pm$ 0.002

Model accuracy score with cross-validation:
learning_rate=0.001 and max_leaf_nodes=15:
0.762 $\pm$ 0.002

Model accuracy score with cross-validation:
learning_rate=0.001 and max_leaf_nodes=20:
0.762 $\pm$ 0.002

Model accuracy score with cross-validation:
learning_rate=0.001 and max_leaf_nodes=30:
0.762 $\pm$ 0.002

Model accuracy score with cross-validation:
learning_rate=0.01 and max_leaf_nodes=3:
0.762 $\pm$ 0.002

Model accuracy score with cross-validation:
learning_rate=0.01 and max_leaf_nodes=5:
0.757 $\pm$ 0.012

Model accuracy score with cross-validation:
learning_rate=0.01 and max_leaf_nodes=10:
0.758 $\pm$ 0.013

Model accuracy score with cross-validation:
learning_rate=0.01 and max_leaf_nodes=15:
0.755 $\pm$ 0.012

Model accuracy score with cross-validation:
learning_rate=0.01 and max_leaf_nodes=20:
0.755 $\pm$ 0.012

Model accuracy score with cross-validation:
learning_rate=0.01 and max_leaf_nodes=30:
0.755 $\pm$ 0.012

Model accuracy score with cross-validation:
learning_rate=0.1 and max_leaf_nodes=3:
0.733 $\pm$ 0.058

Model accuracy score with cross-validation:
learning_rate=0.1 and max_leaf_nodes=5:
0.739 $\pm$ 0.036

Model accuracy score with cross-validation:
learning_rate=0.1 and max_leaf_nodes=10:
0.741 $\pm$ 0.033

Model accuracy score with cross-validation:
learning_rate=0.1 and max_leaf_nodes=15:
0.734 $\pm$ 0.032

Model accuracy score with cross-validation:
learning_rate=0.1 and max_leaf_nodes=20:
0.737 $\pm$ 0.038

Model accuracy score with cross-validation:
learning_rate=0.1 and max_leaf_nodes=30:
0.738 $\pm$ 0.039

Model accuracy score with cross-validation:
learning_rate=1 and max_leaf_nodes=3:
0.734 $\pm$ 0.036

Model accuracy score with cross-validation:
learning_rate=1 and max_leaf_nodes=5:
0.735 $\pm$ 0.039

```

Model accuracy score with cross-validation:
  learning_rate=1 and max_leaf_nodes=10:
0.726 ± 0.041
Model accuracy score with cross-validation:
  learning_rate=1 and max_leaf_nodes=15:
0.726 ± 0.035
Model accuracy score with cross-validation:
  learning_rate=1 and max_leaf_nodes=20:
0.727 ± 0.037
Model accuracy score with cross-validation:
  learning_rate=1 and max_leaf_nodes=30:
0.729 ± 0.036
Model accuracy score with cross-validation:
  learning_rate=10 and max_leaf_nodes=3:
0.413 ± 0.248
Model accuracy score with cross-validation:
  learning_rate=10 and max_leaf_nodes=5:
0.466 ± 0.242
Model accuracy score with cross-validation:
  learning_rate=10 and max_leaf_nodes=10:
0.592 ± 0.229
Model accuracy score with cross-validation:
  learning_rate=10 and max_leaf_nodes=15:
0.644 ± 0.143
Model accuracy score with cross-validation:
  learning_rate=10 and max_leaf_nodes=20:
0.594 ± 0.148
Model accuracy score with cross-validation:
  learning_rate=10 and max_leaf_nodes=30:
0.594 ± 0.148

```

From the cell above, we see that the value of `C=0.01` gives better accuracy.

2.2 Grid Search

Given a hyperparameter space, grid search looks for all the possible combinations that improve the performance of the model. In this method, the underlying distribution of the hyperparameters need not be known.

So, the steps involved in grid search are as follows.

1. Split the data into train and test sets.
2. Define the hyperparameter space.
3. Construct a nested loop of the number of hyperparameters.
4. Perform cross validation on the train set and store the resulting score in each loop.
5. Train the model on the train set and evaluate it on the test set.

The output of the grid search will be equal to the output of the manual search since the model is evaluated at each pair of hyperparameters in our grid.

Just like in the manual tuning section above, we demonstrate how to apply grid search in hyperparameter tuning.

We start by splitting our dataset into training and testing sets.

```
In [39]: from sklearn.model_selection import train_test_split

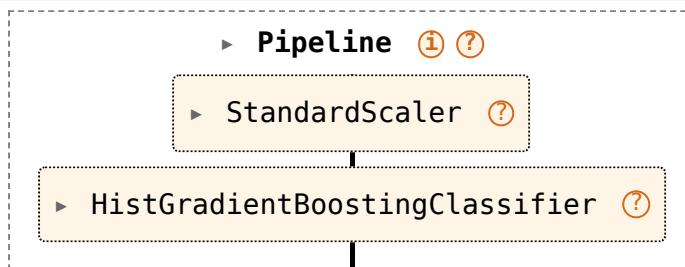
X_train, X_test, y_train, y_test = train_test_split(data, target,
random_state=42)
```

We use the histogram gradient classifier model, which is a tree-based classifier.

```
In [40]: from sklearn.ensemble import HistGradientBoostingClassifier

model = Pipeline(
    [
        ("preprocessor", StandardScaler()),
        ("classifier", HistGradientBoostingClassifier(random_state=42)),
    ]
)
model
```

Out[40]:

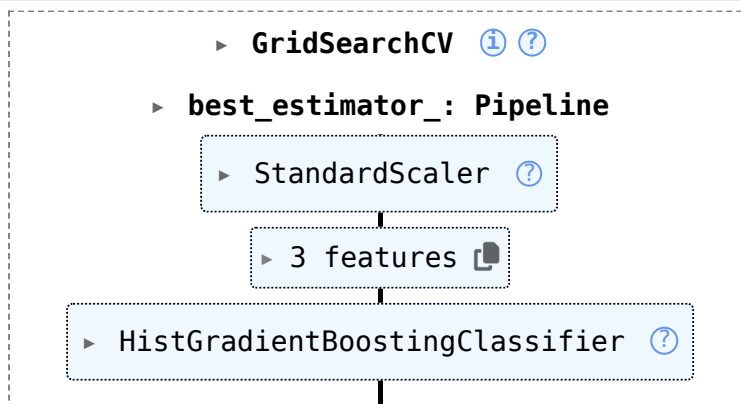


Grid search is costly as it finds a combination of all the hyperparameters in our hyperparameter space. In this example, we share only limited nodes, and we choose to select only two hyperparameters to tune.

```
In [41]: from sklearn.model_selection import GridSearchCV

param_grid = {
    "classifier__learning_rate": (0.001, 0.01, 0.1, 1, 10),
    "classifier__max_leaf_nodes": (3, 5, 10, 15, 20, 30),
}
model_grid_search = GridSearchCV(model, param_grid=param_grid, n_jobs=2,
cv=2)
model_grid_search.fit(X_train, y_train)
```

Out[41]:



We then evaluate the accuracy of our model using the test set.

```
In [42]: accuracy = model_grid_search.score(X_test, y_test)
print(f"The test accuracy score of the grid-searched pipeline is: " f"
{accuracy:.3f}")
```


The test accuracy score of the grid-searched pipeline is: 0.754

The grid search takes the hyperparameter space with all the hyperparameter values in it and finds all the possible combinations of the hyperparameters. Like in our case where we have four learning_rate and max_leaf_nodes values, there will be $4 \times 4 = 16$ combination of hyperparameters, and we select the pair that gives us the best result. This makes the process expensive because for any value we add, the process becomes more and more computationally unsustainable. We can actually see that Grid Search is similar to the manual search we saw in the previous subsection.

From the selected hyperparameters, we can now predict the first five targets.

```
In [43]: model_grid_search.predict(X_test.iloc[0:5])
```

```
Out[43]: array([0, 0, 0, 0, 0])
```

We now visualize the original values in the cell below.

```
In [44]: y_test.iloc[0:5].values
```

```
Out[44]: array([0, 0, 0, 0, 0])
```

By using the `best_params_` attribute, we can see the selected hyperparameters that gives us the highest accuracy.

```
In [45]: print(f"The best set of parameters is: " f"
           {model_grid_search.best_params_}")
```

```
The best set of parameters is: {'classifier__learning_rate': 0.1,
'classifier__max_leaf_nodes': 10}
```

We inspect all the values stored in `cv_results_` using a dataframe as shown below.

```
In [46]: cv_results = pd.DataFrame(model_grid_search.cv_results_).sort_values(
           "mean_test_score", ascending=False
         )
           cv_results.head()
```

Out[46]:

	mean_fit_time	std_fit_time	mean_score_time	std_score_time	param_classifier_learning_rate	param_classifier_max_leaf_nodes	params	split0_test_score	split1_test_score	mean_test_score	std_test_score	rank_test_score
14	0.119287	0.000476	0.008576	0.000223	0.100	10	{'classifier_learning_rate': 0.1, 'classifier...	0.754448	0.785714	0.770081	0.015633	1
13	0.079534	0.002537	0.008313	0.000270	0.100	5	{'classifier_learning_rate': 0.1, 'classifier...	0.750890	0.785714	0.768302	0.017412	2
12	0.060934	0.001282	0.008104	0.000275	0.100	3	{'classifier_learning_rate': 0.1, 'classifier...	0.750890	0.785714	0.768302	0.017412	2
18	0.108400	0.006500	0.008035	0.000091	1.000	3	{'classifier_learning_rate': 1, 'classifier...	0.754448	0.782143	0.768296	0.013847	4
2	0.114330	0.000946	0.008152	0.000099	0.001	10	{'classifier_learning_rate': 0.001, 'classifier...	0.768683	0.767857	0.768270	0.000413	5

We shorten the headers to make the dataframe more readable.

```
In [47]: # get the parameter names
column_results = [f"param_{name}" for name in param_grid.keys()]
column_results += ["mean_test_score", "std_test_score", "rank_test_score"]
cv_results = cv_results[column_results]
```

```
In [48]: def shorten_param(param_name):
    if "__" in param_name:
        return param_name.rsplit("__", 1)[1]
    return param_name

cv_results = cv_results.rename(shorten_param, axis=1)
cv_results.head()
```

```
Out[48]:
```

	learning_rate	max_leaf_nodes	mean_test_score	std_test_score	rank_test_score
14	0.100	10	0.770081	0.015633	1
13	0.100	5	0.768302	0.017412	2
12	0.100	3	0.768302	0.017412	2
18	1.000	3	0.768296	0.013847	4
2	0.001	10	0.768270	0.000413	5

Since we are only interested in our two hyperparameters, the table below displays the model accuracy for each pair.

```
In [49]: pivoted_cv_results = cv_results.pivot_table(
    values="mean_test_score", index=["learning_rate"], columns=
    ["max_leaf_nodes"]
)

pivoted_cv_results
```

```
Out[49]:
```

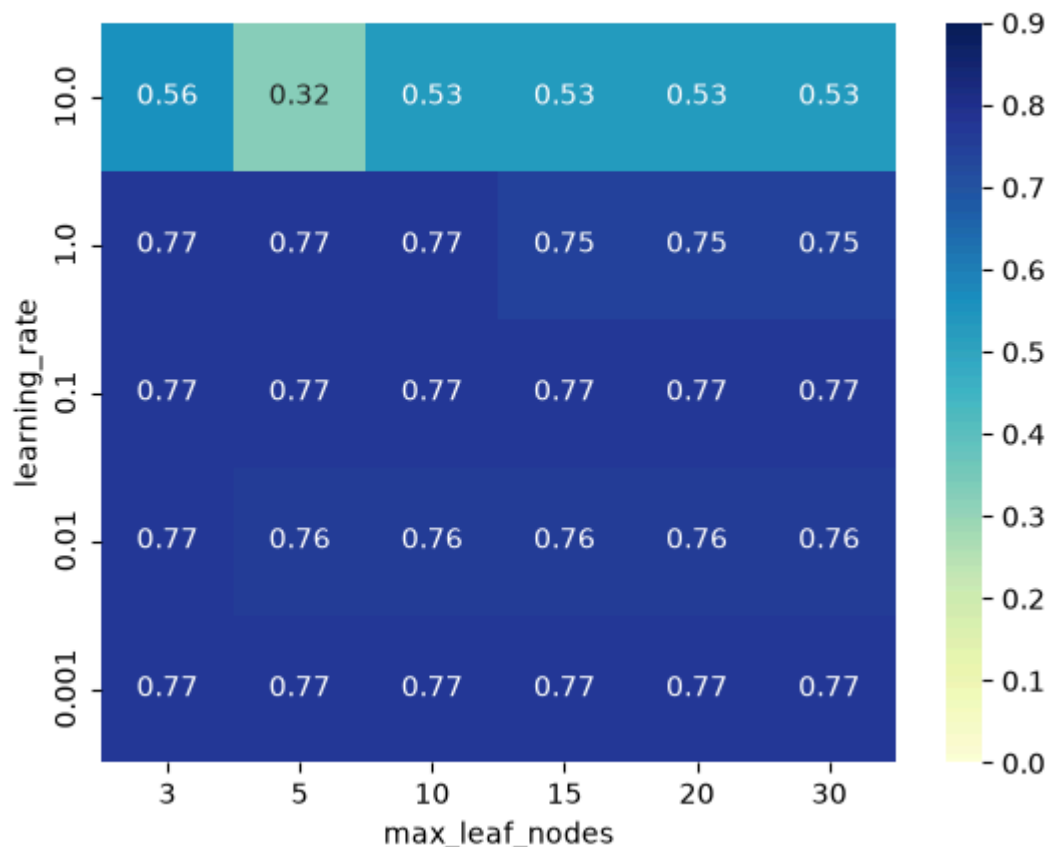
	max_leaf_nodes	3	5	10	15	20	30
learning_rate							
0.001	0.768270	0.768270	0.768270	0.768270	0.768270	0.768270	0.768270
0.010	0.768270	0.764711	0.764711	0.764711	0.764711	0.764711	0.764711
0.100	0.768302	0.768302	0.770081	0.766523	0.766523	0.766523	0.766523
1.000	0.768296	0.766510	0.766510	0.748653	0.748653	0.748653	0.748653
10.000	0.559850	0.324504	0.531031	0.531031	0.531031	0.531031	0.531031

We can visualize the table above in a heatmap as shown in the next cell.

```
In [50]: import seaborn as sns

ax = sns.heatmap(pivoted_cv_results, annot=True, cmap="YlGnBu", vmin=0.0,
vmax=0.9)
ax.invert_yaxis()
plt.suptitle(
    "Fig. 8: Heatmap of the Model Results",
    fontweight="bold",
    horizontalalignment="right",
)
plt.show()
```

Fig. 8: Heatmap of the Model Results



The biggest challenge we have with the two methods we have seen above is that they are very computationally intensive especially when our hyperparameter space is large. In the next subsection, we will discuss the random search method that tries to solve this problem.

2.3 Random Search

This method randomly selects the hyperparameter for each iteration. It outperforms the grid search when the data is small or we have no idea of the range of hyperparameter space. It is also much more efficient when compared to grid search. In random search, the hyperparameter distribution must be specified together with the hyperparameter space, which is wider as compared to the grid search case.

The steps involved in random search are:

1. Split the data into train and test sets.
2. Define the number of trials and set a random seed.
3. Define the hyperparameter space with its distribution.
4. Define an iterator consisting of random hyperparameter combinations.
5. Loop through the iterator.
6. After getting the best possible combination of hyperparameters, we train the model on the full train set and evaluate the model performance.

In the following cells, we demonstrate the random search method for tuning the hyperparameters on our blood transfusion data.

```
In [51]: model = Pipeline(  
    [  
        ("preprocessor", StandardScaler()),  
        ("classifier", HistGradientBoostingClassifier(random_state=42)),  
    ]  
)
```

We can view the default hyperparameter values in the cell below.

```
In [52]: from pprint import pprint  
  
print("Default parameters:\n")  
pprint(model.get_params())
```

Default parameters:

```
{'classifier': HistGradientBoostingClassifier(random_state=42),
 'classifier__categorical_features': 'from_dtype',
 'classifier__class_weight': None,
 'classifier__early_stopping': 'auto',
 'classifier__interaction_cst': None,
 'classifier__l2_regularization': 0.0,
 'classifier__learning_rate': 0.1,
 'classifier__loss': 'log_loss',
 'classifier__max_bins': 255,
 'classifier__max_depth': None,
 'classifier__max_features': 1.0,
 'classifier__max_iter': 100,
 'classifier__max_leaf_nodes': 31,
 'classifier__min_samples_leaf': 20,
 'classifier__monotonic_cst': None,
 'classifier__n_iter_no_change': 10,
 'classifier__random_state': 42,
 'classifier__scoring': 'loss',
 'classifier__tol': 1e-07,
 'classifier__validation_fraction': 0.1,
 'classifier__verbose': 0,
 'classifier__warm_start': False,
 'memory': None,
 'preprocessor': StandardScaler(),
 'preprocessor__copy': True,
 'preprocessor__with_mean': True,
 'preprocessor__with_std': True,
 'steps': [('preprocessor', StandardScaler()),
           ('classifier', HistGradientBoostingClassifier(random_state=42))],
 'transform_input': None,
 'verbose': False}
```

The next cell defines the hyperparameter search space.

```
In [53]: from scipy.stats import loguniform

class loguniform_int:
    """Integer valued version of the log-uniform distribution"""

    def __init__(self, a, b):
        self._distribution = loguniform(a, b)

    def rvs(self, *args, **kwargs):
        """Random variable sample"""
        return self._distribution.rvs(*args, **kwargs).astype(int)

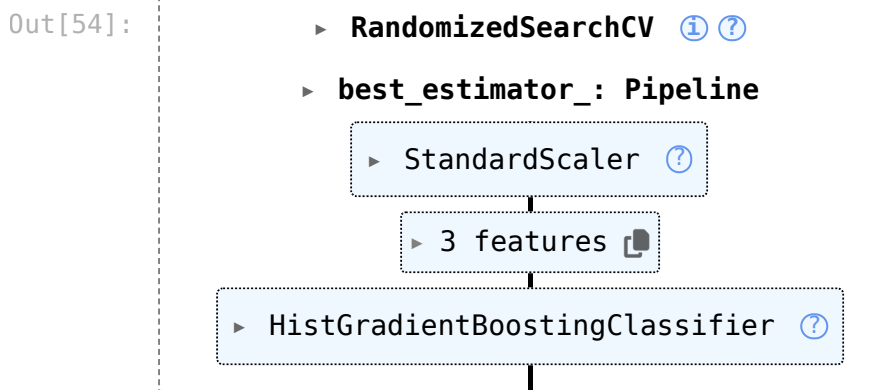
from sklearn.model_selection import RandomizedSearchCV

param_distributions = {
    "classifier__learning_rate": loguniform(1e-6, 1e3),
    "classifier__max_leaf_nodes": loguniform_int(3, 100),
}
```

We can now train the model on our training data using cross-validation that executes 100 iterations of 5 folds.

```
In [54]: model_random_search = RandomizedSearchCV(
    model,
    param_distributions=param_distributions,
    n_iter=10,
    cv=5,
    verbose=1,
    n_jobs=-1,
    return_train_score=True,
)
model_random_search.fit(X_train, y_train)
```

Fitting 5 folds for each of 10 candidates, totalling 50 fits



The random search method for hyperparameter tuning selects the hyperparameter combinations randomly as opposed to grid search where each value in our hyperparameter space was used explicitly. In random search, the choice of one hyperparameter value is independent from the other ones. As opposed to grid search, where our hyperparameter space consisted of hyperparameter values, random search allows the use of sampling distributions.

The next cell displays the best parameters from our model.

```
In [55]: model_random_search.best_params_
```

```
Out[55]: {'classifier__learning_rate': np.float64(0.007337658783361256),
          'classifier__max_leaf_nodes': np.int64(12)}
```

We evaluate our model performance on the test dataset; however, we do not expect good results as the hyperparameters were arbitrarily chosen for the purposes of demonstrating the random search method.

```
In [56]: accuracy = model_random_search.score(X_test, y_test)

print(f"The test accuracy score of the best model is " f"{accuracy:.3f}")
```

The test accuracy score of the best model is 0.743

Just like the grid search process above, we inspect the cv results in the next cell.

```
In [57]: cv_results_rs = pd.DataFrame(model_random_search.cv_results_).sort_values(
    "mean_test_score", ascending=False
)
cv_results_rs.head()
```

Out[57]:

	mean_fit_time	std_fit_time	mean_score_time	std_score_time	param_classification_max_leaf_nodes	param_s	splittest_score	splittest_score	splittest_score	..	mean_test_score	std_test_score	rank_test_score	splittest_rank_score	splittest_rank_score	splittest_rank_score	splittest_rank_score	mean_test_rank_score	std_test_rank_score
1	0.240476	0.000000	0.000000	0.000000	12	{'classification_max_leaf_nodes': 0.000733765878336..	0.76912	0.7677	0.7657	...	0.76828	0.00000	1	0.7677	0.7683	0.7677	0.7674	0.7671	0.7607
2	0.21613	0.00000	0.00000	0.00000	12	{'classification_max_leaf_nodes': 0.000733765878336..	0.76769	0.7666	0.7667	...	0.7668	0.00000	1	0.7677	0.7676	0.7677	0.7677	0.7677	0.7600

	7 2	6 9	1 2	1 7	8 7		fi r_ l_ e a r ni n g_ r_ at e': : 8. 7 0 8 1 5 3 4 7 7 8 9 0 6 4. ..	1 2	5 7	5 7		6 8	2 2		5 7	7 4	7 4	7 4	7 4	7 1	0 7
8	0. 0 6 2 3 4 0	0. 0 0 2 4 2	0. 0 0 7 0 3	0. 0 0 0 6 2	0. 0 0 2 4 3	3	{' cl a s si fi r_ l_ e a r ni n g_ r_ at e': : 0. 0 0 2 1 4 2 9 7 5 5 0 7 2 7. ..	0. 7 6 9 1 2	0. 7 6 7 8 5 7	0. 7 6 7 8 5 7	...	0. 7 6 8 2 6 8	0. 0 0 8 2 2	1	0. 7 6 7 8 5 7	0. 7 6 8 3 7 4	0. 7 6 8 3 7 4	0. 7 6 8 3 7 4	0. 7 6 8 3 7 4	0. 7 6 8 2 7 1	0. 0 0 2 2 9 7 5 5 0 7 2 7. ..

[illegible]

4
7.
..

5 rows × 22 columns

```
In [58]: def shorten_param(param_name):  
          if "__" in param_name:  
              return param_name.rsplit("__", 1)[1]  
          return param_name  
  
cv_results_rs = cv_results_rs.rename(shorten_param, axis=1)  
cv_results_rs.head()
```

Out[58]:

	mean_fit_time	std_fit_time	mean_score_time	std_score_time	learn_rate	max_leaf_nodes	params	splitt_0_score	splitt_1_score	splitt_2_score	..	mean_test_score	std_test_score	rank_test_score	splitt_0_train_score	splitt_1_train_score	splitt_2_train_score	splitt_3_train_score	splitt_4_train_score	mean_train_score	std_train_score
1	0.204076	0.00457	0.00180	0.00002	0.00007	12	{'classifiers': 'Sieve', 'r_learning_rate': 0.00733765878336..	0.79912	0.7677	0.7677	...	0.76828	0.00008	1	0.776785	0.77837	0.77744	0.77744	0.77744	0.771	0.700
2	0.216372	0.00431	0.00242	0.00004	0.00007	12	{'classifiers': 'Sieve', 'r_learning_rate': 0.00733765878336..	0.79912	0.7677	0.7677	...	0.76828	0.00008	1	0.776785	0.77837	0.77744	0.77744	0.77744	0.771	0.700

[illegible]

std_train_score	mean_train_score	split_4_train_score	split_3_train_score	split_2_train_score	split_1_train_score	split_0_train_score	rank_test_score	std_test_score	mean_test_score	..	split_2_test_score	split_1_test_score	split_0_test_score	params	max_leaf_nodes	learning_rate	std_fit_time	mean_fit_time
0.0002	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	1	0.0000	0.0000	...	0.0000	0.0000	0.0000	{'classifiers': 'tree', 'learning_rate': 0.00033471780506546}	23	0.0000	0.0000	0.0000
0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	1	0.0000	0.0000	...	0.0000	0.0000	0.0000	{'classifiers': 'tree', 'learning_rate': 0.00033471780506546}	12	0.0000	0.0000	0.0000

mean_fit_time	std_fit_time	mean_score_time	std_score_time	learn_rate	max_leaf_nodes	parallelism	splitt_0_test_score	splitt_1_test_score	splitt_2_test_score	..	mean_test_score	std_test_score	rank_test_score	splitt_0_train_score	splitt_1_train_score	splitt_2_train_score	splitt_3_train_score	splitt_4_train_score	mean_test_score	std_test_score
---------------	--------------	-----------------	----------------	------------	----------------	-------------	---------------------	---------------------	---------------------	----	-----------------	----------------	-----------------	----------------------	----------------------	----------------------	----------------------	----------------------	-----------------	----------------

g
r
a
t
e
:
0.
0
0
1
5
0
3
9
1
9
9
5
7
4
7.
..

5 rows × 22 columns

The table below displays the pairs that the random search tuning picked by the random search algorithm. Since `n_iter=10`, random search will pick 10 pairs at random.

```
In [59]: pivoted_cv_results_rs = cv_results_rs.pivot_table(
          values="mean_test_score", index=["learning_rate"], columns=
          ["max_leaf_nodes"]
        )

pivoted_cv_results_rs
```

Out[59]:	max_leaf_nodes	3	12	14	23	51	74	75	87
	learning_rate								
	0.000003	NaN	NaN	NaN	0.768268	NaN	NaN	NaN	NaN
	0.000011	NaN	NaN	NaN	NaN	0.768268	NaN	NaN	NaN
	0.000041	NaN	NaN	NaN	NaN	NaN	0.768268	NaN	NaN
	0.000087	NaN	0.768268	NaN	NaN	NaN	NaN	NaN	NaN
	0.001504	NaN	0.768268	NaN	NaN	NaN	NaN	NaN	NaN
	0.002143	0.768268	NaN	NaN	NaN	NaN	NaN	NaN	NaN
	0.007338	NaN	0.768268	NaN	NaN	NaN	NaN	NaN	NaN
	224.395295	NaN	NaN	NaN	NaN	NaN	NaN	0.616735	NaN
	521.577177	NaN	NaN	NaN	NaN	NaN	NaN	NaN	0.609592
	595.088035	NaN	NaN	0.661378	NaN	NaN	NaN	NaN	NaN

We can observe that hyperparameter tuning using random search does not evaluate the model at all pairs but randomly selects the best possible combination among pairs. Since random search does not pick all possible pairs, it tends to be faster than manual search and grid search, and its accuracy is nearly equal to grid search.

2.4 Bayesian Optimization

Bayesian optimization improves search space based on what it learns from the previous iterations. In this kind of hyperparameter selection, we first build the probability model of the objective function and then choose the optimal hyperparameters.

There are several techniques of the Bayesian Optimization listed below.

- Gaussian Process (GP).
- Sequential model-based optimization
- Tree Parzen Estimator (TPE)

Bayesian optimization memorizes the model's past performance, which it uses to find the probability of a score given a parameter, that is,

$P(\text{score}|\text{hyperparameters})$.

The model can be optimized easily and used to find the next hyperparameter(s).

The steps of a Bayesian optimization are:

1. Find the probability model of a score given a hyperparameter.
2. Find the optimal hyperparameters of the model in (1).
3. Plug the hyperparameters into the objective function.
4. Update the probability model given the new results.
5. Repeat steps to 1 to 4 until we get a satisfactory result.

We will discuss more on Bayesian optimization in lesson 4.

3. Conclusion

In this lesson, we have seen the various techniques for improving model performance and the impact of hyperparameters on model performance. We have also discussed how to automate the search for hyperparameters using grid search and randomized search. In the next lesson, we will apply these methods to a real-world example.

References

1. Badillo, Solveig, et al. "An Introduction to Machine Learning." *Clinical Pharmacology & Therapeutics*, vol. 107, no. 4, 2020, pp. 871-885.
2. Srivastava, Amiy, et al. "Ensemble Prediction of Mean Bubble Size in a Continuous Casting Mold Using Data Driven Modeling Techniques." *Machine Learning with Applications*, vol. 6, 2021.
3. Yang, Li, and Abdallah Shami. "On Hyperparameter Optimization of Machine Learning Algorithms: Theory and Practice." *Neurocomputing*, vol. 415, 2020, pp. 295-316.

Copyright 2025 WorldQuant University. This content is licensed solely for personal use. Redistribution or publication of this material is strictly prohibited.

Required Readings

Required Readings for this program are open access, which means you should be able to access them at no cost. The link provided in the citation will take you directly to the reading or to a page where you can download it.

1. Ho, Winky K.O, et al. "Predicting Property Prices with Machine Learning Algorithms." *Journal of Property Research*, vol. 38, no. 1, 2021, <https://www.tandfonline.com/doi/full/10.1080/09599916.2020.1832558>.

- **Required pages:** Section 3
- **Estimated reading time:** 60 minutes

MACHINE LEARNING IN FINANCE

MODULE 7 | LESSON 2

HOUSE PREDICTION AND HYPERPARAMETER TUNING

Reading Time	35 minutes
Prior Knowledge	Grid Search, Random Search
Keywords	Grid search

In the last lesson, we introduced hyperparameter tuning in machine learning and the merits and demerits of the various algorithms used. In this lesson, we will apply the skills learned to a house prediction exercise.

1. Introduction

In this lesson we will use the housing data from the [Geo Data and Lab](#) website to predict the housing prices using Random Forest Regression and later on fine-tune its hyperparameters so as to improve the model performance.

We begin by first loading the relevant packages below.

```
In [1]: # data manipulation and plotting
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

# from Scikit-learn
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
```

We then load the downloaded file to our working space and read it.

```
In [2]: df = pd.read_csv("kc_house_data.csv")
```

```
In [3]: df.head()
```

Out[3]:

	id	date	price	bedrooms	bathrooms	sqt_living	sqt_lot	floors	waterfront	view	..	grade	sqt_above	sqt_basement	y_rbuilt	y_renovated	zipcode	lat	long	sqt_living15	sqt_lot15
0	7129300520	2014100000	229000	3	1.00	1180	5650	1.0	0	0	...	7	1180	0	1955	0	98178	47.512	-122.257	1340	5650
1	641410192	2014100000	538000	3	2.25	2570	7242	2.0	0	0	...	7	2170	400	1951	1991	98125	47.7210	-122.319	1690	7639
2	563150400	20150225T000000	180000	2	1.00	770	10000	1.0	0	0	...	6	770	0	1933	0	98028	47.739	-122.233	2720	8062
3	24872000	2018140200	604000	4	3.00	1960	5000	1.0	0	0	...	7	1050	910	1965	0	98136	47.5208	-122.393	1360	5000

id	date	price	bedrooms	bathrooms	sqft_living	sqft_lot	floors	waterfront	view	..	grade	sqft_above	sqft_basement	yrbuilt	yrenovated	zipcode	lat	long	sqft_living15	sqft_lot15	
875	9T00000																				
4	1954400510	20150008T0000000	510000	3	2.000	1680	880	1.0	0	0	...	8	1680	0	1987	0	98074	47.6168	-122.045	1800	7503

5 rows × 21 columns

Since the data is already processed, we will lightly touch on the exploratory data analysis phase and concentrate mostly on the modeling phase.

Below we can see the size of the data and examine missing values.

2. Exploratory Data Analysis

```
In [4]: print(f"The dataset contains {df.shape[0]} samples and " f"{df.shape[1]} features")
```

The dataset contains 21613 samples and 21 features

```
In [5]: df.isnull().sum()
```

```
Out[5]: id          0
        date        0
        price       0
        bedrooms    0
        bathrooms   0
        sqft_living  0
        sqft_lot     0
        floors       0
        waterfront  0
        view         0
        condition   0
        grade        0
        sqft_above   0
        sqft_basement 0
        yr_built     0
        yr_renovated 0
        zipcode      0
        lat          0
        long         0
        sqft_living15 0
        sqft_lot15   0
        dtype: int64
```

We then drop the id and date column from our data as they will not be useful for our prediction exercise. We also separate the target variable from the predictor variables.

```
In [6]: X = df.drop(["id", "price", "date"], axis=1)
        y = df["price"]
```

The next step is to pick out categorical variables from numerical variables using column selector from `sklearn`'s `compose`.

```
In [7]: from sklearn.compose import make_column_selector as selector

        categorical_columns_selector = selector(dtype_include=object)
        categorical_columns = categorical_columns_selector(X)
        categorical_columns
```

```
Out[7]: []
```

```
In [8]: # now let's identify the numerical variables

        num_vars = [var for var in X.columns if var not in categorical_columns]

        # number of numerical variables
        print(len(num_vars))
        print(num_vars)
```

```
18
```

```
['bedrooms', 'bathrooms', 'sqft_living', 'sqft_lot', 'floors', 'waterfront',
'view', 'condition', 'grade', 'sqft_above', 'sqft_basement', 'yr_built',
'yr_renovated', 'zipcode', 'lat', 'long', 'sqft_living15', 'sqft_lot15']
```

We note that some categorical variables are numerical and we can put them into their correct list.

```
In [9]: # let's make a list of discrete variables
categorical_vars = [var for var in num_vars if len(X[var].unique()) < 20]

print("Number of categorical variables: ", len(categorical_vars))
print(categorical_vars)
```

Number of categorical variables: 6
['bedrooms', 'floors', 'waterfront', 'view', 'condition', 'grade']

```
In [10]: # let's visualize the categorical variables

X[categorical_vars].head()
```

```
Out[10]:
```

	bedroom s	floors	waterfro nt	view	condition	grade
0	3	1.0	0	0	3	7
1	3	2.0	0	0	3	7
2	2	1.0	0	0	3	6
3	4	1.0	0	0	5	7
4	3	1.0	0	0	3	8

Therefore, the remaining numerical variables will now be classified as continuous variables.

```
In [11]: # make list of continuous variables
cont_vars = [var for var in num_vars if var not in categorical_vars]

print("Number of continuous variables: ", len(cont_vars))
print(cont_vars)
```

Number of continuous variables: 12
['bathrooms', 'sqft_living', 'sqft_lot', 'sqft_above', 'sqft_basement',
'yr_built', 'yr_renovated', 'zipcode', 'lat', 'long', 'sqft_living15',
'sqft_lot15']

```
In [12]: # let's visualize the continuous variables

X[cont_vars].head()
```

Out[12]:

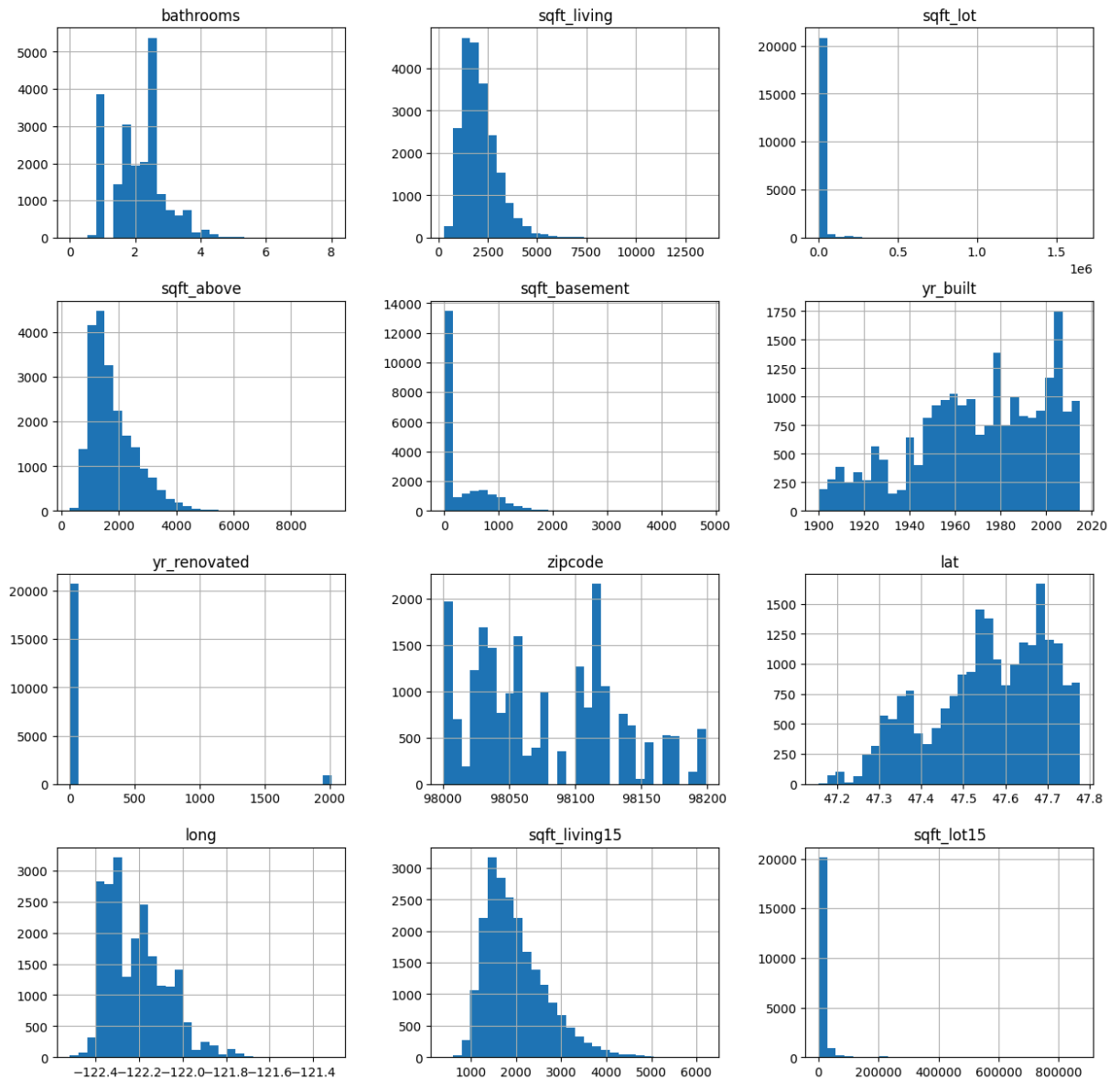
	bat hro ms	sqft _livi ng	sqft _lot	sqft _ab ove	sqft _ba sement	yr_b uilt	yr_r eno vated	zipc ode	lat	lon g	sqft _livi ng1 5	sqft _lot 15
0	1.00	118 0	565 0	118 0	0	195 5	0	981 78	47.5 112	-122 .257	134 0	565 0
1	2.25	257 0	724 2	217 0	400	195 1	199 1	981 25	47.7 210	-122 .319	169 0	763 9
2	1.00	770	100 00	770	0	193 3	0	980 28	47.7 379	-122 .233	272 0	806 2
3	3.00	196 0	500 0	105 0	910	196 5	0	981 36	47.5 208	-122 .393	136 0	500 0
4	2.00	168 0	808 0	168 0	0	198 7	0	980 74	47.6 168	-122 .045	180 0	750 3

We now visualize the variables' distribution using histograms as shown below.All rights reserved WQU WorldQuant University QQQQ

In [13]: *# lets plot histograms for all continuous variables*

```
X[cont_vars].hist(bins=30, figsize=(15, 15))
plt.suptitle("Fig. 1: Histogram Plot", fontweight="bold",
horizontalalignment="right")
plt.show()
```


Fig. 1: Histogram Plot



We can now standardize the continuous variables and encode the discrete variables to eliminate possible bias towards larger values.

3. Modeling Pipeline

```
In [14]: from sklearn.preprocessing import OneHotEncoder, StandardScaler

categorical_preprocessor = OneHotEncoder(handle_unknown="ignore")
numerical_preprocessor = StandardScaler()
```

```
In [15]: from sklearn.compose import ColumnTransformer

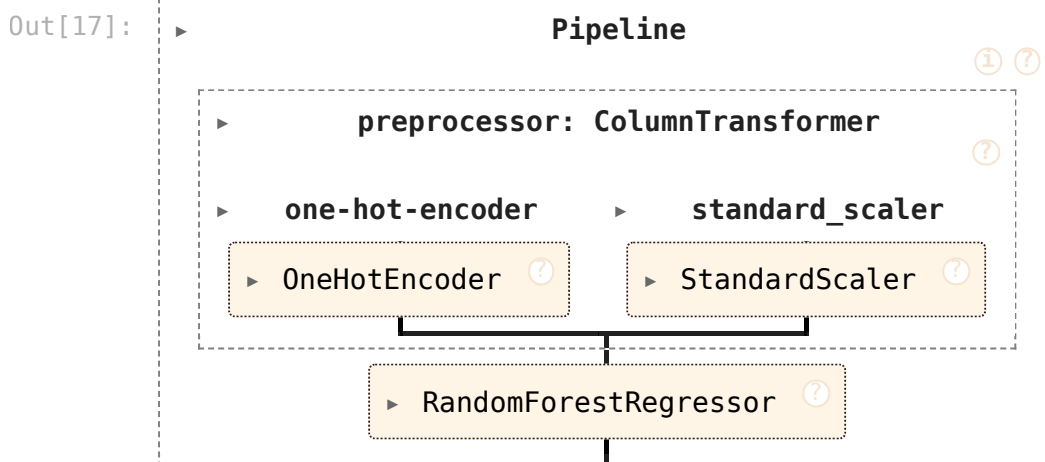
preprocessor = ColumnTransformer(
    [
        ("one-hot-encoder", categorical_preprocessor, categorical_vars),
        ("standard_scaler", numerical_preprocessor, cont_vars),
    ]
)
```

We now create a pipeline of our regression model as shown below.

```
In [16]: model = Pipeline(
    [
        ("preprocessor", preprocessor),
        ("regressor", RandomForestRegressor(random_state=42)),
    ]
)
```

```
In [17]: from sklearn import set_config

set_config(display="diagram")
model
```



The data is now split into training data and testing data.

```
In [18]: # Let's separate into train and test set
# Remember to set the seed (random_state for this `sklearn` function)

X_train, X_test, y_train, y_test = train_test_split(
    X, # predictive variables
    y, # target
    test_size=0.3, # portion of dataset to allocate to test set
    random_state=0, # we are setting the seed here
)

X_train.shape, X_test.shape
```

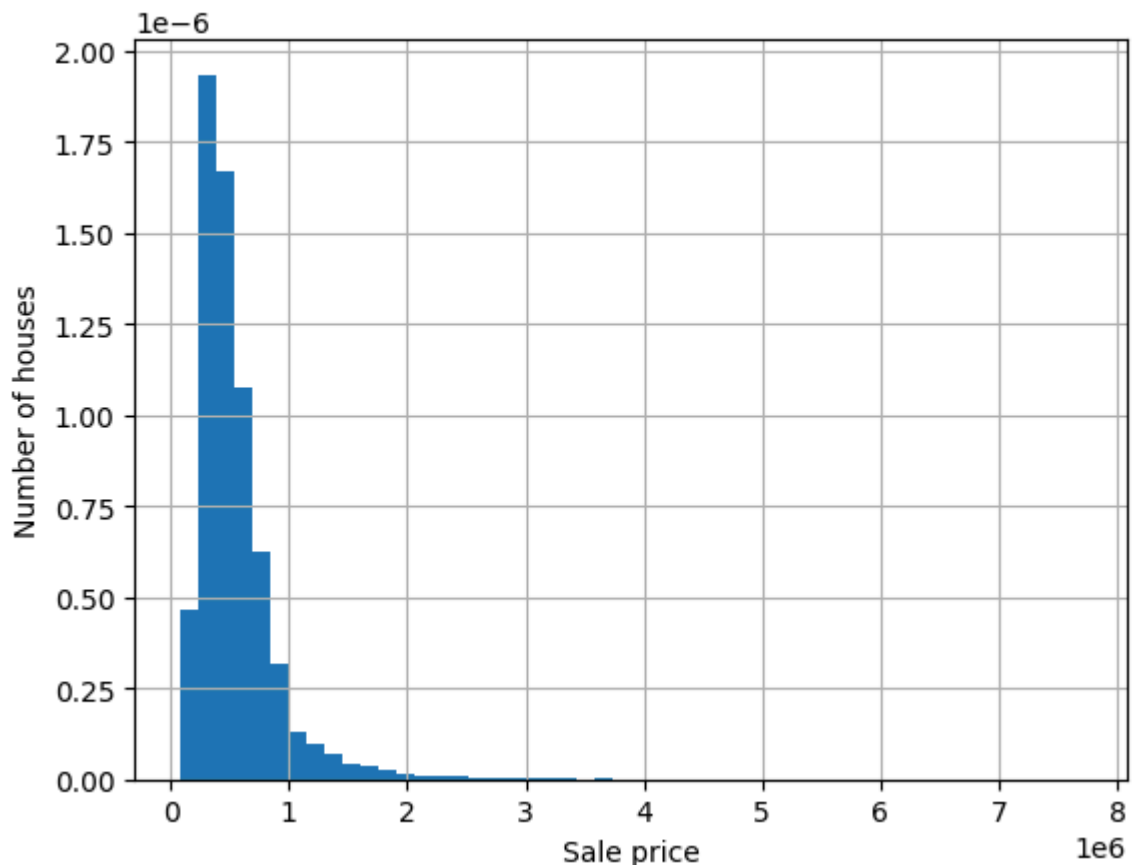
```
Out[18]: ((15129, 18), (6484, 18))
```

We can note below that the target variable is skewed to the left and our model will have a tendency to predict values to the most common value. Therefore, we transform it to make it more normal.

```
In [19]: # histogram to evaluate target distribution
```

```
df["price"].hist(bins=50, density=True)
plt.ylabel("Number of houses")
plt.xlabel("Sale price")
plt.suptitle(
    "Fig. 2: Target Variable Distribution",
    fontweight="bold",
    horizontalalignment="right",
)
plt.show()
```

Fig. 2: Target Variable Distribution

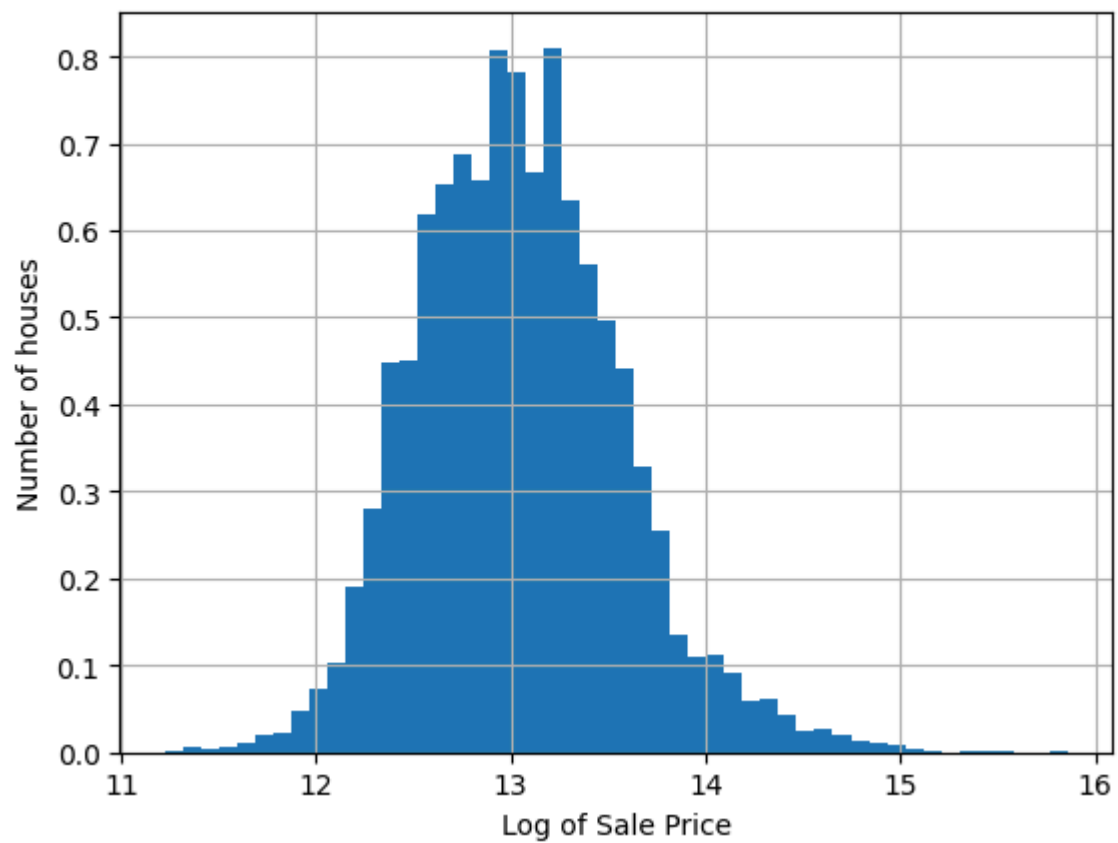


After log transforming the target variable, it now looks more normally distributed and will therefore use the log transform variable as our target.

```
In [20]: # let's transform the target using the logarithm
```

```
np.log(df["price"]).hist(bins=50, density=True)
plt.ylabel("Number of houses")
plt.xlabel("Log of Sale Price")
plt.suptitle(
    "Fig. 3: Log Transformed Target", fontweight="bold",
    horizontalalignment="right"
)
plt.show()
```

Fig. 3: Log Transformed Target



```
In [21]: y_train = np.log(y_train)
         y_test = np.log(y_test)
```

```
In [22]: X_train.head()
```

Out[22]:

	b e d r o o m s	b a t h r o o m s	s q f t _ l i v i n g	s q f t _ l o t	f l o o r s	w a t e r f r o n t	v i e w	c o n d i t i o n	g r a d e	s q f t _ a b o v e	s q f t _ b a s e m e n t	y r _ b u i l t	y r _ r e n o v a t e d	z i p c o d e	l a t	l o n g	s q f t _ l i v i n g 1 5	s q f t _ l o t 1 5
1468	4	1.50	1390	7200	1.0	0	0	3	7	1140	250	1965	0	98133	47.7224	-122.332	1630	7702
15590	3	1.50	1450	7316	1.0	0	0	3	7	1450	0	1961	0	98133	47.725	-122.49	1440	7316
18552	5	2.75	2860	5379	2.0	0	0	3	9	2860	0	2005	0	98052	47.7082	-122.104	2980	6018
10535	2	1.00	1050	4125	1.0	0	0	4	7	1050	0	1909	0	98144	47.5736	-122.307	1650	4125
1069	2	1.00	1240	57000	1.0	0	0	3	7	1240	0	1962	0	98075	47.5970	-122.059	3050	25545

We now train the model using the pipeline we formulated above.

In [23]: `_ = model.fit(X_train, y_train)`

In [24]: `X_test.head()`

Out[24]:

	b e d r o o m s	b a t h r o o m s	s q f t _ l i v i n g	s q f t _ l o t	f l o o r s	w a t e r f r o n t	v i e w	c o n d i t i o n	g r a d e	s q f t _ a b o v e	s q f t _ b a s e m e n t	y r _ b u i l t	y r _ r e n o v a t e d	z i p c o d e	l a t	l o n g	s q f t _ l i v i n g 1 5	s q f t _ l o t 1 5
1 7 3 8 4	2	1. 50	14 30	16 50	3. 0	0	0	3	7	14 30	0	19 99	0	98 12 5	47 .7 22 2	-1 22 .2 90	14 30	16 50
7 2 2	4	3. 25	46 70	51 83 6	2. 0	0	0	4	12	46 70	0	19 88	0	98 00 5	47 .6 35 0	-1 22 .1 64	42 30	41 07 5
2 6 8 0	2	0. 75	14 40	37 00	1. 0	0	0	3	7	12 00	24 0	19 14	0	98 10 7	47 .6 70 7	-1 22 .3 64	14 40	43 00
1 8 7 5 4	2	1. 00	11 30	26 40	1. 0	0	0	4	8	11 30	0	19 27	0	98 10 9	47 .6 43 8	-1 22 .3 57	16 80	32 00
1 4 5 5 4	4	2. 50	31 80	96 03	2. 0	0	2	3	9	31 80	0	20 02	0	98 15 5	47 .7 71 7	-1 22 .2 77	24 40	15 26 1

We then use the model to make predictions about unseen test data and evaluate its performance.

```
In [25]: target_predicted = model.predict(X_test)

print(
    f"Mean squared error on the testing set: "
    f"{mean_squared_error(y_test, target_predicted):.3f}"
)
```

Mean squared error on the testing set: 0.031

4. Regression Metrics

There are various metrics we can use to evaluate the performance of a regression model.

```
In [26]: model.score(X_test, y_test)
```

Out[26]: 0.8880587100455746

The model performed really well, and we will now want to see if we can improve this performance using hyperparameter tuning.

```
In [27]: from sklearn.dummy import DummyRegressor

dummy_regressor = DummyRegressor(strategy="mean")
dummy_regressor.fit(X_train, y_train)
print(
    f"R2 score for a regressor predicting the mean:"
    f"{dummy_regressor.score(X_test, y_test):.3f}"
)
```

R2 score for a regressor predicting the mean:-0.000

```
In [28]: from sklearn.metrics import mean_absolute_error

print(f"Mean absolute error: " f"{mean_absolute_error(y_test,
target_predicted):.3f} $")
```

Mean absolute error: 0.124 \$

5. Hyperparameter Tuning

5.1 Random Search

We will need to define the hyperparameter grid to be able to use random search.

```
In [29]: from pprint import pprint

# Look at parameters used by our current forest
print("Parameters currently in use:\n")
pprint(model.get_params())
```

Parameters currently in use:

```
{'memory': None,
 'preprocessor': ColumnTransformer(transformers=[('one-hot-encoder',
                                                OneHotEncoder(handle_unknown='ignore'),
                                                ['bedrooms', 'floors', 'waterfront',
                                                'view',
                                                'condition', 'grade'])),
                                ('standard_scaler', StandardScaler(),
                                ['bathrooms', 'sqft_living', 'sqft_lot',
                                'sqft_above', 'sqft_basement', 'yr_built',
                                'yr_renovated', 'zipcode', 'lat', 'long',
                                'sqft_living15', 'sqft_lot15'])]),
 'preprocessor__force_int_remainder_cols': True,
 'preprocessor__n_jobs': None,
 'preprocessor__one-hot-encoder': OneHotEncoder(handle_unknown='ignore'),
 'preprocessor__one-hot-encoder__categories': 'auto',
 'preprocessor__one-hot-encoder__drop': None,
 'preprocessor__one-hot-encoder__dtype': <class 'numpy.float64'>,
 'preprocessor__one-hot-encoder__feature_name_combiner': 'concat',
 'preprocessor__one-hot-encoder__handle_unknown': 'ignore',
 'preprocessor__one-hot-encoder__max_categories': None,
 'preprocessor__one-hot-encoder__min_frequency': None,
 'preprocessor__one-hot-encoder__sparse_output': True,
 'preprocessor__remainder': 'drop',
 'preprocessor__sparse_threshold': 0.3,
 'preprocessor__standard_scaler': StandardScaler(),
 'preprocessor__standard_scaler__copy': True,
 'preprocessor__standard_scaler__with_mean': True,
 'preprocessor__standard_scaler__with_std': True,
 'preprocessor__transformer_weights': None,
 'preprocessor__transformers': [('one-hot-encoder',
                                OneHotEncoder(handle_unknown='ignore'),
                                ['bedrooms',
                                'floors',
                                'waterfront',
                                'view',
                                'condition',
                                'grade'])),
                                ('standard_scaler',
                                StandardScaler(),
                                ['bathrooms',
                                'sqft_living',
                                'sqft_lot',
                                'sqft_above',
                                'sqft_basement',
                                'yr_built',
                                'yr_renovated',
                                'zipcode',
                                'lat',
                                'long',
                                'sqft_living15',
                                'sqft_lot15'])]),
 'preprocessor__verbose': False,
 'preprocessor__verbose_feature_names_out': True,
 'regressor': RandomForestRegressor(random_state=42),
 'regressor__bootstrap': True,
 'regressor__ccp_alpha': 0.0,
 'regressor__criterion': 'squared_error',
 'regressor__max_depth': None,
```



```

'regressor__max_features': 1.0,
'regressor__max_leaf_nodes': None,
'regressor__max_samples': None,
'regressor__min_impurity_decrease': 0.0,
'regressor__min_samples_leaf': 1,
'regressor__min_samples_split': 2,
'regressor__min_weight_fraction_leaf': 0.0,
'regressor__monotonic_cst': None,
'regressor__n_estimators': 100,
'regressor__n_jobs': None,
'regressor__oob_score': False,
'regressor__random_state': 42,
'regressor__verbose': 0,
'regressor__warm_start': False,
'steps': [('preprocessor',
            ColumnTransformer(transformers=[('one-hot-encoder',
                                            OneHotEncoder(handle_unknown='ignore'),
                                            ['bedrooms', 'floors', 'waterfront',
'view',
                                            'condition', 'grade'])),
            ('standard_scaler', StandardScaler(),
            ['bathrooms', 'sqft_living', 'sqft_lot',
            'sqft_above', 'sqft_basement', 'yr_built',
            'yr_renovated', 'zipcode', 'lat', 'long',
            'sqft_living15', 'sqft_lot15'])))],
            ('regressor', RandomForestRegressor(random_state=42))],
'transform_input': None,
'verbose': False}

```

Random search has the advantage of being fast as it does not use all the values in the hyperparameter space but rather randomly selects variables.

In [30]: `from sklearn.model_selection import RandomizedSearchCV`

```

# Number of trees in random forest
n_estimators = [int(x) for x in np.linspace(start=10, stop=90, num=10)]
# Number of features to consider at every split
max_features = ["auto", "sqrt"]
# Maximum number of levels in tree
max_depth = [int(x) for x in np.linspace(10, 50, num=21)]
max_depth.append(None)
# Minimum number of samples required to split a node
min_samples_split = [2, 5, 10]
# Minimum number of samples required at each leaf node
min_samples_leaf = [1, 2, 4]
# Method of selecting samples for training each tree
bootstrap = [True, False]
# Create the random grid
random_grid = {
    "regressor__n_estimators": n_estimators,
    "regressor__max_features": max_features,
    "regressor__max_depth": max_depth,
    "regressor__min_samples_split": min_samples_split,
    "regressor__min_samples_leaf": min_samples_leaf,
    "regressor__bootstrap": bootstrap,
}
pprint(random_grid)

```

```

{'regressor__bootstrap': [True, False],
 'regressor__max_depth': [10,
                           12,
                           14,
                           16,
                           18,
                           20,
                           22,
                           24,
                           26,
                           28,
                           30,
                           32,
                           34,
                           36,
                           38,
                           40,
                           42,
                           44,
                           46,
                           48,
                           50,
                           None],
 'regressor__max_features': ['auto', 'sqrt'],
 'regressor__min_samples_leaf': [1, 2, 4],
 'regressor__min_samples_split': [2, 5, 10],
 'regressor__n_estimators': [10, 18, 27, 36, 45, 54, 63, 72, 81, 90]}

```

We now create a model with all the parameters specified above.

```

In [31]: model_random_search = RandomizedSearchCV(
        model,
        param_distributions=random_grid,
        n_iter=10,
        cv=5,
        verbose=1,
    )
    model_random_search.fit(X_train, y_train)

```

Fitting 5 folds for each of 10 candidates, totalling 50 fits

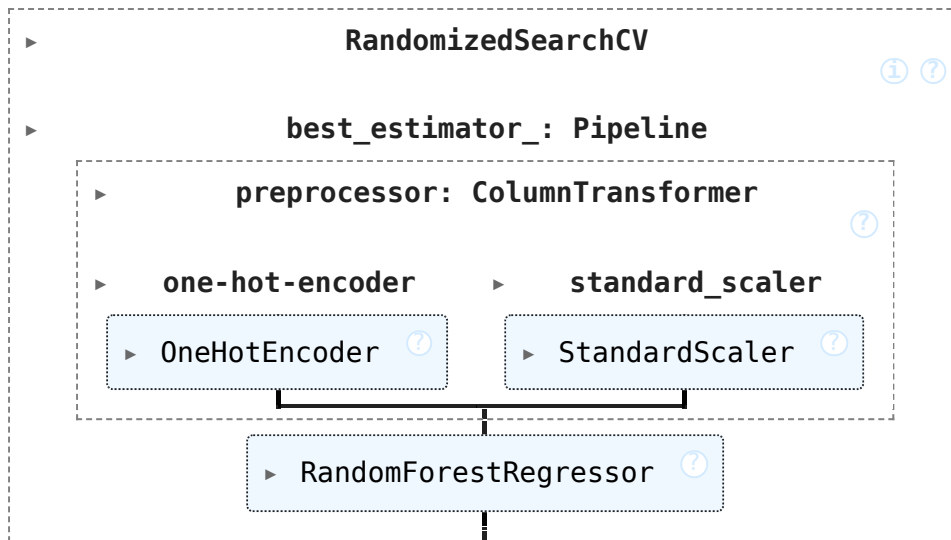
```
/usr/local/lib/python3.11/site-  
packages/sklearn/model_selection/_validation.py:528: FitFailedWarning:  
20 fits failed out of a total of 50.  
The score on these train-test partitions for these parameters will be set to  
nan.  
If these failures are not expected, you can try to debug them by setting  
error_score='raise'.
```

Below are more details about the failures:

```
-----  
-----  
20 fits failed with the following error:  
Traceback (most recent call last):  
  File "/usr/local/lib/python3.11/site-  
packages/sklearn/model_selection/_validation.py", line 866, in  
_fit_and_score  
    estimator.fit(X_train, y_train, **fit_params)  
  File "/usr/local/lib/python3.11/site-packages/sklearn/base.py", line 1389,  
in wrapper  
    return fit_method(estimator, *args, **kwargs)  
    ~~~~~  
  File "/usr/local/lib/python3.11/site-packages/sklearn/pipeline.py", line  
662, in fit  
    self._final_estimator.fit(Xt, y, **last_step_params["fit"])  
  File "/usr/local/lib/python3.11/site-packages/sklearn/base.py", line 1382,  
in wrapper  
    estimator._validate_params()  
  File "/usr/local/lib/python3.11/site-packages/sklearn/base.py", line 436,  
in _validate_params  
    validate_parameter_constraints(  
  File "/usr/local/lib/python3.11/site-  
packages/sklearn/utils/_param_validation.py", line 98, in  
validate_parameter_constraints  
    raise InvalidParameterError(  
sklearn.utils._param_validation.InvalidParameterError: The 'max_features'  
parameter of RandomForestRegressor must be an int in the range [1, inf), a  
float in the range (0.0, 1.0], a str among {'log2', 'sqrt'} or None. Got  
'auto' instead.  
  
    warnings.warn(some_fits_failed_message, FitFailedWarning)  
/usr/local/lib/python3.11/site-  
packages/sklearn/model_selection/_search.py:1108: UserWarning: One or more  
of the test scores are non-finite: [0.86166991 0.85890986      nan  
0.868487      nan      nan  
0.82284465 0.87015995      nan 0.85436498]  
    warnings.warn(  

```

Out[31]:



We can now visualize the best hyperparameters that we will fit back to our model.

```
In [32]: model_random_search.best_params_
```

```
Out[32]: {'regressor__n_estimators': 63,
          'regressor__min_samples_split': 5,
          'regressor__min_samples_leaf': 2,
          'regressor__max_features': 'sqrt',
          'regressor__max_depth': None,
          'regressor__bootstrap': False}
```

By using this hyperparameter values, we now evaluate our algorithm's performance.

```
In [33]: def evaluate(model, test_features, test_labels):
          predictions = model.predict(test_features)
          errors = abs(predictions - test_labels)
          mape = 100 * np.mean(errors / test_labels)
          accuracy = 100 - mape
          print("Model Performance")
          print("Average Error: {:.4f} dollars.".format(np.mean(errors)))
          print("Accuracy = {:.2f}%.".format(accuracy))

          return accuracy

          base_model = RandomForestRegressor(n_estimators=10, random_state=42)
          base_model.fit(X_train, y_train)
          base_accuracy = evaluate(base_model, X_test, y_test)
```

```
Model Performance
Average Error: 0.1299 dollars.
Accuracy = 99.00%.
```

```
In [34]: best_random = model_random_search.best_estimator_
          random_accuracy = evaluate(best_random, X_test, y_test)
```

```
Model Performance
Average Error: 0.1297 dollars.
Accuracy = 99.00%.
```

Comparing the performance of our model without and with hyperparameter tuning show us a marginal increase in accuracy.

```
In [35]: print(
    "Improvement of {:.2f}%".format(
        100 * (random_accuracy - base_accuracy) / base_accuracy
    )
)
```

Improvement of 0.00%.

5.2 Grid Search

We now provide a framework for performing grid search on our model hyperparameters. Note that we restricted the hyperparameters to a smaller space since the grid search algorithm is very heavy. We therefore do not expect a better performance. Students are allowed to experiment with bigger spaces and see how the model performs.

```
In [36]: from sklearn.model_selection import GridSearchCV

# Create the parameter grid based on the results of random search
param_grid = {
    "regressor__bootstrap": [True],
    "regressor__max_depth": [int(x) for x in np.linspace(10, 20, num=6)],
    "regressor__max_features": [2, 3],
    "regressor__min_samples_leaf": [3, 4, 5],
    "regressor__min_samples_split": [8, 10, 12],
    "regressor__n_estimators": [
        int(x) for x in np.linspace(start=70, stop=100, num=11)
    ],
}
# Create a based model
rf = RandomForestRegressor()
# Instantiate the grid search model
grid_search = GridSearchCV(
    estimator=model, param_grid=param_grid, cv=3, n_jobs=-1, verbose=2
)
```

```
In [37]: # Fit the grid search to the data
grid_search.fit(X_train, y_train)
grid_search.best_params_
```

[illegible]

[illegible]

[illegible]


```

1.1s
[CV] END regressor__bootstrap=True, regressor__max_depth=20,
regressor__max_features=3, regressor__min_samples_leaf=5,
regressor__min_samples_split=12, regressor__n_estimators=94; total time=
1.2s
[CV] END regressor__bootstrap=True, regressor__max_depth=20,
regressor__max_features=3, regressor__min_samples_leaf=5,
regressor__min_samples_split=12, regressor__n_estimators=94; total time=
1.2s
[CV] END regressor__bootstrap=True, regressor__max_depth=20,
regressor__max_features=3, regressor__min_samples_leaf=5,
regressor__min_samples_split=12, regressor__n_estimators=94; total time=
1.2s
[CV] END regressor__bootstrap=True, regressor__max_depth=20,
regressor__max_features=3, regressor__min_samples_leaf=5,
regressor__min_samples_split=12, regressor__n_estimators=97; total time=
1.2s
[CV] END regressor__bootstrap=True, regressor__max_depth=20,
regressor__max_features=3, regressor__min_samples_leaf=5,
regressor__min_samples_split=12, regressor__n_estimators=97; total time=
1.2s
[CV] END regressor__bootstrap=True, regressor__max_depth=20,
regressor__max_features=3, regressor__min_samples_leaf=5,
regressor__min_samples_split=12, regressor__n_estimators=97; total time=
1.2s
[CV] END regressor__bootstrap=True, regressor__max_depth=20,
regressor__max_features=3, regressor__min_samples_leaf=5,
regressor__min_samples_split=12, regressor__n_estimators=100; total time=
1.3s
[CV] END regressor__bootstrap=True, regressor__max_depth=20,
regressor__max_features=3, regressor__min_samples_leaf=5,
regressor__min_samples_split=12, regressor__n_estimators=100; total time=
1.2s
[CV] END regressor__bootstrap=True, regressor__max_depth=20,
regressor__max_features=3, regressor__min_samples_leaf=5,
regressor__min_samples_split=12, regressor__n_estimators=100; total time=
1.3s

```

```

Out[37]: {'regressor__bootstrap': True,
          'regressor__max_depth': 20,
          'regressor__max_features': 3,
          'regressor__min_samples_leaf': 3,
          'regressor__min_samples_split': 10,
          'regressor__n_estimators': 97}

```

```

In [38]: best_grid = grid_search.best_estimator_
         grid_accuracy = evaluate(best_grid, X_test, y_test)

```

Model Performance

Average Error: 0.1731 dollars.

Accuracy = 98.67%.

We also compare the performance of the grid search algorithm with that of the original random forest algorithm.

```

In [39]: print(
          "Improvement of {:.2f}%.".format(
              100 * (grid_accuracy - base_accuracy) / base_accuracy
          )
        )

```

Improvement of -0.33%.

Conclusion

In this lesson, we have seen how to apply hyperparameter tuning to a regression problem. In the next lesson, we will study objective functions and other techniques used to evaluate machine learning models.

References

1. Breiman, Leo. "Random Forests." *Machine Learning*, vol. 45, no. 1, 2001, pp 5-32.
2. GeoDa Data and Lab. "2014-15 Home Sales in King County, WA."
<https://geodacenter.github.io/data-and-lab/KingCounty-HouseSales2015/>
3. Ramadhan, Muhammad Murtadha, et al. "Parameter Tuning in Random Forest based on Grid Search Method for Gender Classification based on Voice Frequency." *DEStech Transactions on Computer Science and Engineering*, vol. 10, 2017.

Copyright 2025 WorldQuant University. This content is licensed solely for personal use. Redistribution or publication of this material is strictly prohibited.

Required Readings

Required Readings for this program are open access, which means you should be able to access them at no cost. The link provided in the citation will take you directly to the reading or to a page where you can download it.

1. Shekhar, Shashank et al. "A Comparative Study of Hyper-Parameter Optimization Tools." *arXiv*, 2022, <https://arxiv.org/abs/2201.06433>.

- **Required pages:** Section 3c
- **Estimated reading time:** 15 minutes

MACHINE LEARNING IN FINANCE

MODULE 7 | LESSON 3

MODEL EVALUATION AND REGULARIZATION

Reading Time	40 minutes
Prior Knowledge	Introduction to hyperparameter tuning, loss functions
Keywords	simulated annealing

In the previous lesson, we applied grid search and random search hyperparameter tuning techniques to a house prediction problem and evaluated the model performance. In this lesson, we will present insights on when to use different cross-validation strategies. We will also discuss model evaluation metrics and introduce the notion of having a baseline model to compare with our model performance.

1. Model Evaluation

In the previous modules, we have encountered different learning algorithms, and in all of them, we are interested in minimizing errors when presented with never-before-seen data. To achieve this, we will discuss the following concepts in this section:

1. Choosing an appropriate cross-validation strategy.
2. Building a baseline model that acts as a reference for the actual model.
3. Nested cross-validation concepts and their evaluation.
4. Metrics used in classification and regression models.

1.1 Building Baseline Models

The baseline model can be thought of as a model reference to the actual model. We should consider the following points when building a baseline model:

1. It should be a simple model that is used for comparison.
2. It should be easy to explain.
3. It should be based on the actual data.

Baseline models are important for the following reasons;

Understanding the Data

When we train a baseline model on our dataset, we can infer the following:

- A baseline model where prediction of our target variable is completely off indicates how difficult it will be for weaker models to get patterns in the dataset.
- It can help us point out the segment of the data that gives the model a higher error.

Faster Modeling

Once we have a baseline model in place, building the actual model pipeline becomes easier as we understand the data better.

Performance Benchmark

The performance of the baseline model help us to decide whether we will need a complex model or a simple one.

We can divide the baseline model into two types:

1. Simple baseline model.
2. Machine learning baseline model.

In this subsection, we will consider a simple baseline model that uses simple logic to build the baseline model.

```
In [1]: from sklearn.datasets import fetch_california_housing  
  
data, target = fetch_california_housing(return_X_y=True, as_frame=True)
```

We split the data using `ShuffleSplit` with 20% of the data used for validation.

```
In [2]: from sklearn.model_selection import ShuffleSplit  
  
cv = ShuffleSplit(n_splits=30, test_size=0.2, random_state=0)
```

We define a decision tree regression function as our model for this section.

```
In [3]: import pandas as pd  
from sklearn.model_selection import cross_validate  
from sklearn.tree import DecisionTreeRegressor  
  
regressor = DecisionTreeRegressor()  
cv_results_tree_regressor = cross_validate(  
    regressor, data, target, cv=cv, scoring="neg_mean_absolute_error",  
    n_jobs=2  
)  
  
errors_tree_regressor = pd.Series(  
    -cv_results_tree_regressor["test_score"], name="Decision tree  
    regressor"  
)
```

We use a dummy regressor as our baseline that predicts the mean of the target computed.

```
In [4]: from sklearn.dummy import DummyRegressor

dummy = DummyRegressor(strategy="mean")
result_dummy = cross_validate(
    dummy, data, target, cv=cv, scoring="neg_mean_absolute_error",
    n_jobs=2
)
errors_dummy_regressor = pd.Series(-result_dummy["test_score"],
    name="Dummy regressor")
errors_dummy_regressor.describe()
```

```
Out[4]:
```

	Dummy regressor
count	30.000000
mean	0.911400
std	0.008211
min	0.897576
25%	0.905437
50%	0.910346
75%	0.919790
max	0.924772

dtype: float64

In the next cell, we prepare a table that compares the performance of the decision tree regressor and the dummy regressor.

```
In [5]: all_errors = pd.concat(
    [errors_tree_regressor, errors_dummy_regressor],
    axis=1,
)
all_errors
```


Out[5]:

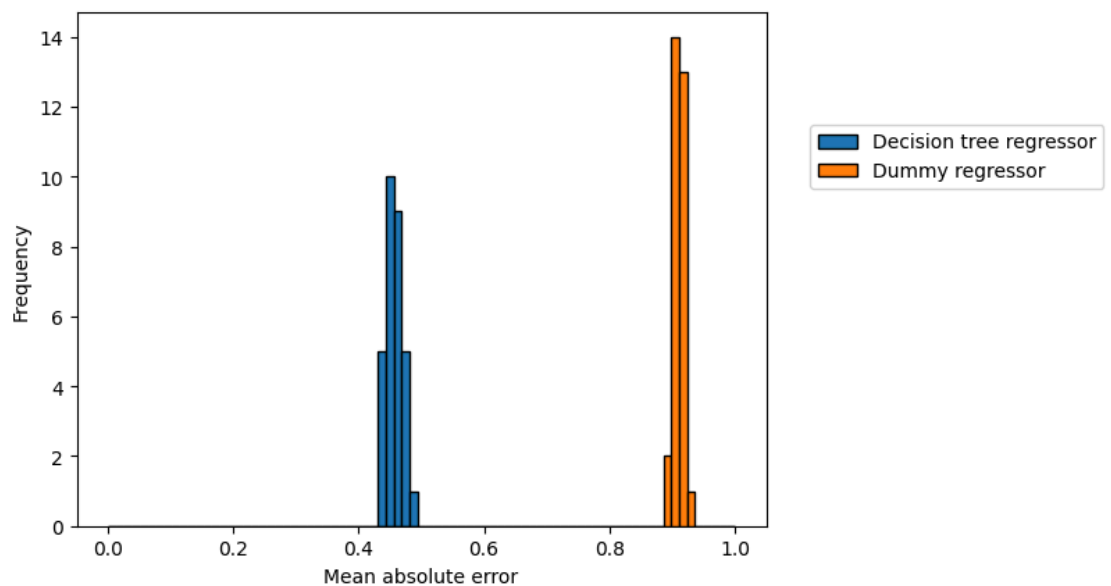
	Decision tree regressor	Dummy regressor
0	0.471738	0.907132
1	0.470964	0.905394
2	0.439831	0.919419
3	0.435998	0.902139
4	0.481446	0.920159
5	0.449905	0.905425
6	0.441220	0.897576
7	0.446698	0.924772
8	0.447942	0.909480
9	0.454922	0.919914
10	0.463170	0.920236
11	0.456946	0.905570
12	0.457347	0.915396
13	0.453824	0.911852
14	0.468695	0.922990
15	0.442110	0.910846
16	0.455470	0.909845
17	0.465725	0.899817
18	0.451124	0.905471
19	0.466326	0.898202
20	0.434392	0.917687
21	0.460897	0.923056
22	0.455746	0.905030
23	0.471483	0.921480
24	0.465491	0.913863
25	0.452337	0.908157
26	0.446684	0.922166
27	0.462099	0.901075
28	0.454428	0.906203
29	0.480872	0.911653

We see from the table above that in all the splits, the decision tree regressor has fewer errors compared to the dummy regressor. In the next cell, we can visualize the distribution of errors for both regression functions.

```
In [6]: import matplotlib.pyplot as plt
import numpy as np

bins = np.linspace(start=0, stop=1, num=80)
all_errors.plot.hist(bins=bins, edgecolor="black")
plt.legend(bbox_to_anchor=(1.05, 0.8), loc="upper left")
plt.xlabel("Mean absolute error")
plt.suptitle(
    "Fig. 1: Cross-validation testing errors.",
    fontweight="bold",
    horizontalalignment="right",
)
plt.show()
```

Fig. 1: Cross-validation testing errors.



1.2 Choosing a Cross-Validation Strategy

So far we have only used `ShuffleSplit` or `KFold` to perform cross-validation, but they may not work all the time. As such, it's important to choose a cross-validation strategy wisely, depending on the task at hand. Let's consider the example below to demonstrate why it is important to choose a cross-validation strategy carefully.

```
In [7]: from sklearn.datasets import load_iris

data, target = load_iris(as_frame=True, return_X_y=True)
```

We now create a simple classification algorithm for this task.

```
In [8]: from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

model = make_pipeline(StandardScaler(), LogisticRegression())
```

We now use this model to train and generalize on unseen data.

```
In [9]: from sklearn.model_selection import KFold, cross_validate

cv = KFold(n_splits=3)
results = cross_validate(model, data, target, cv=cv)
test_score = results["test_score"]
print(f"The average accuracy is " f"{test_score.mean():.3f} ±
{test_score.std():.3f}")
```

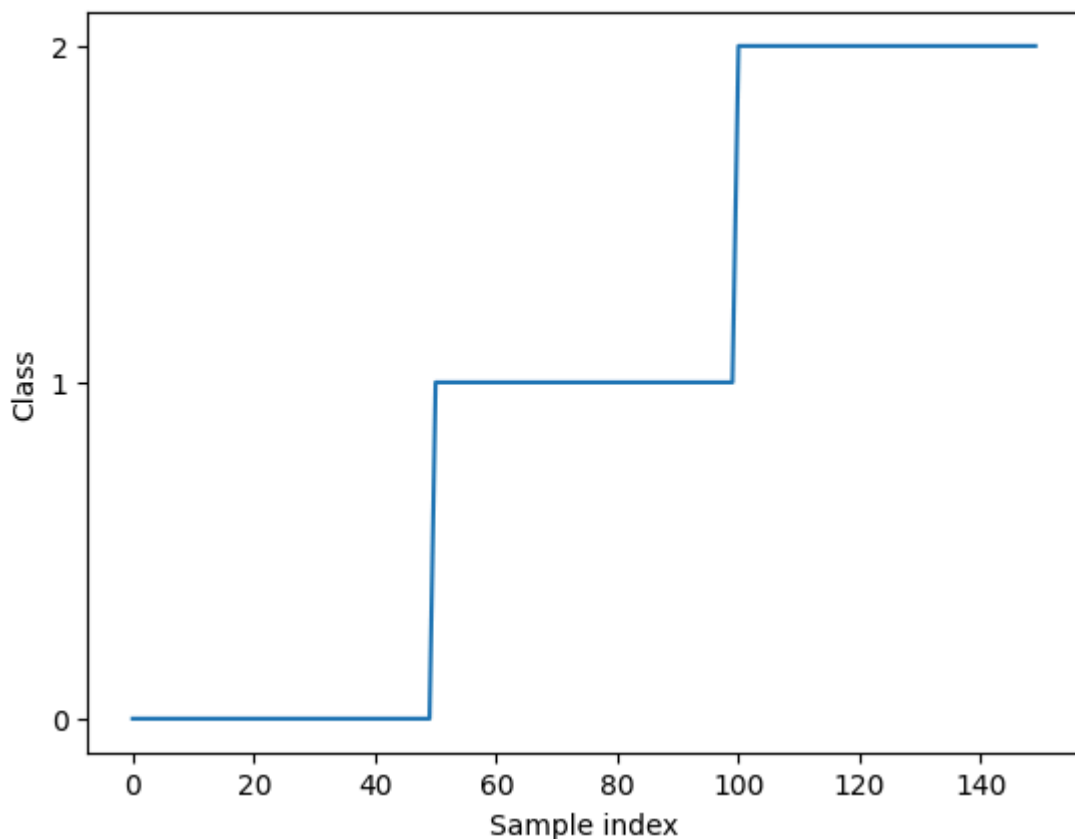
The average accuracy is 0.000 ± 0.000

The output above shows that the model was unable to get any prediction correct when exposed to validation data. To understand this peculiar output, we investigate the target feature.

```
In [10]: import matplotlib.pyplot as plt

target.plot()
plt.xlabel("Sample index")
plt.ylabel("Class")
plt.yticks(target.unique())
plt.suptitle(
    "Fig. 2: Class value in target y.", fontweight="bold",
    horizontalalignment="right"
)
plt.show()
```

Fig. 2: Class value in target y.



We can observe from the above diagram that the target column is ordered, and this causes an unexpected outcome when we use cross-validation. We will now compute the class counts for the training and validation sets in the cells below.

```
In [11]: import pandas as pd

n_splits = 3
cv = KFold(n_splits=n_splits)

train_cv_counts = []
test_cv_counts = []
for fold_idx, (train_idx, test_idx) in enumerate(cv.split(data, target)):
    target_train, target_test = target.iloc[train_idx],
    target.iloc[test_idx]

    train_cv_counts.append(target_train.value_counts())
    test_cv_counts.append(target_test.value_counts())
```

We display this in the table below.

```
In [12]: train_cv_counts = pd.concat(
    train_cv_counts, axis=1, keys=[f"Fold #{idx}" for idx in
range(n_splits)]
)
train_cv_counts.index.name = "Class label"
train_cv_counts
```

```
Out[12]:
```

	Fold #0	Fold #1	Fold #2
Class label			
1	50.0	NaN	50.0
2	50.0	50.0	NaN
0	NaN	50.0	50.0

We can see that when training the model, in each fold, only two of the classes are available, and thus, it becomes difficult to predict on unseen data.

```
In [13]: test_cv_counts = pd.concat(
    test_cv_counts, axis=1, keys=[f"Fold #{idx}" for idx in
range(n_splits)]
)
test_cv_counts.index.name = "Class label"
test_cv_counts
```

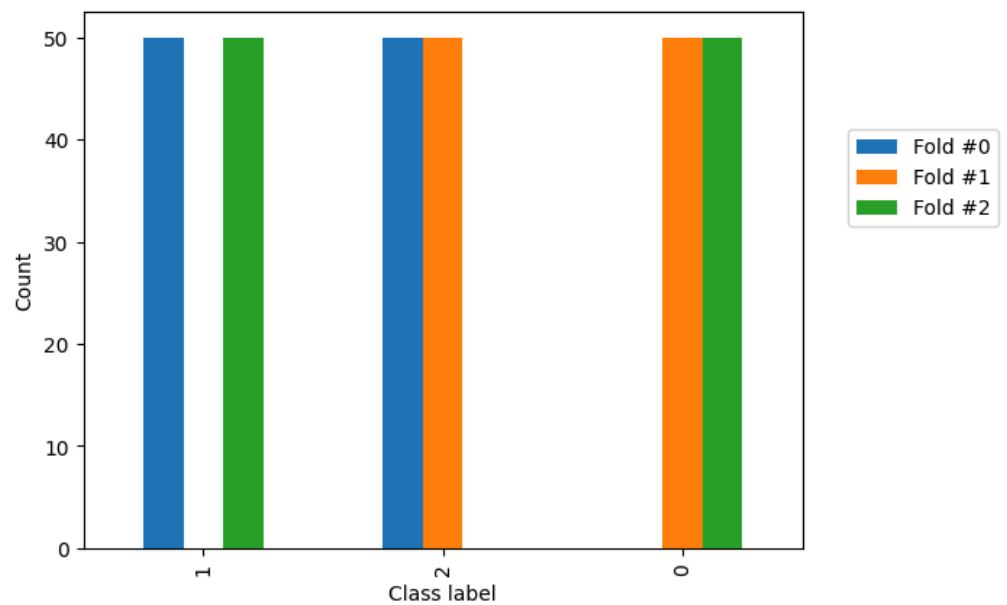
```
Out[13]:
```

	Fold #0	Fold #1	Fold #2
Class label			
0	50.0	NaN	NaN
1	NaN	50.0	NaN
2	NaN	NaN	50.0

In the test data, only one class, which was not available in training, is used. This makes it difficult to predict, as the class was not available when training. The cells below helps visualize the above tables.

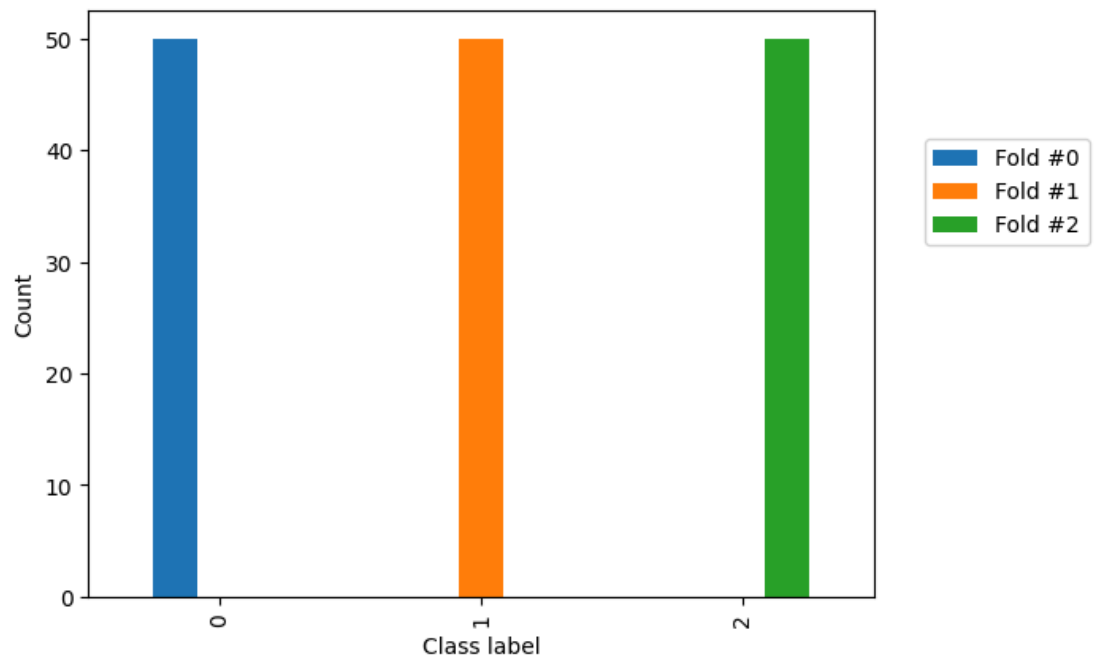
```
In [14]: train_cv_counts.plot.bar()
plt.legend(bbox_to_anchor=(1.05, 0.8), loc="upper left")
plt.ylabel("Count")
plt.suptitle(
    "Fig. 3: Training Set Data Distribution in Folds.",
    fontweight="bold",
    horizontalalignment="right",
)
plt.show()
```

Fig. 3: Training Set Data Distribution in Folds.



```
In [15]: test_cv_counts.plot.bar()
plt.legend(bbox_to_anchor=(1.05, 0.8), loc="upper left")
plt.ylabel("Count")
plt.suptitle(
    "Fig. 4: Data distribution in Testing Set",
    fontweight="bold",
    horizontalalignment="right",
)
plt.show()
```

Fig. 4: Data distribution in Testing Set



To solve the problems above, it is a good practice to shuffle our dataset before we start the modeling phase as shown below.

```
In [16]: cv = KFold(n_splits=3, shuffle=True, random_state=0)
results = cross_validate(model, data, target, cv=cv)
test_score = results["test_score"]
print(f"The average accuracy is " f"{test_score.mean():.3f} ±
{test_score.std():.3f}")
```

The average accuracy is 0.953 ± 0.009

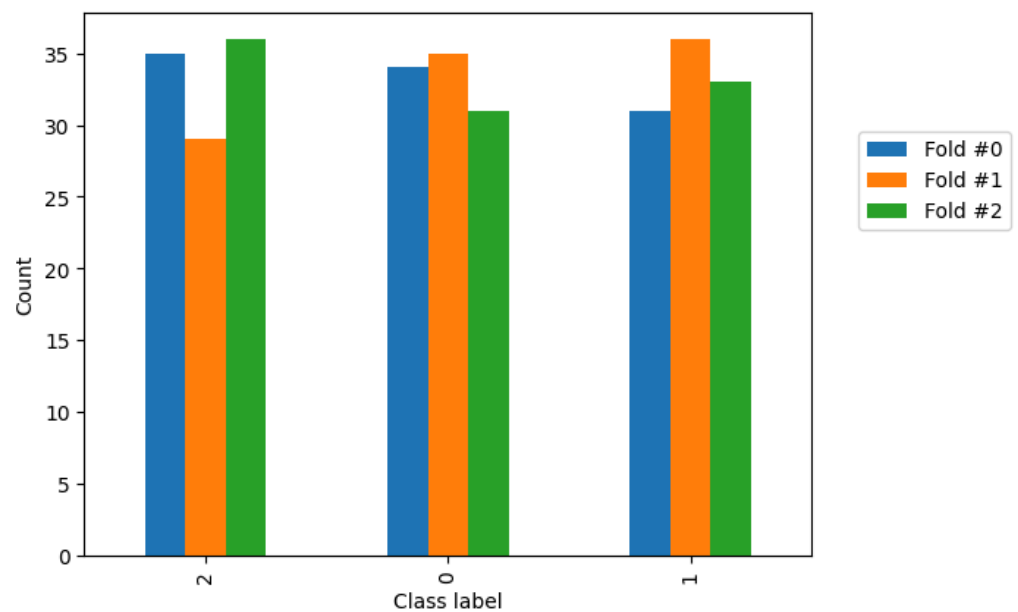
We can now see that the model accuracy has really improved to more than 95%. We can now visualize the count of class in each fold for both the training and testing samples.

```
In [17]: train_cv_counts = []
test_cv_counts = []
for fold_idx, (train_idx, test_idx) in enumerate(cv.split(data, target)):
    target_train, target_test = target.iloc[train_idx],
    target.iloc[test_idx]

    train_cv_counts.append(target_train.value_counts())
    test_cv_counts.append(target_test.value_counts())
train_cv_counts = pd.concat(
    train_cv_counts, axis=1, keys=[f"Fold #{idx}" for idx in
range(n_splits)]
)
test_cv_counts = pd.concat(
    test_cv_counts, axis=1, keys=[f"Fold #{idx}" for idx in
range(n_splits)]
)
train_cv_counts.index.name = "Class label"
test_cv_counts.index.name = "Class label"
```

```
In [18]: train_cv_counts.plot.bar()
plt.legend(bbox_to_anchor=(1.05, 0.8), loc="upper left")
plt.ylabel("Count")
plt.suptitle(
    "Fig. 5: Training set Data Distribution (kFold).",
    fontweight="bold",
    horizontalalignment="right",
)
plt.show()
```

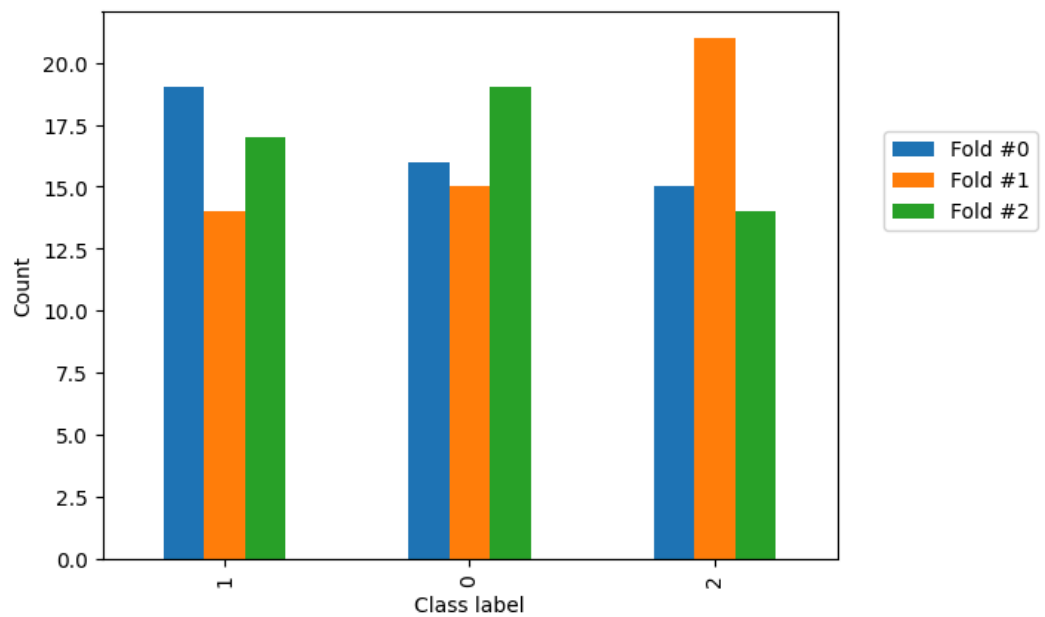
Fig. 5: Training set Data Distribution (kFold).



From the bar plots above, we can observe that the classes are fairly distributed in each fold. This is also true on the testing dataset below.

```
In [19]: test_cv_counts.plot.bar()
plt.legend(bbox_to_anchor=(1.05, 0.8), loc="upper left")
plt.ylabel("Count")
plt.suptitle(
    "Fig. 6: Testing set Data Distribution (kFold).",
    fontweight="bold",
    horizontalalignment="right",
)
plt.show()
```

Fig. 6: Testing set Data Distribution (kFold).



Another technique will involve splitting our dataset so as to preserve the original class counts in each validation split. This is done using stratification as shown in the cells below. The steps will remain as what we have seen in the two strategies before.

```
In [20]: from sklearn.model_selection import StratifiedKFold  
  
cv = StratifiedKFold(n_splits=3)
```

```
In [21]: results = cross_validate(model, data, target, cv=cv)  
test_score = results["test_score"]  
print(f"The average accuracy is " f"{test_score.mean():.3f} ±  
{test_score.std():.3f}")
```

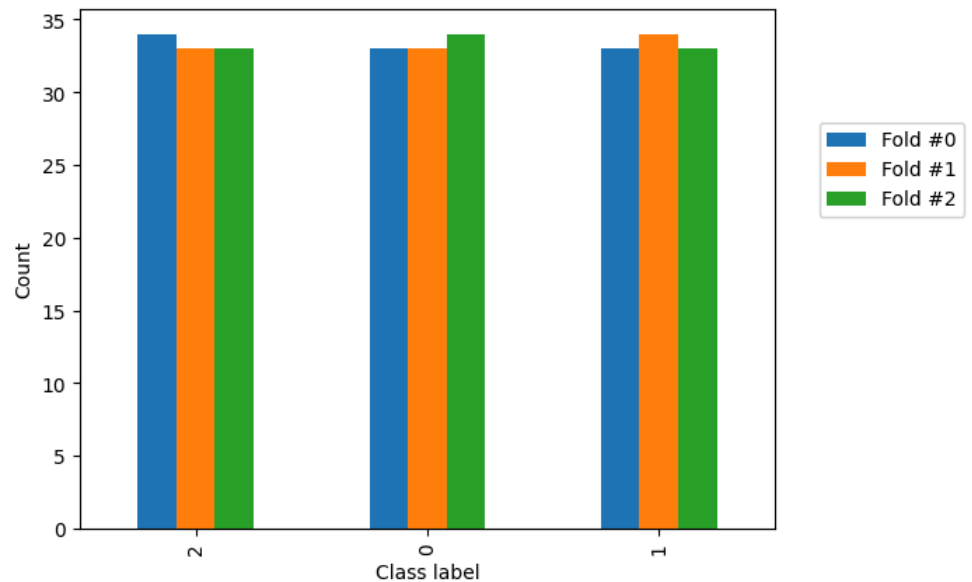
The average accuracy is 0.967 ± 0.009

```
In [22]: train_cv_counts = []  
test_cv_counts = []  
for fold_idx, (train_idx, test_idx) in enumerate(cv.split(data, target)):  
    target_train, target_test = target.iloc[train_idx],  
    target.iloc[test_idx]  
  
    train_cv_counts.append(target_train.value_counts())  
    test_cv_counts.append(target_test.value_counts())  
train_cv_counts = pd.concat(  
    train_cv_counts, axis=1, keys=[f"Fold #{idx}" for idx in  
    range(n_splits)]  
)  
test_cv_counts = pd.concat(  
    test_cv_counts, axis=1, keys=[f"Fold #{idx}" for idx in  
    range(n_splits)]  
)  
train_cv_counts.index.name = "Class label"  
test_cv_counts.index.name = "Class label"
```



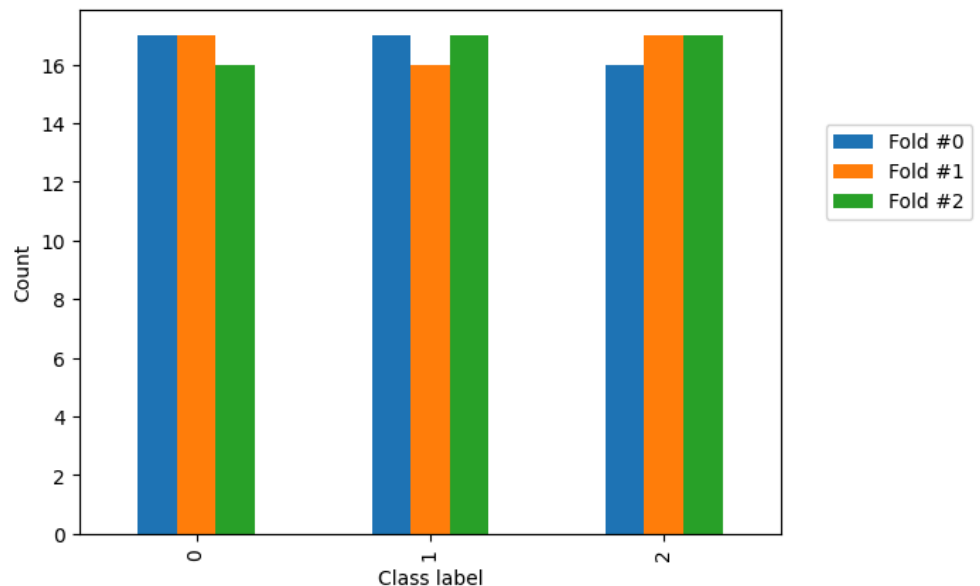
```
In [23]: train_cv_counts.plot.bar()
plt.legend(bbox_to_anchor=(1.05, 0.8), loc="upper left")
plt.ylabel("Count")
plt.suptitle(
    "Fig. 7: Training set Data Distribution (Stratified).",
    fontweight="bold",
    horizontalalignment="right",
)
plt.show()
```

Fig. 7: Training set Data Distribution (Stratified).



```
In [24]: test_cv_counts.plot.bar()
plt.legend(bbox_to_anchor=(1.05, 0.8), loc="upper left")
plt.ylabel("Count")
plt.suptitle(
    "Fig. 8: Testing set Data Distribution (Stratified).",
    fontweight="bold",
    horizontalalignment="right",
)
plt.show()
```

Fig. 8: Testing set Data Distribution (Stratified).



We can see that the counts in the training and testing sets are not too far from each other.

1.3 Nested Cross-Validation

The k-fold cross validation technique that we have discussed so far is important for hyperparameter optimization and model evaluation on the validation set. The fact that we use cross-validation on the same dataset to tune our hyperparameters and choose a model from the same process may cause the model evaluation to be biased.

A work-around to this limitation is to create an outer loop of cross-validation. This process is called **nested cross-validation** and we will demonstrate this in the cells below.

We then configure the cross-validation process below. We will use the make classification dataset.

```
In [25]: from numpy import mean, std
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import GridSearchCV, KFold, cross_val_score
```

We then split the dataset into a train and test dataset where the train dataset will also be used to train and validate the model.

```
In [26]: from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    data, target, shuffle=True, random_state=0, test_size=0.3
)
```

We then define the model to be used for this problem.

```
In [27]: model = RandomForestClassifier(random_state=1)
```

We now configure the inner cross-validation that will be used to train the hyperparameters.

```
In [28]: cv_inner = KFold(n_splits=3, shuffle=True, random_state=1)
```

As we have seen from Lesson 1, we define the search space for our hyperparameters to be tuned and apply grid search to get the best combination of hyperparameters.

```
In [29]: space = dict()
space["n_estimators"] = [10, 100, 500]
space["max_features"] = [2, 4, 6]
# Search for the best hyperparameter combination
search = GridSearchCV(
    model, space, scoring="accuracy", n_jobs=1, cv=cv_inner, refit=True
)
```

We now configure the cross-validation that will help us evaluate the model.

```
In [30]: cv_outer = KFold(n_splits=10, shuffle=True, random_state=1)
```

The next cell performs the nested cross validation on our data and we output the result.

```
In [31]: # execute the nested cross-validation
scores = cross_val_score(
    search, X_train, y_train, scoring="accuracy", cv=cv_outer, n_jobs=-1
)
# report performance
print("Accuracy: %.3f (%.3f)" % (mean(scores), std(scores)))
```

Accuracy: 0.952 (0.063)

We get the average model accuracy from the 10 iterations, and the result shows that our model has a better accuracy. But in classification, we need to go beyond accuracy as a measurement of model performance. In the next section, we learn about classification metrics.

2. Metrics in Machine Learning

When building machine learning models, we need to measure their performance, and understanding the type of metric to use is vital if we are to judge whether we are making progress or not. As we are all aware now, machine learning tasks can be divided into regression or classification, and they are treated differently when modeling and also when measuring their performance. We will start by looking at regression metrics.

2.1 Regression Metrics

Linear regression involves finding the linear relationship that exists between the independent variables and the target variable, that is,

$$y_i(x_i, w) = w_0 + w_1x_{i1} + w_2x_{i2} + \cdots w_mx_{im} + \varepsilon_i$$

where $y \in \mathbb{R}$ is the target variable, $x \in \mathbb{R}^m$ is the predictor variables and ε_i is the random error. The random error occurs as a result of one of the following

- Choosing a wrong linear model.
- Selecting less relevant variables and omitting relevant ones.
- Measurement errors.

A good linear regression estimates the coefficient of the features, w_i , and minimizes the residual errors, $e_i = y_i - \hat{y}_i$. This is possible by minimizing the objective function. The objective function can take one of the forms below.

2.1.1 Mean Squared Error (MSE)

The mean squared error is given by

$$L = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2$$

2.1.2 Mean Absolute Error (MAE)

The mean absolute error is given by

$$L = \frac{1}{n} \sum_{i=1}^n |\hat{y}_i - y_i|$$

2.1.3 Mean Squared Logarithmic Error

This is similar to MSE, but the dependent variable is in logarithmic scale:

$$L = \frac{1}{n} \sum_{i=1}^n (\log(\hat{y}_i + 1) - \log(y_i + 1))^2$$

In the next cells, we will look at different objective functions in regression for measuring errors.

```
In [32]: from sklearn.model_selection import train_test_split

data_train, data_test, target_train, target_test = train_test_split(
    data, target, shuffle=True, random_state=0
)
```

We first train the data using a decision-tree regressor and then evaluate on the test dataset. In the next cells, we look at the various metrics used to evaluate

machine learning models. We start with most commonly used metric, the mean squared error.

```
In [33]: from sklearn.metrics import mean_squared_error

regressor.fit(data_train, target_train)
target_predicted = regressor.predict(data_train)

print(
    f"Mean squared error on the training set: "
    f"{mean_squared_error(target_train, target_predicted):.3f}"
)
```

Mean squared error on the training set: 0.000

We measure the mean squared error on the test data below.

```
In [34]: target_predicted = regressor.predict(data_test)

print(
    f"Mean squared error on the testing set: "
    f"{mean_squared_error(target_test, target_predicted):.3f}"
)
```

Mean squared error on the testing set: 0.026

To calculate the coefficient of determination, R^2 , we calculate the `score()` function as shown below.

```
In [35]: regressor.score(data_test, target_test)
```

Out[35]: 0.9536585365853658

We can also calculate the mean absolute error for our model as shown below.

```
In [36]: from sklearn.metrics import mean_absolute_error

target_predicted = regressor.predict(data_test)
print(
    f"Mean absolute error: " f"{mean_absolute_error(target_test,
target_predicted):.3f}"
)
```

Mean absolute error: 0.026

The median absolute error is equally calculated.

```
In [37]: from sklearn.metrics import median_absolute_error

print(
    f"Median absolute error: "
    f"{median_absolute_error(target_test, target_predicted):.3f} "
)
```

Median absolute error: 0.000

2.1.4 Regularization

Regularization techniques are used in instances where a regression model overfits. The objective of regularization is to add a penalty term to the objective function that regularizes the size of the parameters, that is,

$$L = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2 + \lambda R(\vec{w})$$

where $R(\vec{w})$ is the penalty term, a hyperparameter.

The most commonly used regularization techniques are ridge regression and lasso regression.

1. Ridge Regression

Ridge regression adds an $L2$ -norm term to the objective function. The term causes weight decay to the variables. The loss function of a ridge regression is given by

$$L = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2 + \frac{\lambda}{2} ||w||^2$$

The λ hyperparameter controls the amount of decay where larger values of λ increase decay and thus make the model more robust to multi-collinearity. Note that for the value of $\lambda = 0$, ridge regression is equivalent to linear regression. An increase in the value of λ makes the coefficients of our model approach zero.

We start by importing the libraries for this demonstration.

```
In [38]: import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
from sklearn.metrics import mean_squared_error

sns.set_style("darkgrid")
```

We import and split the dataset into train and test sets.

```
In [39]: from sklearn.datasets import fetch_california_housing

X, y = fetch_california_housing(as_frame=True, return_X_y=True)
X.head()

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=25
)
```

We build the ridge model below and fit it to the training set.

```
In [40]: from sklearn.linear_model import Ridge
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import PolynomialFeatures, StandardScaler

steps = [
    ("scalar", StandardScaler()),
    ("poly", PolynomialFeatures(degree=2)),
    ("model", Ridge()),
]

ridge_pipe = Pipeline(steps)
ridge_pipe.fit(X_train, y_train)
# Predicting the Test set results
y_pred = ridge_pipe.predict(X_test)
```

We apply k-Fold cross-validation on the dataset.

```
In [41]: from sklearn.model_selection import cross_val_score

accuracies = cross_val_score(estimator=ridge_pipe, X=X_train, y=y_train,
cv=10)
print("accuracy mean", accuracies.mean())
print("accuracy standard deviation", accuracies.std())
```

```
accuracy mean 0.38636248096169873
accuracy standard deviation 0.47498873508081924
```

We apply grid search to find the best model and the best alpha parameter.

```
In [42]: from sklearn.metrics import make_scorer
from sklearn.model_selection import GridSearchCV

parameters = [{"model__alpha": np.arange(0, 0.2, 0.01)}]

scoring_func = make_scorer(mean_squared_error)
grid_search = GridSearchCV(
    estimator=ridge_pipe, param_grid=parameters, scoring=scoring_func,
    cv=10, n_jobs=-1
)
grid_search = grid_search.fit(X_train, y_train)
```

In the cell below, we display the alpha values that we intend to tune. All rights reserved WQU WorldQuant University QQQQ

```
In [43]: grid_search.cv_results_["params"]
```

```
Out[43]: [{'model__alpha': np.float64(0.0)},
          {'model__alpha': np.float64(0.01)},
          {'model__alpha': np.float64(0.02)},
          {'model__alpha': np.float64(0.03)},
          {'model__alpha': np.float64(0.04)},
          {'model__alpha': np.float64(0.05)},
          {'model__alpha': np.float64(0.06)},
          {'model__alpha': np.float64(0.07)},
          {'model__alpha': np.float64(0.08)},
          {'model__alpha': np.float64(0.09)},
          {'model__alpha': np.float64(0.1)},
          {'model__alpha': np.float64(0.11)},
          {'model__alpha': np.float64(0.12)},
          {'model__alpha': np.float64(0.13)},
          {'model__alpha': np.float64(0.14)},
          {'model__alpha': np.float64(0.15)},
          {'model__alpha': np.float64(0.16)},
          {'model__alpha': np.float64(0.17)},
          {'model__alpha': np.float64(0.18)},
          {'model__alpha': np.float64(0.19)}]
```

As discussed in the model evaluation section, we will evaluate the performance of ridge regression on the dataset.

```
In [44]: from sklearn.linear_model import Lasso, LinearRegression
          from sklearn.model_selection import (
              GridSearchCV,
              KFold,
              RepeatedKFold,
              cross_validate,
              train_test_split,
          )
          from sklearn.preprocessing import PolynomialFeatures, StandardScaler
```

We apply linear regression in the dataset and generate prediction on the test set.

```
In [45]: lreg = LinearRegression()
          lreg.fit(X_train, y_train)

          lreg_y_pred = lreg.predict(X_test)

          # calculating Mean Squared Error (MSE)
          mean_squared_error = np.mean((lreg_y_pred - y_test) ** 2)
          print("Mean squared Error on test set : ", mean_squared_error)

          # Putting together the coefficient and their corresponding variable names
          lreg_coefficient = pd.DataFrame()
          lreg_coefficient["Columns"] = X_train.columns
          lreg_coefficient["Coefficient Estimate"] = pd.Series(lreg.coef_)
          print(lreg_coefficient)
```


Mean squared Error on test set : 0.5385446422423643

	Columns	Coefficient Estimate
0	MedInc	4.411819e-01
1	HouseAge	9.707308e-03
2	AveRooms	-1.199939e-01
3	AveBedrms	7.847091e-01
4	Population	-3.394667e-07
5	AveOccup	-3.282391e-03
6	Latitude	-4.236797e-01
7	Longitude	-4.393118e-01

In the next cell, we display the feature coefficients.

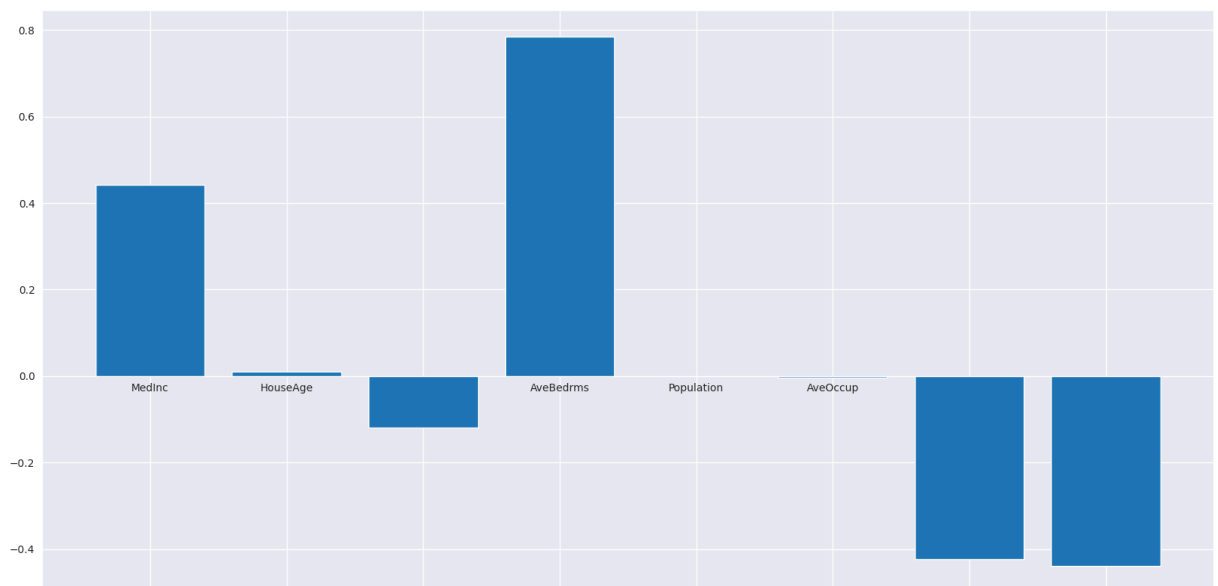
```
In [46]: # plotting the coefficient score
fig, ax = plt.subplots(figsize=(20, 10))

ax.bar(
    lreg_coefficient["Columns"],
    lreg_coefficient["Coefficient Estimate"],
)

ax.spines["bottom"].set_position("zero")

plt.style.use("ggplot")
plt.suptitle(
    "Fig. 9: Coefficients Scores.", fontweight="bold",
    horizontalalignment="right"
)
plt.show()
```

Fig. 9: Coefficients Scores.



```
In [47]: # import ridge regression from `sklearn` library
from sklearn.linear_model import Ridge

# Train the model
ridgeR = Ridge(alpha=1)
ridgeR.fit(X_train, y_train)
y_pred = ridgeR.predict(X_test)

# calculate mean square error
mean_squared_error_ridge = np.mean((y_pred - y_test) ** 2)
print(mean_squared_error_ridge)

# get ridge coefficient and print them
ridge_coefficient = pd.DataFrame()
ridge_coefficient["Columns"] = X_train.columns
ridge_coefficient["Coefficient Estimate"] = pd.Series(ridgeR.coef_)
print(ridge_coefficient)
```

```
0.538464993251617
   Columns  Coefficient Estimate
0    MedInc      4.409897e-01
1   HouseAge      9.709428e-03
2   AveRooms    -1.196302e-01
3   AveBedrms      7.826149e-01
4  Population    -3.331482e-07
5   AveOccup    -3.281877e-03
6   Latitude    -4.236763e-01
7  Longitude    -4.392840e-01
```

2. Lasso Regression

Lasso (Least Absolute Shrinkage and Selection Operator) is another regularization model that adds an $L1$ -term to the objective function. It works by equating irrelevant parameters to zero. The loss function of a lasso regression is given by

$$L = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2 + \frac{\lambda}{2} ||w||$$

Just like in ridge regression, λ controls the penalty's strength. Just like ridge regression above, when $\lambda = 0$, we have a linear regression equation. As the value of λ increases shrinkage to unimportant features and some will be equal to zero, the variance of the model reduces and bias increases.

In the cells below, we demonstrate how to apply lasso on our dataset.

```
In [48]: steps = [("scalar", StandardScaler()), ("model", Lasso())]

lasso_pipe = Pipeline(steps)
```

We apply cross-validation and print out the mean absolute error and the best alpha value.

```
In [49]: cv = RepeatedKfold(n_splits=10, n_repeats=3, random_state=1)
model__alpha = np.linspace(0.1, 2, 21)
lasso = Lasso()
grid = dict()
grid["model__alpha"] = model__alpha
gscv = GridSearchCV(
    lasso_pipe, grid, scoring="neg_mean_absolute_error", cv=cv, n_jobs=-1
)
results = gscv.fit(X, y)
print("MAE: %.5f" % results.best_score_)
print("Config: %s" % results.best_params_)
```

MAE: -0.62043

Config: {'model__alpha': np.float64(0.1)}

We can now import the libraries and compute the model metric below.

```
In [50]: from sklearn.linear_model import Lasso

# Train the model
lasso = Lasso(alpha=1)
lasso.fit(X_train, y_train)
y_pred1 = gscv.predict(X_test)

# Calculate Mean Squared Error
mean_squared_error = np.mean((y_pred1 - y_test) ** 2)
print("Mean squared error on test set", mean_squared_error)
lasso_coeff = pd.DataFrame()
lasso_coeff["Columns"] = X_train.columns
lasso_coeff["Coefficient Estimate"] = pd.Series(lasso.coef_)

print(lasso_coeff)
```

Mean squared error on test set 0.6745553621321495

	Columns	Coefficient Estimate
0	MedInc	0.144938
1	HouseAge	0.006047
2	AveRooms	0.000000
3	AveBedrms	-0.000000
4	Population	-0.000008
5	AveOccup	-0.000000
6	Latitude	-0.000000
7	Longitude	-0.000000

From the results above, we see the coefficients of features not relevant rounded down to zero.

2.2 Classification Metrics

In the next cells, we are going to discuss and demonstrate different classification metrics on the iris dataset.

```
In [51]: from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split

data, target = load_iris(as_frame=True, return_X_y=True)

X_train, X_test, y_train, y_test = train_test_split(
    data, target, shuffle=True, random_state=0, test_size=0.3
)

model = RandomForestClassifier(random_state=1)
model.fit(X_train, y_train)
target_predicted = model.predict(X_test)
```

Accuracy

We use accuracy as a baseline. Accuracy simply finds the proportion of classes we classified correctly, that is,

```
In [52]: y_test == target_predicted
```

Out[52]:

	target
114	True
62	True
33	True
107	True
7	True
100	True
40	True
86	True
76	True
71	True
134	True
51	True
73	True
54	True
63	True
37	True
78	True
90	True
45	True
16	True
121	True
66	True
24	True
8	True
126	True
22	True
44	True
97	True
93	True
26	True
137	True
84	True
27	True
127	True
132	True

	target
59	True
18	True
83	False
61	True
92	True
112	True
2	True
141	True
43	True
10	True

dtype: bool

To get the accuracy of our model, we get the mean of the number of times we predicted correctly as shown below.

```
In [53]: import numpy as np

np.round(np.mean(y_test == target_predicted), 3)
```

```
Out[53]: np.float64(0.978)
```

This is implemented in `sklearn` using the `accuracy_score` as shown below.

```
In [54]: from sklearn.metrics import accuracy_score

accuracy = accuracy_score(y_test, target_predicted)
print(f"Accuracy: {accuracy:.3f}")
```

```
Accuracy: 0.978
```

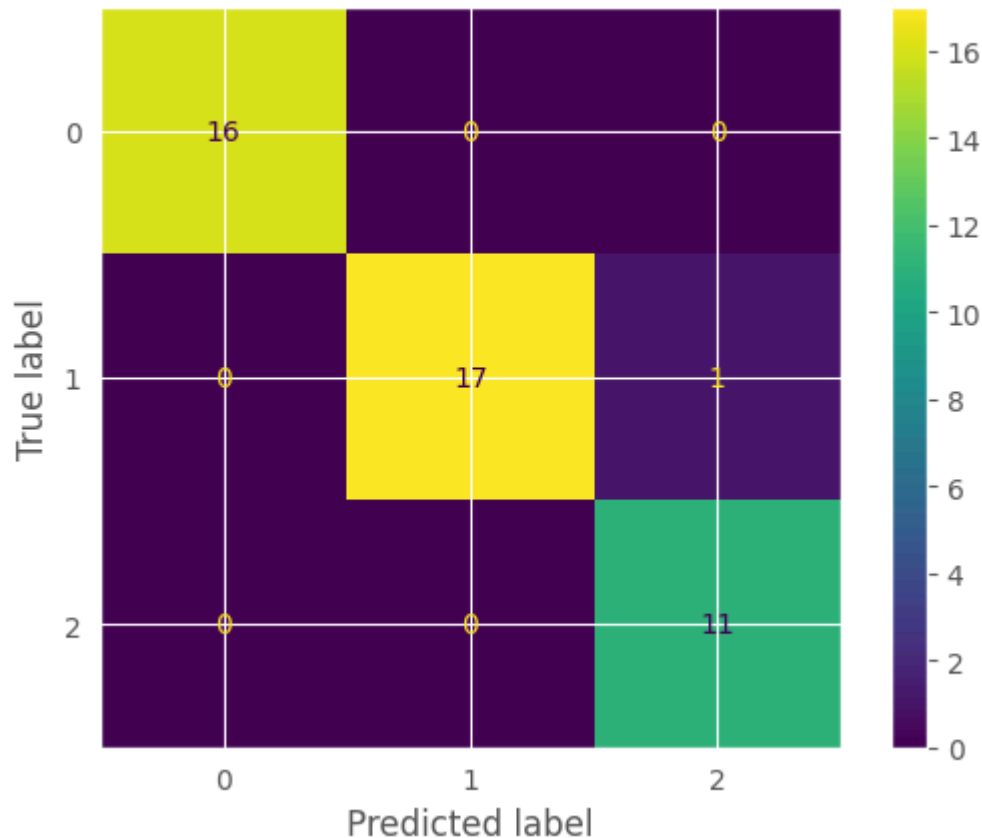
Confusion Matrix

Accuracy might not be a good measure of a model performance, especially when the target variable is imbalanced. The most common class will always have more weight. To get the finer details of the model performance, it is recommended to construct a confusion matrix to independently determine what error is for each class. The cell below shows how to construct a confusion matrix

```
In [55]: from sklearn.metrics import ConfusionMatrixDisplay

_ = ConfusionMatrixDisplay.from_estimator(model, X_test, y_test)
plt.suptitle(
    "Fig. 10: Confusion Matrix.", fontweight="bold",
    horizontalalignment="right"
)
plt.show()
```

Fig. 10: Confusion Matrix.



The diagonal entries represent the correct predictions and the off-diagonal entries are the incorrect predictions. For more on confusion matrices, we ask students to read [A Review on Evaluation Metrics for Data Classification Evaluations](#).

3. Conclusion

In this lesson we discussed simulated annealing as one of the techniques for hyperparameter tuning and also introduced objective functions in machine learning. In the next lesson we are going to see how we can apply Bayesian hyperparameter tuning in a bitcoin trading strategy.

References

1. Ampatzis, Christos, and Dario Izzo. "Machine Learning Techniques for Approximation of Objective Functions in Trajectory Optimisation." Proceedings of the IJCAI-09 workshop on Artificial Intelligence in Space, 2009.
2. Botchkarev, Alexei. "Evaluating Performance of Regression Machine Learning Models Using Multiple Error Metrics in Azure Machine Learning Studio." 2018.
3. Hossin, Mohammad, and Md Nasir Sulaiman. "A Review on Evaluation Metrics for Data Classification Evaluations." *International Journal of Data Mining & Knowledge Management Process*, vol 5, no. 2, 2015.

Required Readings

Required Readings for this program are open access, which means you should be able to access them at no cost. The link provided in the citation will take you directly to the reading or to a page where you can download it.

1. Grandini, Margherhita. "Metrics for Multi-Class Classification: An Overview." *arXiv*, 2020, <https://arxiv.org/abs/2008.05756>.

- **Required pages:** Sections 1 and 2
- **Estimated reading time:** 10 minutes

2. Attia, Joseph. "Evaluating the Effectiveness of Common Trading Models." *arXiv*, 2019, <https://arxiv.org/ftp/arxiv/papers/1907/1907.10407.pdf>.

- **Required pages:** Section 5
- **Estimated reading time:** 60 minutes

MACHINE LEARNING IN FINANCE

MODULE 7 | LESSON 4

BITCOIN TRADING STRATEGY

Reading Time	30 minutes
Prior Knowledge	Hyperparameter tuning, supervised machine learning, classification algorithms
Keywords	Bayesian optimization, signal

In the last lesson, we discussed objective functions and extended our knowledge on hyperparameter tuning. In this lesson, we will train classification algorithms on a Bitcoin trading strategy problem and improve the performance of one machine learning model by tuning its parameters using grid search and Bayesian optimization techniques.

1. Introduction

In this problem we want to predict when to buy or sell Bitcoin. We will start by defining a *buy or sell signal* that will be represented by 1 or 0. We arrive at the signal by comparing the price trend of short-term and long-term behavior, that is, a short-term moving average greater than a long-term moving average. Then, we buy bitcoin; otherwise, we sell bitcoin. This is, therefore, a classification problem where we are interested in getting the direction of Bitcoin movement right. We will split our dataset into train and validation sets in the ratio of 4:1.

2. Loading Helper Packages and Data

2.1 Loading Helper Packages

We load the Python packages that will assist us in achieving our objective.

```
In [1]: !pip install --upgrade yfinance
```

Requirement already satisfied: yfinance in /usr/local/lib/python3.11/site-packages (1.5.1)

Requirement already satisfied: pandas>=1.3.0 in /usr/local/lib/python3.11/site-packages (from yfinance) (2.2.3)

Requirement already satisfied: numpy>=1.16.5 in /usr/local/lib/python3.11/site-packages (from yfinance) (2.1.3)

Requirement already satisfied: requests>=2.31 in /usr/local/lib/python3.11/site-packages (from yfinance) (2.32.3)

Requirement already satisfied: multitasking>=0.0.7 in /usr/local/lib/python3.11/site-packages (from yfinance) (0.0.11)

Requirement already satisfied: platformdirs>=2.0.0 in /usr/local/lib/python3.11/site-packages (from yfinance) (4.3.6)

Requirement already satisfied: pytz>=2022.5 in /usr/local/lib/python3.11/site-packages (from yfinance) (2025.1)

Requirement already satisfied: peewee>=3.16.2 in /usr/local/lib/python3.11/site-packages (from yfinance) (3.18.2)

Requirement already satisfied: beautifulsoup4>=4.11.1 in /usr/local/lib/python3.11/site-packages (from yfinance) (4.13.3)

Requirement already satisfied: curl_cffi>=0.15 in /usr/local/lib/python3.11/site-packages (from yfinance) (0.15.0)

Requirement already satisfied: protobuf>=3.19.0 in /usr/local/lib/python3.11/site-packages (from yfinance) (5.29.5)

Requirement already satisfied: websockets>=13.0 in /usr/local/lib/python3.11/site-packages (from yfinance) (16.1)

Requirement already satisfied: soupsieve>1.2 in /usr/local/lib/python3.11/site-packages (from beautifulsoup4>=4.11.1->yfinance) (2.6)

Requirement already satisfied: typing-extensions>=4.0.0 in /usr/local/lib/python3.11/site-packages (from beautifulsoup4>=4.11.1->yfinance) (4.12.2)

Requirement already satisfied: cffi>=2.0.0 in /usr/local/lib/python3.11/site-packages (from curl_cffi>=0.15->yfinance) (2.1.0)

Requirement already satisfied: certifi>=2024.2.2 in /usr/local/lib/python3.11/site-packages (from curl_cffi>=0.15->yfinance) (2025.1.31)

Requirement already satisfied: rich in /usr/local/lib/python3.11/site-packages (from curl_cffi>=0.15->yfinance) (14.0.0)

Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.11/site-packages (from pandas>=1.3.0->yfinance) (2.9.0.post0)

Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.11/site-packages (from pandas>=1.3.0->yfinance) (2025.1)

Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.11/site-packages (from requests>=2.31->yfinance) (3.4.1)

Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.11/site-packages (from requests>=2.31->yfinance) (3.10)

Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.11/site-packages (from requests>=2.31->yfinance) (2.3.0)

Requirement already satisfied: pycparser in /usr/local/lib/python3.11/site-packages (from cffi>=2.0.0->curl_cffi>=0.15->yfinance) (2.22)

Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.11/site-packages (from python-dateutil>=2.8.2->pandas>=1.3.0->yfinance) (1.17.0)

Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.11/site-packages (from rich->curl_cffi>=0.15->yfinance) (3.0.0)

Requirement already satisfied: pygments<3.0.0,>=2.13.0 in
/usr/local/lib/python3.11/site-packages (from rich->curl_cffi>=0.15-
>yfinance) (2.19.1)

Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.11/site-
packages (from markdown-it-py>=2.2.0->rich->curl_cffi>=0.15->yfinance)
(0.1.2)

WARNING: Running pip as the 'root' user can result in broken permissions and
conflicting behaviour with the system package manager, possibly rendering
your system unusable. It is recommended to use a virtual environment
instead: <https://pip.pypa.io/warnings/venv>. Use the --root-user-action
option if you know what you are doing and want to suppress this warning.

[notice] A new release of pip is available: 25.0.1 -> 26.1.2

[notice] To update, run: `pip install --upgrade pip`

```
In [2]: import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns

sns.set()
```

```
In [3]: from time import time

import joblib

# display all columns
pd.set_option("display.max_columns", None)
pd.set_option("display.max_rows", None)

from bayes_opt import BayesianOptimization

# Libraries for Deep Learning Models
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.dummy import DummyClassifier
from sklearn.ensemble import (
    AdaBoostClassifier,
    GradientBoostingClassifier,
    RandomForestClassifier,
)
from sklearn.linear_model import LogisticRegression

# needed for `HistGradientBoostingClassifier`
from sklearn.metrics import (
    accuracy_score,
    classification_report,
    confusion_matrix,
    f1_score,
    precision_score,
    recall_score,
)
from sklearn.model_selection import (
    GridSearchCV,
    KFold,
    StratifiedKFold,
    cross_val_score,
    cross_validate,
    train_test_split,
)
from sklearn.naive_bayes import GaussianNB
from sklearn.neighbors import KNeighborsClassifier
from sklearn.neural_network import MLPClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.tree import DecisionTreeClassifier

# from `keras.optimizers` import SGD
```

2.2 Data Loading

We fetch Bitcoin data price from Yahoo finance for the last five years.

```
In [4]: # To fetch financial data
import yfinance as yf

# Set the ticker as 'BTC-USD'
BTC_Ticker = yf.Ticker("BTC-USD")
BTC_Data = BTC_Ticker.history(period="5y")
```

```
In [5]: # BTC_Ticker = yf.Ticker("BTC-USD")
# BTC_Data = BTC_Ticker.history(period="max")
BTC_Data.head()
```

```
Out[5]:
```

	Open	High	Low	Close	Volume	Dividen ds	Stock Splits
Date							
2021-07-14 00:00:00+00:00	32723.845703	33061.398438	31639.125000	32822.347656	21376531210	0.0	0.0
2021-07-15 00:00:00+00:00	32827.875000	33159.640625	31175.708984	31780.730469	21300524237	0.0	0.0
2021-07-16 00:00:00+00:00	31841.550781	32218.406250	31100.673828	31421.539062	23699476918	0.0	0.0
2021-07-17 00:00:00+00:00	31397.308594	31935.945312	31223.990234	31533.068359	18895018942	0.0	0.0
2021-07-18 00:00:00+00:00	31533.884766	32398.996094	31215.492188	31796.810547	18787986667	0.0	0.0

3. Exploratory Data Analysis (EDA)

This stage explores the data to help us see trends in the data.

```
In [6]: BTC_Data.shape
```

```
Out[6]: (1826, 7)
```

```
In [7]: BTC_Data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
DatetimeIndex: 1826 entries, 2021-07-14 00:00:00+00:00 to 2026-07-14
00:00:00+00:00
Data columns (total 7 columns):
#   Column          Non-Null Count  Dtype
---  -
0   Open             1826 non-null   float64
1   High             1826 non-null   float64
2   Low              1826 non-null   float64
3   Close            1826 non-null   float64
4   Volume           1826 non-null   int64
5   Dividends        1826 non-null   float64
6   Stock Splits     1826 non-null   float64
dtypes: float64(6), int64(1)
memory usage: 114.1 KB
```

```
In [8]: BTC_Data.describe()
```

```
Out[8]:
```

	Open	High	Low	Close	Volume	Dividen ds	Stock Splits
count	1826.00 0000	1826.00 0000	1826.00 0000	1826.00 0000	1.82600 0e+03	1826.0	1826.0
mean	57408.0 60898	58438.1 27165	56322.0 93694	57423.1 80282	3.49122 1e+10	0.0	0.0
std	29776.7 17769	30200.8 90031	29302.9 85172	29771.7 23469	2.04341 2e+10	0.0	0.0
min	15782.3 00781	16253.0 47852	15599.0 46875	15787.2 84180	5.33117 3e+09	0.0	0.0
25%	29410.1 85059	29988.9 02344	29068.8 35938	29410.3 23730	2.07681 7e+10	0.0	0.0
50%	55594.0 99609	57517.4 96094	54118.7 65625	55941.7 12891	3.07381 1e+10	0.0	0.0
75%	78685.3 84766	80978.5 19531	77330.7 79297	78680.9 58984	4.26321 1e+10	0.0	0.0
max	124752. 140625	126198. 070312	123196. 046875	124752. 531250	1.81746 4e+11	0.0	0.0

```
In [9]: # Checking for any null values and removing the null values'''
print("Null Values =", BTC_Data.isnull().values.any())
```

```
Null Values = False
```

4. Data Preparation

We start by creating our target variable, that is, the buy or sell signal. The target variable constitutes our trading strategy. When the shorter term moving average goes above the longer term moving average, then it is an indicator to buy and the vice versa is also true. Read more here: [Picking buy-sell signals: A practitioner's perspective on key technical indicators for selected Indian firms.](#)

```
In [10]: # Create short simple moving average over the short window
BTC_Data["short_moving_avg"] = (
    BTC_Data["Close"].rolling(window=10, min_periods=1,
center=False).mean()
)

# Create long simple moving average over the long window
BTC_Data["long_maving_avg"] = (
    BTC_Data["Close"].rolling(window=60, min_periods=1,
center=False).mean()
)

# Create signals
BTC_Data["signal"] = np.where(
    BTC_Data["short_moving_avg"] > BTC_Data["long_maving_avg"], 1.0, 0.0
)
```

```
In [11]: BTC_Data.head()
```


Out[11]:

	Open	High	Low	Close	Volume	Dividends	Stock Splits	short_moving_avg	long_moving_avg	signal
Date										
2021-07-14 00:00:00+00:00	3272.3845703	33061.398438	31639.125000	32822.347656	21376531210	0.0	0.0	32822.347656	32822.347656	0.0
2021-07-15 00:00:00+00:00	32827.875000	33159.640625	31175.708984	31780.730469	21300524237	0.0	0.0	32301.539062	32301.539062	0.0
2021-07-16 00:00:00+00:00	31841.550781	32218.406250	31100.673828	31421.539062	23699476918	0.0	0.0	32008.205729	32008.205729	0.0
2021-07-17 00:00:00+00:00	31397.308594	31935.945312	31223.990234	31533.068359	18895018942	0.0	0.0	31889.421387	31889.421387	0.0
2021-07-18 00:00:00+00:00	31533.884766	32398.996094	31215.492188	31796.810547	18787986667	0.0	0.0	31870.899219	31870.899219	0.0

4.1 Feature Engineering

We now create additional features in our dataset that will help improve our model performance. The additional features are:

1. Exponential Moving Average: This gives us the price trend of the data.
2. Relative Strength Indicator (RSI): RSI measures the change in prices in a recent time frame.
3. Rate of Change: This measures the percentage change between the stock's current price and past prices.
4. Stochastic Oscillator: This compares the current closing price of a stock with its previous closing prices.

In [12]: *# calculation of exponential moving average*

```
def EMA(df, n):  
    EMA = pd.Series(df["Close"].ewm(span=n, min_periods=n).mean(),  
name="EMA_" + str(n))  
    return EMA
```

```
BTC_Data["EMA10"] = EMA(BTC_Data, 10)  
BTC_Data["EMA30"] = EMA(BTC_Data, 30)  
BTC_Data["EMA200"] = EMA(BTC_Data, 200)  
BTC_Data.head()
```

calculation of rate of change

```
def RC(df, n):  
    M = df.diff(n - 1)  
    N = df.shift(n - 1)  
    RC = pd.Series(((M / N) * 100), name="RC_" + str(n))  
    return RC
```

```
BTC_Data["RC10"] = RC(BTC_Data["Close"], 10)  
BTC_Data["RC30"] = RC(BTC_Data["Close"], 30)
```

Calculation of price momentum

```
def MOM(df, n):  
    MOM = pd.Series(df.diff(n), name="Momentum_" + str(n))  
    return MOM
```

```
BTC_Data["MOM10"] = MOM(BTC_Data["Close"], 10)  
BTC_Data["MOM30"] = MOM(BTC_Data["Close"], 30)
```

calculation of relative strength index

```
def RSI(series, period):  
    delta = series.diff().dropna()  
    u = delta * 0  
    d = u.copy()  
    u[delta > 0] = delta[delta > 0]  
    d[delta < 0] = -delta[delta < 0]  
    u[u.index[period - 1]] = np.mean(u[:period]) # first value is sum of  
avg gains  
    u = u.drop(u.index[: (period - 1)])  
    d[d.index[period - 1]] = np.mean(d[:period]) # first value is sum of  
avg losses  
    d = d.drop(d.index[: (period - 1)])  
    rs = (  
        u.ewm(com=period - 1, adjust=False).mean()  
        / d.ewm(com=period - 1, adjust=False).mean()  
    )  
    return 100 - 100 / (1 + rs)
```

```
BTC_Data["RSI10"] = RSI(BTC_Data["Close"], 10)  
BTC_Data["RSI30"] = RSI(BTC_Data["Close"], 30)  
BTC_Data["RSI200"] = RSI(BTC_Data["Close"], 200)
```

```

# calculation of stochastic oscillator.

def STOK(close, low, high, n):
    STOK = (
        (close - low.rolling(n).min()) / (high.rolling(n).max() -
low.rolling(n).min())
    ) * 100
    return STOK

def STOD(close, low, high, n):
    STOK = (
        (close - low.rolling(n).min()) / (high.rolling(n).max() -
low.rolling(n).min())
    ) * 100
    STOD = STOK.rolling(3).mean()
    return STOD

BTC_Data["%K10"] = STOK(BTC_Data["Close"], BTC_Data["Low"],
BTC_Data["High"], 10)
BTC_Data["%D10"] = STOD(BTC_Data["Close"], BTC_Data["Low"],
BTC_Data["High"], 10)
BTC_Data["%K30"] = STOK(BTC_Data["Close"], BTC_Data["Low"],
BTC_Data["High"], 30)
BTC_Data["%D30"] = STOD(BTC_Data["Close"], BTC_Data["Low"],
BTC_Data["High"], 30)
BTC_Data["%K200"] = STOK(BTC_Data["Close"], BTC_Data["Low"],
BTC_Data["High"], 200)
BTC_Data["%D200"] = STOD(BTC_Data["Close"], BTC_Data["Low"],
BTC_Data["High"], 200)

```

We can display the dataframe to look at the features we have just feature-engineered.

```
In [13]: BTC_Data.tail()
```

Out[13]:

Open	High	Low	Close	Volume	Dividends	Stock Splits	Short-moving_avg	Long-moving_avg	Signal	EMA10	EMA30	EMA200	RC10	RC30	MO10	MO30	RSI10	RSI30	RSI200	%K10	%D10	%K30	%D30	%K200	%D200
2026-07-09 00:00:00+00:00	66.2257316	66.3422457316	66.139006	2654318	0.0	0	62197450	67837825	0.0	622523425738	634381237413	751559309	7.99	2.83784	3054.7631	1549.3688	54.016038	44.851678	46.51866	79.49069	75.431704	54.40776	1357638	12.84338	
2026-07-10 00:00:00+00:00	63.2194359	64.6589	62.9020	241270	0.0	0	627539715	675429421	0.0	6228169874	634826913	75046180	6.87061	0.89068	556.2853	2677.8513	58.230483	46.751864	46.73086	92.304840	79.2138415	67.147205	57.312053	15903575	13.57348

00:

```
:  
0  
0  
+  
0  
0  
:  
0  
0
```

In the next cell, we calculate the moving average.

```
In [14]: def MA(df, n):  
          MA = pd.Series(df["Close"].rolling(n, min_periods=n).mean(),  
                          name="MA_" + str(n))  
          return MA  
  
BTC_Data["MA21"] = MA(BTC_Data, 10)  
BTC_Data["MA63"] = MA(BTC_Data, 30)  
BTC_Data["MA252"] = MA(BTC_Data, 200)  
BTC_Data.tail()
```

Open	High	Low	Close	Volume	Dividends	Stock_Splits	Short_moving_avg	Long_moving_avg	Signal	EMA10	EMA30	EMA200	RC10	RC30	MOM10	MOM30	RSI10	RSI30	RSI200	%K10	%D10	%K30	%D30	%K200	%D200	MA21	MA63	MA252	Date
																													2026-07-09 00:00:00+00:00
62257.316406	63422.914400	61162.52.7316406	633193.52.14843838	265462.5167	0.0	0.0	62197.1450	67835.78125	0.0	62525.731678	63438.1213	75155.93109	7.91389	2.83784	3054.7631	1549.368	54.01648	46.51866	79.4966	75.31704	54.40766	5.35163	13.7138	12.84375	62219.7150	62259.4975	74224.261	2026-07-09 00:00:00+00:00	

[illegible]
$$\begin{array}{ccccccc} & : & 0 & 0 & : & 0 & 0 \\ & 0 & 0 & 0 & 0 & + & 0 \\ & 0 & 0 & 0 & 0 & 0 & : \\ & 0 & 0 & 0 & 0 & 0 & 0 \end{array}$$
[illegible]
$$\begin{array}{ccccccc} & : & 0 & 0 & : & 0 & 0 \\ & 0 & 0 & : & 0 & 0 & + \\ & 0 & 0 & : & 0 & 0 & : \\ & 0 & 0 & : & 0 & 0 & 0 \end{array}$$
[illegible]

00


```
:  
0  
0  
+  
0  
0  
:  
0  
0
```

We can now exclude features that are irrelevant for our prediction task such as the Bitcoin `High`, `Low` and `Open` values. Since we have already defined our trading signals, we drop the short and long moving average features.

```
In [15]: # excluding columns that are not needed for our prediction.
```

```
BTC_Data = BTC_Data.drop(  
    [  
        "High",  
        "Low",  
        "Open",  
        "short_moving_avg",  
        "long_maving_avg",  
        "Dividends",  
        "Stock Splits",  
    ],  
    axis=1,  
)
```

```
In [16]: BTC_Data = BTC_Data.dropna(axis=0)
```

We can now visualize the Bitcoin price trend over the four years as shown in the figure below.

```
In [17]: BTC_Data[["Close"]].plot(grid=True)  
plt.suptitle(  
    "Fig. 1: Bitcoin Price trend", fontweight="bold",  
    horizontalalignment="right"  
)  
plt.show()
```

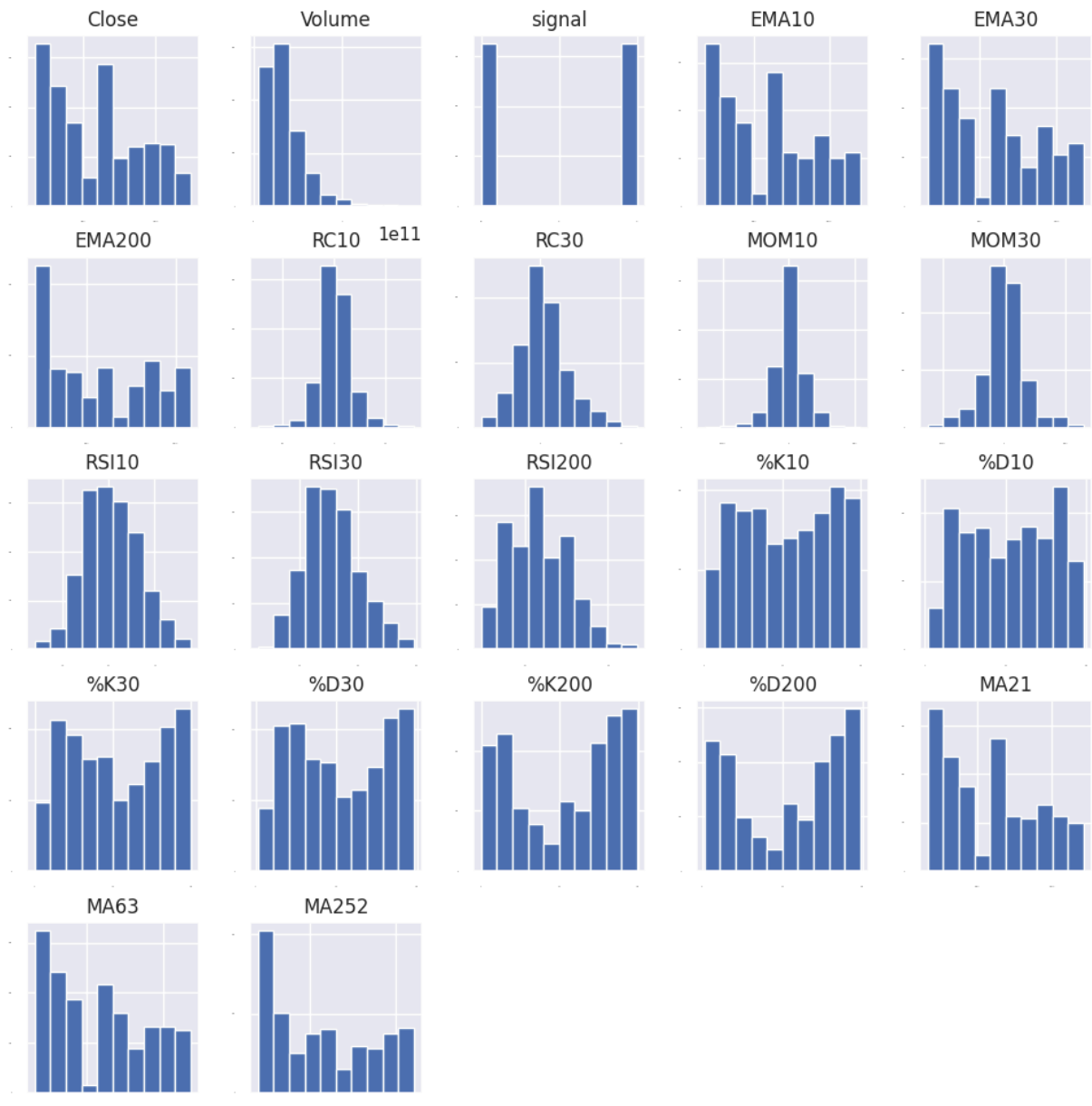
Fig. 1: Bitcoin Price trend



In the cell below, we display the distribution of features that helps the modeler determine how to treat the individual features in our prediction dataset.

```
In [18]: # histograms
BTC_Data.hist(sharex=False, sharey=False, xlabelsize=1, ylabelsize=1,
figsize=(12, 12))
plt.suptitle(
    "Fig. 2: Variable Distribution", fontweight="bold",
    horizontalalignment="right"
)
plt.show()
```

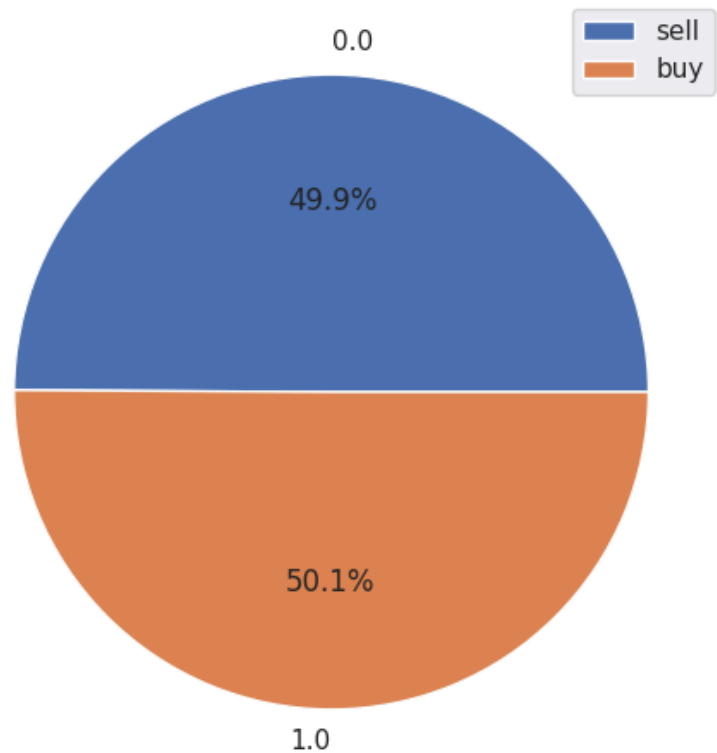
Fig. 2: Variable Distribution



Our interest as stated in the introduction is to predict when to buy or sell a Bitcoin. We draw a pie chart to visualize the proportion of sell to buy signals in our dataset.

```
In [19]: fig = plt.figure()
plot = (
    BTC_Data.groupby("signal")
    .size()
    .plot(kind="pie", figsize=(6, 6), autopct="%1.1f%%", y="Signal")
)
plt.suptitle(
    "Fig. 3: A Pie Chart of buy and sell signal",
    fontweight="bold",
    horizontalalignment="right",
)
plt.legend(["sell", "buy"])
plt.show()
```

Fig. 3: A Pie Chart of buy and sell signal



The count of this signals can also be seen below.

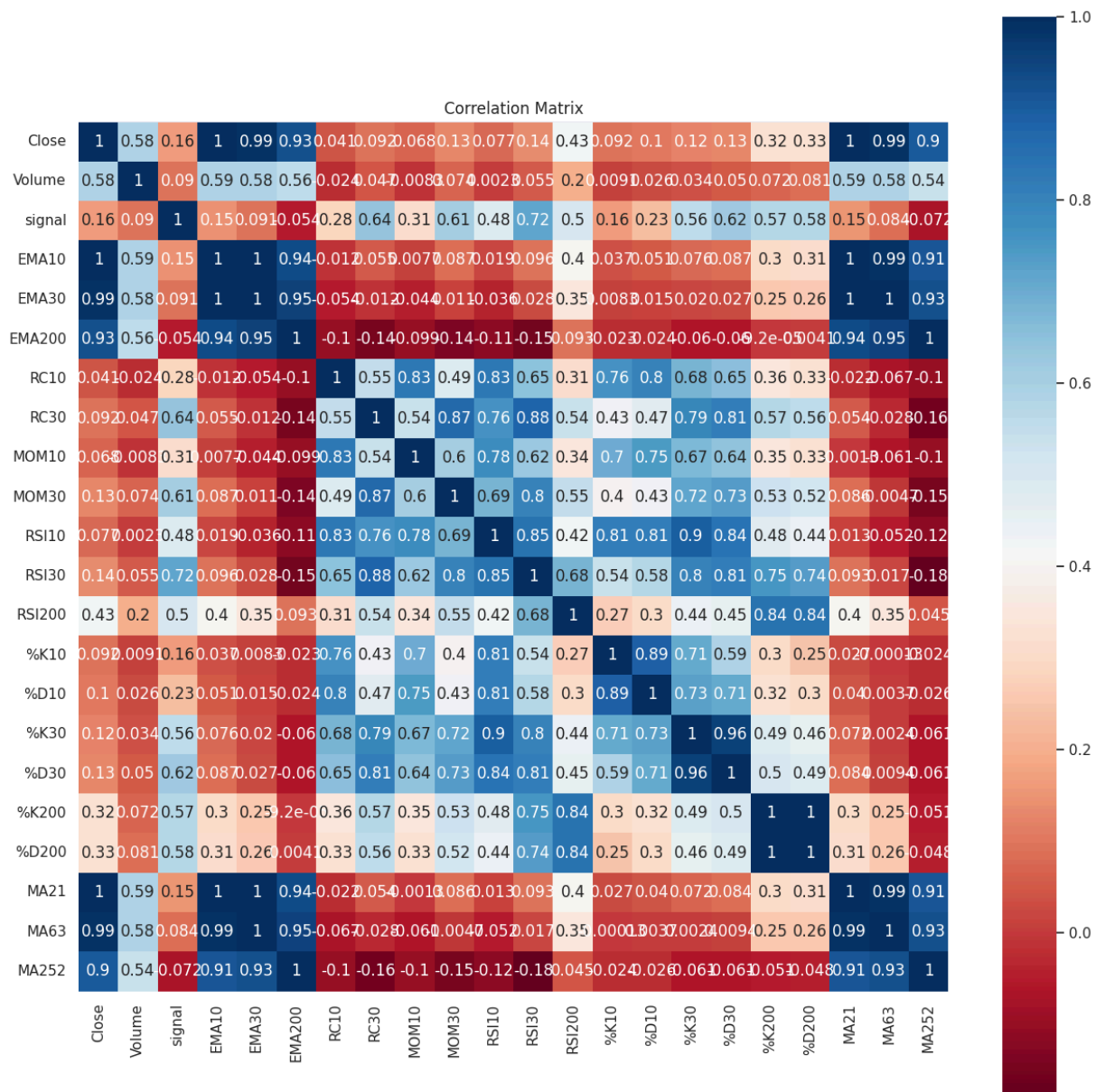
```
In [20]: BTC_Data.groupby(["signal"]).size()
```

```
Out[20]: signal
0.0      811
1.0      814
dtype: int64
```

We can now look at the relationship between our variables using a correlation matrix of the returns.

```
In [21]: # correlation
BTC_data_returns = BTC_Data.shift(1)
correlation = BTC_data_returns.corr()
plt.figure(figsize=(15, 15))
plt.title("Correlation Matrix")
sns.heatmap(correlation, vmax=1, square=True, annot=True, cmap="RdBu")
plt.suptitle(
    "Fig. 4: Correlation Matrix of BITCOIN return",
    fontweight="bold",
    horizontalalignment="right",
)
plt.show()
```

Fig. 4: Correlation Matrix of BITCOIN return



5. Modeling

Now that we have our dataset in place, we split our dataset into a training and a holdout set, in our case the test set that will be used to backtest on our model.

5.1 Train test Split

We start our modeling by splitting the dataset into train and test sets in the ratio of 80:20.

```
In [22]: def get_holdout_set(target, features, period=6):
    idx = pd.IndexSlice
    label = target.name
    dates = np.sort(y.index.get_level_values("Date").unique())
    cv_start, cv_end = dates[0], dates[-period - 2]
    holdout_start, holdout_end = dates[-period - 1], dates[-1]

    df = features.join(target.to_frame())
    train = df.loc[idx[:, cv_start:cv_end], :]
    y_train, X_train = train[label], train.drop(label, axis=1)

    test = df.loc[idx[:, holdout_start:holdout_end], :]
    y_test, X_test = test[label], test.drop(label, axis=1)
    return y_train, X_train, y_test, X_test
```

```
In [23]: subset_validation = BTC_Data[1301:]
```

```
In [24]: subset_validation.shape
```

```
Out[24]: (324, 22)
```

```
In [25]: # split out validation dataset for the end
subset_dataset = BTC_Data[:1300]
y = subset_dataset["signal"]
X = subset_dataset.loc[:, BTC_Data.columns != "signal"]
X.index = pd.to_datetime(X.index)
validation_size = 0.2
seed = 1
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=validation_size, random_state=1
)
```

```
In [26]: subset_dataset.shape
```

```
Out[26]: (1300, 22)
```

We create a **validation** dataset that consists of the remaining data we did not pick in the subset dataset. This validation dataset will only be used in backtesting our models.

```
In [27]: subset_validation = BTC_Data[1300:]
y_validation = subset_validation["signal"]
X_validation = subset_validation.loc[:, BTC_Data.columns != "signal"]
```

```
In [28]: X_validation.shape
```

```
Out[28]: (325, 21)
```

5.1 Baseline Classifier

As discussed in the previous module, it is important to train a baseline classifier that will be used as a baseline to compare with the real classifiers we will use for this task. We will use `sklearn DummyClassifier` as our baseline classifier and choose `strategy=stratified` that predicts the most frequent class.

We start by defining our evaluation metrics.

```
In [29]: # test options for classification
num_folds = 10
seed = 42
scoring = "accuracy"
metrics = {
    "balanced_accuracy": "Accuracy",
    "roc_auc": "AUC",
    "neg_log_loss": "Log Loss",
    "f1_weighted": "F1",
    "precision_weighted": "Precision",
    "recall_weighted": "Recall",
}
```

We define a cross-validation function that will help us evaluate different algorithms.

```
In [30]: def run_cv(clf, X=X, y=y, metrics=metrics, cv=10, fit_params=None,
n_jobs=-1):
    start = time()
    scores = cross_validate(
        estimator=clf,
        X=X,
        y=y,
        scoring=list(metrics.keys()),
        cv=cv,
        return_train_score=True,
        n_jobs=n_jobs,
        verbose=1,
        params=fit_params,
    )
    duration = time() - start
    return scores, duration
```

We also define a function that will help us manipulate the cross-validation output and plot them as will be seen in examples applied in this lesson.

```
In [31]: def stack_results(scores):
    columns = pd.MultiIndex.from_tuples(
        [tuple(m.split("_", 1)) for m in scores.keys()], names=["Dataset",
"Metric"]
    )
    data = np.array(list(scores.values())).T
    df = pd.DataFrame(data=data, columns=columns).iloc[:, 2:]
    results = pd.melt(df, value_name="Value")
    results.Metric = results.Metric.apply(lambda x: metrics.get(x))
    results.Dataset = results.Dataset.str.capitalize()
    return results
```

We finally define a plot function that will display the metrics of our models as box plots so as to allow us to visualize the distribution of the model performance.

```
In [32]: def plot_result(df, model=None, fname=None):
m = list(metrics.values())
g = sns.catplot(
    x="Dataset",
    y="Value",
    hue="Dataset",
    col="Metric",
    data=df,
    col_order=m,
    order=["Train", "Test"],
    kind="box",
    col_wrap=3,
    sharey=False,
    height=4,
    aspect=1.2,
)
df = df.groupby(["Metric", "Dataset"]).Value.mean().unstack().loc[m]
for i, ax in enumerate(g.axes.flat):
    s = f"Train: {df.loc[m[i], 'Train']:>7.4f}\nTest: {df.loc[m[i], 'Test']:>7.4f}"
    ax.text(
        0.05,
        0.85,
        s,
        fontsize=10,
        transform=ax.transAxes,
        bbox=dict(facecolor="white", edgecolor="grey",
boxstyle="round,pad=0.5"),
    )
g.fig.suptitle(model, fontsize=16)
g.fig.subplots_adjust(top=0.9)
if fname:
    g.savefig(fname, dpi=300);
```

We now start by defining the dummy classifier as shown below.

```
In [33]: dummy_clf = DummyClassifier(strategy="stratified", random_state=42)
algo = "dummy_clf"
```

We run the dummy classifier using cross-validation with the defined metrics used to measure model performance.

```
In [34]: fname = f"{algo}.joblib"

dummy_cv_result, run_time_algo = run_cv(dummy_clf)
joblib.dump(dummy_cv_result, fname)
```

```
Out[34]: ['dummy_clf.joblib']
```

We can now create a table with the model performance on each metric as shown below.

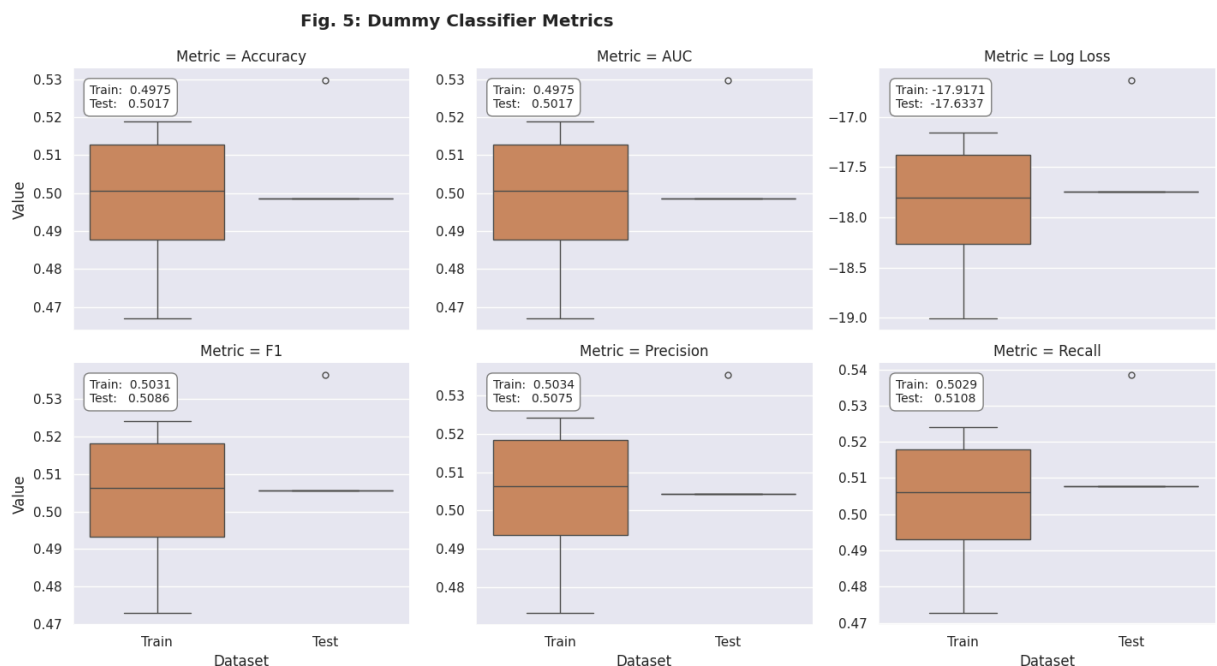
```
In [35]: dummy_result = stack_results(dummy_cv_result)
dummy_result.groupby(["Metric", "Dataset"]).Value.mean().unstack()
```


Out[35]:

	Dataset	Test	Train
Metric			
AUC		0.501676	0.497539
Accuracy		0.501676	0.497539
F1		0.508644	0.503128
Log Loss		-17.633664	-17.917084
Precision		0.507473	0.503368
Recall		0.510769	0.502906

Clearly, the model performance is kind of guess work as the model is half-right nearly all the time with its model metrics showing a very naïve model, as we expected. This performance will act as our baseline, and when building a model, we strive to get a model that outperforms the dummy classifier. In the next cell, we visualize the above metrics using box plots.

```
In [36]: plot_result(dummy_result, model="Dummy Classifier")
plt.suptitle(
    "Fig. 5: Dummy Classifier Metrics", fontweight="bold",
    horizontalalignment="right"
)
plt.show()
```



We can now starting searching for better models that will be used to beat the dummy classifier.

5.2 Defining the Classification Algorithms

We start by defining several models that will be evaluated and we select one of them.

```
In [37]: # spot check the algorithms
models = []
models.append(("LR", LogisticRegression(n_jobs=-1)))
models.append(("LDA", LinearDiscriminantAnalysis()))
models.append(("KNN", KNeighborsClassifier()))
models.append(("CART", DecisionTreeClassifier()))
models.append(("NB", GaussianNB()))
# Neural Network
models.append(("NN", MLPClassifier()))
# Ensemble Models
# Boosting methods
models.append(("AB", AdaBoostClassifier()))
models.append(("GBM", GradientBoostingClassifier()))
# Bagging methods
models.append(("RF", RandomForestClassifier(n_jobs=-1)))
```

We now evaluate these models' performance using `kFold` cross-validation and record their mean and variance.

```
In [38]: results = []
names = []
for name, model in models:
    kfold = KFold(n_splits=num_folds)
    cv_results = cross_val_score(model, X_train, y_train, cv=kfold,
    scoring=scoring)
    results.append(cv_results)
    names.append(name)
    msg = "%s: %f (%f)" % (name, cv_results.mean(), cv_results.std())
    print(msg)
```

```
/usr/local/lib/python3.11/site-  
packages/sklearn/linear_model/_logistic.py:465: ConvergenceWarning: lbfgs  
failed to converge (status=1):  
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT.
```

Increase the number of iterations (max_iter) or scale the data as shown in:
<https://scikit-learn.org/stable/modules/preprocessing.html>

Please also refer to the documentation for alternative solver options:

https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression

```
n_iter_i = _check_optimize_result(  
/usr/local/lib/python3.11/site-  
packages/sklearn/linear_model/_logistic.py:465: ConvergenceWarning: lbfgs  
failed to converge (status=1):  
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT.
```

Increase the number of iterations (max_iter) or scale the data as shown in:
<https://scikit-learn.org/stable/modules/preprocessing.html>

Please also refer to the documentation for alternative solver options:

https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression

```
n_iter_i = _check_optimize_result(  
/usr/local/lib/python3.11/site-  
packages/sklearn/linear_model/_logistic.py:465: ConvergenceWarning: lbfgs  
failed to converge (status=1):  
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT.
```

Increase the number of iterations (max_iter) or scale the data as shown in:
<https://scikit-learn.org/stable/modules/preprocessing.html>

Please also refer to the documentation for alternative solver options:

https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression

```
n_iter_i = _check_optimize_result(  
/usr/local/lib/python3.11/site-  
packages/sklearn/linear_model/_logistic.py:465: ConvergenceWarning: lbfgs  
failed to converge (status=1):  
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT.
```

Increase the number of iterations (max_iter) or scale the data as shown in:
<https://scikit-learn.org/stable/modules/preprocessing.html>

Please also refer to the documentation for alternative solver options:

https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression

```
n_iter_i = _check_optimize_result(  
/usr/local/lib/python3.11/site-  
packages/sklearn/linear_model/_logistic.py:465: ConvergenceWarning: lbfgs  
failed to converge (status=1):  
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT.
```

Increase the number of iterations (max_iter) or scale the data as shown in:
<https://scikit-learn.org/stable/modules/preprocessing.html>

Please also refer to the documentation for alternative solver options:

https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression

```
n_iter_i = _check_optimize_result(  
/usr/local/lib/python3.11/site-  
packages/sklearn/linear_model/_logistic.py:465: ConvergenceWarning: lbfgs  
failed to converge (status=1):  
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT.
```

Increase the number of iterations (max_iter) or scale the data as shown in:
<https://scikit-learn.org/stable/modules/preprocessing.html>
Please also refer to the documentation for alternative solver options:
https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
n_iter_i = _check_optimize_result(
/usr/local/lib/python3.11/site-
packages/sklearn/linear_model/_logistic.py:465: ConvergenceWarning: lbfgs
failed to converge (status=1):
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT.

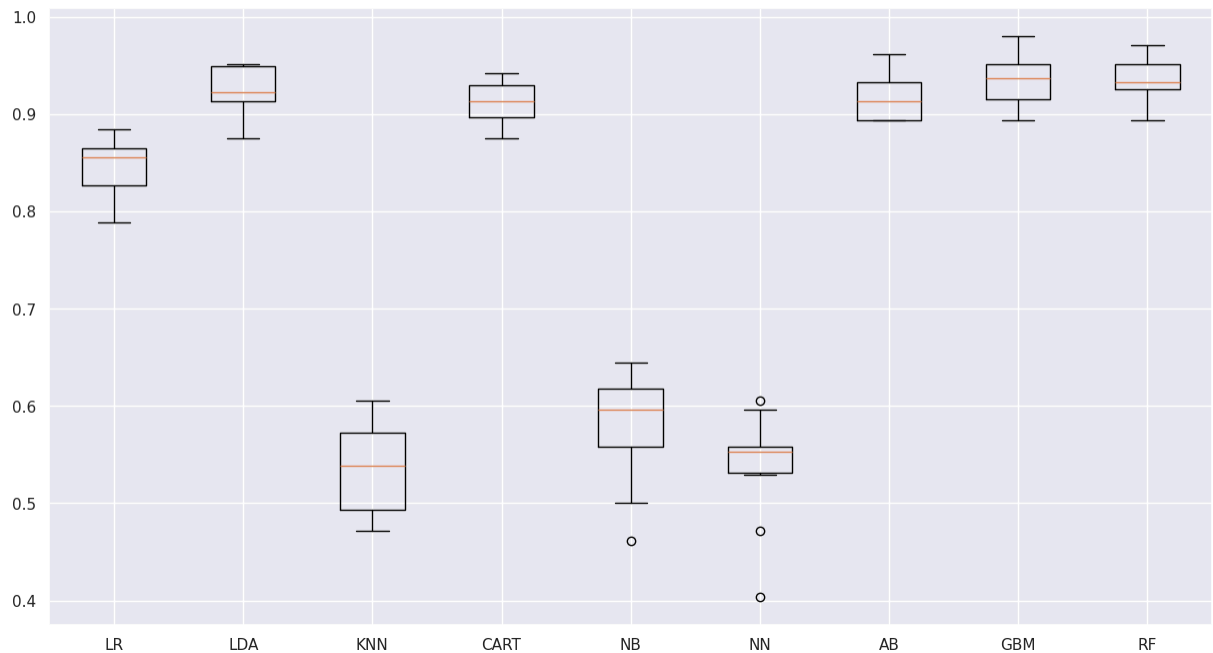
Increase the number of iterations (max_iter) or scale the data as shown in:
<https://scikit-learn.org/stable/modules/preprocessing.html>
Please also refer to the documentation for alternative solver options:
https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
n_iter_i = _check_optimize_result(
/usr/local/lib/python3.11/site-
packages/sklearn/linear_model/_logistic.py:465: ConvergenceWarning: lbfgs
failed to converge (status=1):
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in:
<https://scikit-learn.org/stable/modules/preprocessing.html>
Please also refer to the documentation for alternative solver options:
https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
n_iter_i = _check_optimize_result(
LR: 0.846154 (0.026854)
LDA: 0.925000 (0.023865)
KNN: 0.537500 (0.046602)
CART: 0.911538 (0.021414)
NB: 0.579808 (0.057056)
NN: 0.536538 (0.056363)
AB: 0.916346 (0.021948)
GBM: 0.936538 (0.024777)
RF: 0.937500 (0.023257)

Looking at the outputs recorded above, Random Forest Classifier outperforms the other models, and we select it as our preferred model. We can also visualize the performance of the algorithms in a box plot as shown below.

```
In [39]: # compare algorithms
fig = plt.figure()
ax = fig.add_subplot(111)
plt.boxplot(results)
ax.set_xticklabels(names)
fig.set_size_inches(15, 8)
plt.suptitle(
    "Fig. 6: Algorithm Comparison", fontweight="bold",
    horizontalalignment="right"
)
plt.show()
```

Fig. 6: Algorithm Comparison



6. Hyperparameter Tuning

Since random forest was our best performer, we seek to tune its hyperparameters to improve the performance even further.

6.1 Grid Search

```
In [40]: # Grid Search: Random Forest Classifier

scaler = StandardScaler().fit(X_train)
rescaledX = scaler.transform(X_train)
n_estimators = [20, 80]
max_depth = [5, 10]
criterion = ["gini", "entropy"]
param_grid = dict(n_estimators=n_estimators, max_depth=max_depth,
criterion=criterion)
model = RandomForestClassifier(n_jobs=-1)
kfold = KFold(n_splits=num_folds)
grid = GridSearchCV(estimator=model, param_grid=param_grid,
scoring=scoring, cv=kfold)
grid_result = grid.fit(rescaledX, y_train)

# Print Results
print("Best: %f using %s" % (grid_result.best_score_,
grid_result.best_params_))
means = grid_result.cv_results_["mean_test_score"]
stds = grid_result.cv_results_["std_test_score"]
params = grid_result.cv_results_["params"]
ranks = grid_result.cv_results_["rank_test_score"]
for mean, stdev, param, rank in zip(means, stds, params, ranks):
    print("#%d %f (%f) with: %r" % (rank, mean, stdev, param))
```

```

Best: 0.941346 using {'criterion': 'gini', 'max_depth': 10, 'n_estimators': 80}
#8 0.916346 (0.023177) with: {'criterion': 'gini', 'max_depth': 5, 'n_estimators': 20}
#5 0.923077 (0.026508) with: {'criterion': 'gini', 'max_depth': 5, 'n_estimators': 80}
#4 0.929808 (0.029496) with: {'criterion': 'gini', 'max_depth': 10, 'n_estimators': 20}
#1 0.941346 (0.022936) with: {'criterion': 'gini', 'max_depth': 10, 'n_estimators': 80}
#6 0.919231 (0.027264) with: {'criterion': 'entropy', 'max_depth': 5, 'n_estimators': 20}
#7 0.917308 (0.026227) with: {'criterion': 'entropy', 'max_depth': 5, 'n_estimators': 80}
#2 0.940385 (0.021842) with: {'criterion': 'entropy', 'max_depth': 10, 'n_estimators': 20}
#2 0.940385 (0.025000) with: {'criterion': 'entropy', 'max_depth': 10, 'n_estimators': 80}

```

We can now fix the best hyperparameters to our model and use it to compare with the dummy classifier. We repeat the same steps as in the dummy classifier problem.

```

In [41]: # prepare model
model = RandomForestClassifier(
    criterion="gini", n_estimators=80, max_depth=10, n_jobs=-1
) # `rbf` is default kernel

```

```

In [42]: algo_forest = "random_forest"

```

```

In [43]: fname = f"{algo_forest}.joblib"

random_forest_cv_result, run_time_algo_forest = run_cv(model)
joblib.dump(random_forest_cv_result, fname)

```

```

Out[43]: ['random_forest.joblib']

```

We can now display the model performance table below.

```

In [44]: rf_result = stack_results(random_forest_cv_result)
rf_result.groupby(["Metric", "Dataset"]).Value.mean().unstack()

```

```

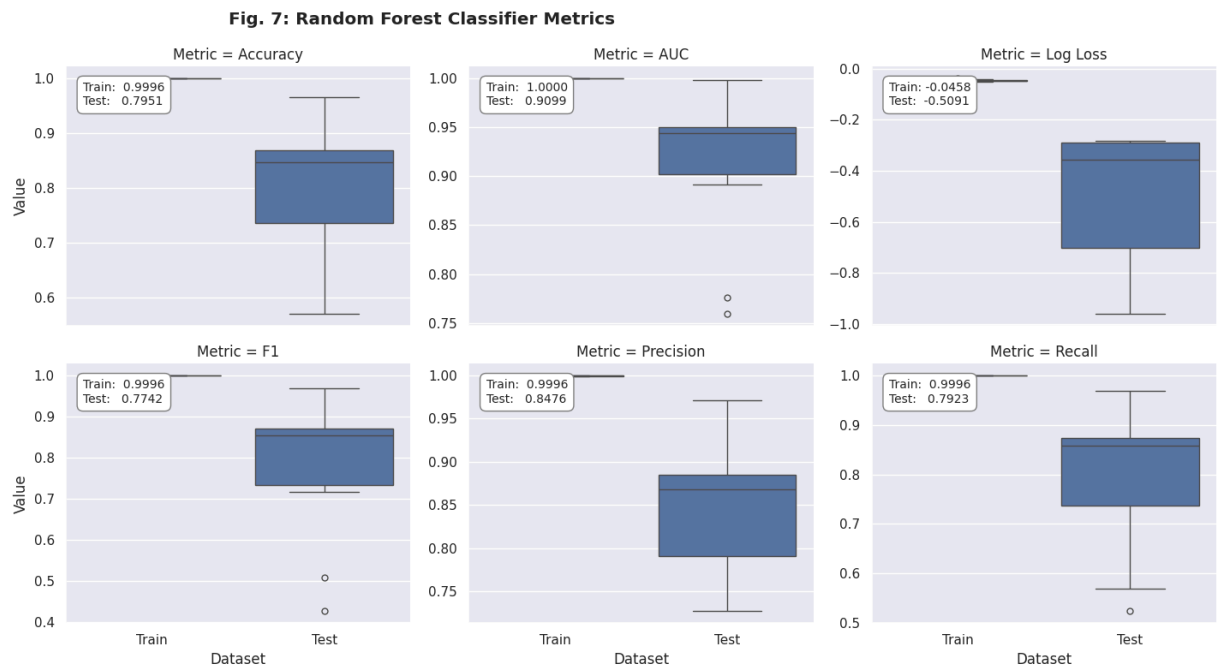
Out[44]:

```

	Dataset	Test	Train
	Metric		
	AUC	0.909938	1.000000
	Accuracy	0.795067	0.999614
	F1	0.774216	0.999573
	Log Loss	-0.509108	-0.045781
	Precision	0.847631	0.999574
	Recall	0.792308	0.999573

The cell below displays the above metrics as box plots.

```
In [45]: plot_result(rf_result, model="Random Forest Classifier")
plt.suptitle(
    "Fig. 7: Random Forest Classifier Metrics",
    fontweight="bold",
    horizontalalignment="right",
)
plt.show()
```



Clearly, random forest classifier outperforms the dummy classifier by far, but we seem to face a problem of overfitting. In the prediction task, we would like to look at ways of mitigating this. The table below shows a comparison of the random forest classifier and the dummy classifier.

```
In [46]: results = {
    "Baseline": dummy_result,
    "Random Forest": rf_result,
}
df = pd.DataFrame()
for model, result in results.items():
    df = pd.concat(
        [
            df,
            result.groupby(["Metric", "Dataset"])
                .Value.mean()
                .unstack()["Test"]
                .to_frame(model),
        ],
        axis=1,
    )

df.T.sort_values("AUC", ascending=False)
```

Out[46]:	Metric	AUC	Accuracy	F1	Log Loss	Precision	Recall
	Random Forest	0.909938	0.795067	0.774216	-0.509108	0.847631	0.792308
	Baseline	0.501676	0.501676	0.508644	-17.633664	0.507473	0.510769

Having settled on random forest, we can now train it on the data and analyze its performance on the holdout dataset.

```
In [47]: model = RandomForestClassifier(
            criterion="gini", n_estimators=80, max_depth=10, n_jobs=-1
        )
        model.fit(X_train, y_train)
        # estimate accuracy on validation set
        predictions = model.predict(X_test)
        print(accuracy_score(y_test, predictions))
        print(confusion_matrix(y_test, predictions))
        print(classification_report(y_test, predictions))
```

```
0.9461538461538461
[[106   6]
 [  8 140]]
```

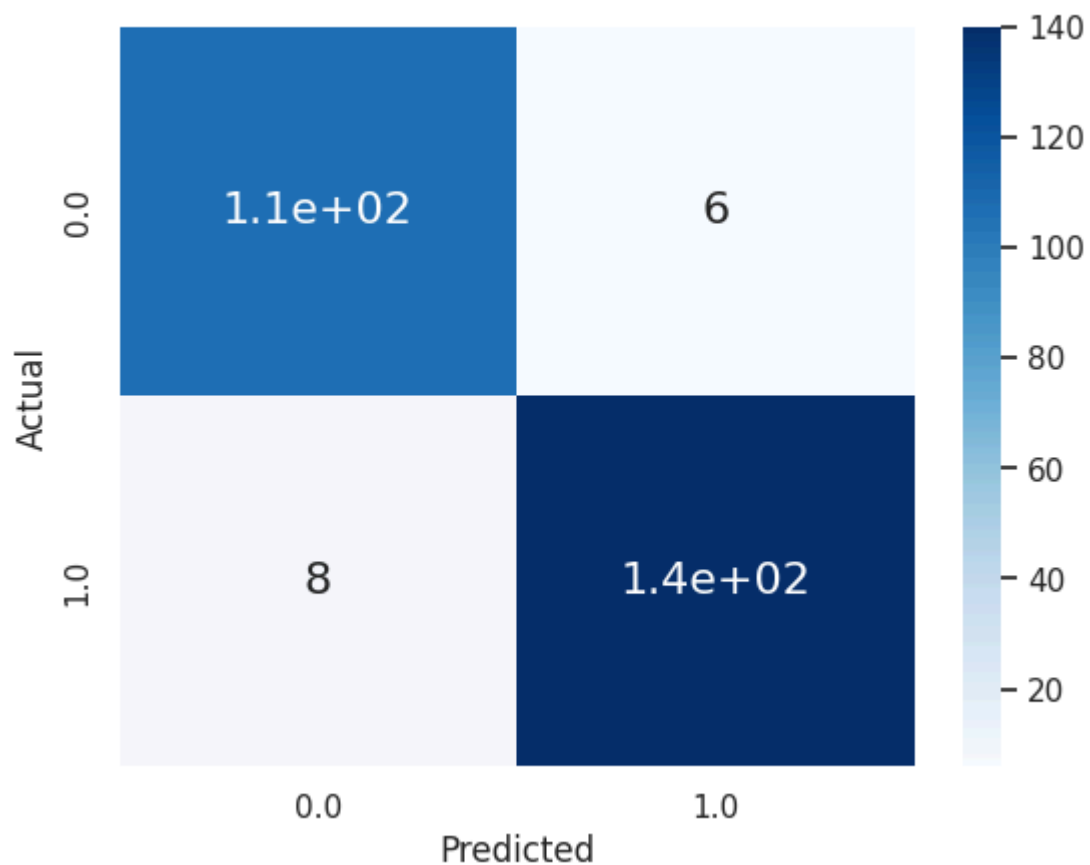
	precision	recall	f1-score	support
0.0	0.93	0.95	0.94	112
1.0	0.96	0.95	0.95	148
accuracy			0.95	260
macro avg	0.94	0.95	0.95	260
weighted avg	0.95	0.95	0.95	260

The model shows a better prediction potential on the holdout dataset with performance being upwards of 94%.All rights reserved WQU WorldQuant University QQQQ

To evaluate the model's performance, we can draw the confusion matrix that allows us to visualize the models' recall, specificity, accuracy, and precision.

```
In [48]: df_cm = pd.DataFrame(
            confusion_matrix(y_test, predictions),
            columns=np.unique(y_test),
            index=np.unique(y_test),
        )
        df_cm.index.name = "Actual"
        df_cm.columns.name = "Predicted"
        sns.heatmap(df_cm, cmap="Blues", annot=True, annot_kws={"size": 16}) #
        font sizes
        plt.suptitle("Fig. 8: Confusion Matrix", fontweight="bold",
            horizontalalignment="right")
        plt.show()
```

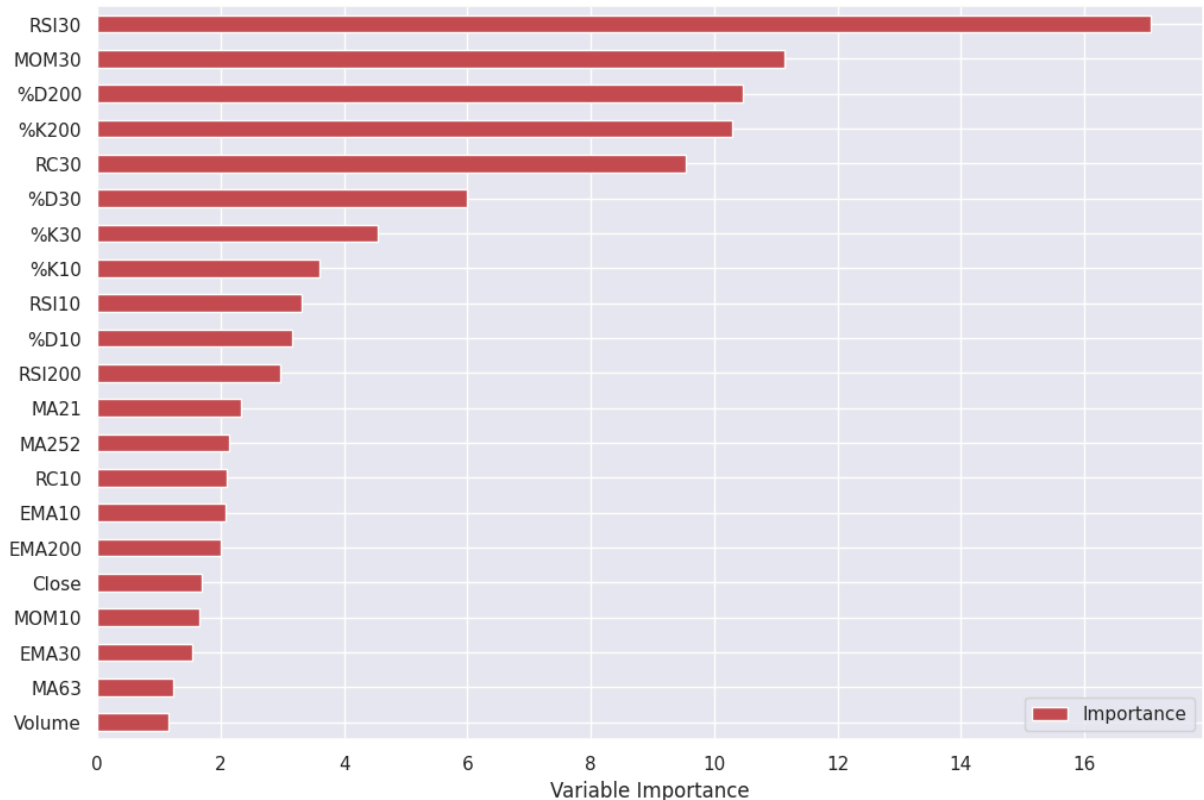

Fig. 8: Confusion Matrix



We determine the variables with more predictive powers as shown below.

```
In [49]: Importance = pd.DataFrame(  
    {"Importance": model.feature_importances_ * 100}, index=X.columns  
)  
Importance.sort_values("Importance", axis=0, ascending=True).plot(  
    kind="barh", color="r", figsize=(12, 8)  
)  
plt.xlabel("Variable Importance")  
plt.suptitle(  
    "Fig. 9: Variable Importance", fontweight="bold",  
    horizontalalignment="right"  
)  
plt.show()
```

Fig. 9: Variable Importance



6.2 Bayesian Optimization

The Bayesian optimization process uses the Gaussian process to model the mean and variance of our objective function. In every iteration, it tries different combinations of hyperparameter values and builds a new model by using the model's past information. This has the advantage of reducing training time to achieve better results.

We start by defining the stratified `kFold` function that will be used to perform cross-validation on the training dataset.

```
In [50]: def stratified_kfold_score(clf, X_train, y_train, n_fold):
X, y = X_train.values, y_train.values
strat_kfold = StratifiedKFold(n_splits=n_fold, shuffle=True,
random_state=1)
accuracy_list = []

for train_index, test_index in strat_kfold.split(X, y):
x_train_fold, x_test_fold = X[train_index], X[test_index]
y_train_fold, y_test_fold = y[train_index], y[test_index]
clf.fit(x_train_fold, y_train_fold)
preds = clf.predict(x_test_fold)
accuracy_test = accuracy_score(preds, y_test_fold)
accuracy_list.append(accuracy_test)

return np.array(accuracy_list).mean()
```

We also define a function with the model hyperparameters we would like to tune, that is, `max_samples`, `max_features` and `n_estimators`.

```
In [51]: def bo_params_rf(max_samples, n_estimators, max_features):
        params = {
            "max_samples": max_samples,
            "max_features": max_features,
            "n_estimators": int(n_estimators),
        }
        clf = RandomForestClassifier(
            max_samples=params["max_samples"],
            max_features=params["max_features"],
            n_estimators=params["n_estimators"],
        )
        score = stratified_kfold_score(clf, X_train, y_train, 5)
        return score
```

In the cell that follows, we give a range of hyperparameter space that the Bayesian optimization will search.

```
In [52]: rf_bo = BayesianOptimization(
        bo_params_rf,
        {"max_samples": (0.2, 1), "max_features": (0.5, 1), "n_estimators":
        (50, 200)},
    )
```

We can now apply Bayesian optimization to search through the hyperparameter space and output the results at each iteration. The cell below outputs the performance for each hyperparameter combination it finds and labels in blue the combination with a higher score than the previous combinations it has already selected.

```
In [53]: results = rf_bo.maximize(n_iter=200, init_points=20)
```

iter	target	max_sa...	max_fe...	n_esti...
1	0.9230769	0.5724624	0.7953968	163.25316
2	0.9230769	0.5411636	0.8880234	74.161925
3	0.9163461	0.2236453	0.5696041	136.94190
4	0.9307692	0.7177223	0.7266936	61.550797
5	0.9163461	0.2727402	0.9613471	177.46666
6	0.925	0.7105819	0.9383236	114.17439
7	0.9211538	0.3664933	0.7482529	68.007222
8	0.9192307	0.3886558	0.5526125	59.403461
9	0.9336538	0.9827582	0.5241808	54.221457
10	0.9096153	0.2115428	0.6600535	66.349187
11	0.9336538	0.8719149	0.7616757	173.06555
12	0.9269230	0.7594846	0.9472900	63.556498
13	0.9240384	0.5110481	0.8782922	66.618828
14	0.9269230	0.4599149	0.8361600	107.56904
15	0.9182692	0.2422423	0.7270673	75.657309
16	0.925	0.5931637	0.5437501	91.489347
17	0.9221153	0.4260971	0.7143414	105.45641
18	0.9182692	0.2554415	0.5157451	79.129160
19	0.9394230	0.8567185	0.5251637	161.95673
20	0.9278846	0.9202999	0.8219508	166.39162
21	0.9269230	0.4838802	0.8106658	137.79349
22	0.9355769	0.8018800	0.5285941	161.97960
23	0.9307692	0.8993388	0.6252669	181.03025
24	0.9298076	0.6436381	0.6087827	194.31644
25	0.9375	0.8026532	0.5344270	161.98362
26	0.9269230	0.4198287	0.9116225	178.87341
27	0.9336538	0.8427919	0.5573998	161.98569
28	0.9307692	0.7215914	0.7756409	191.21162
29	0.9240384	0.3548243	0.7011180	147.02091
30	0.9307692	0.5326916	0.6883358	110.91747
31	0.9432692	0.8505863	0.5110046	161.98821
32	0.9346153	0.8616058	0.7909809	173.02032
33	0.9317307	0.8762530	0.8208919	173.06402
34	0.9365384	0.8837895	0.5	161.98254
35	0.9365384	0.8451659	0.5	161.96620
36	0.9317307	0.8320041	0.5165267	162.02338
37	0.9336538	0.9112985	0.7761076	173.02733
38	0.9230769	0.6532195	0.9463152	101.64192
39	0.9173076	0.2841648	0.6989136	54.639624
40	0.9326923	0.8700646	0.7371067	173.01150
41	0.9346153	0.8701005	0.5285394	161.99051
42	0.9298076	0.8198620	0.7659025	173.04239
43	0.9307692	0.7844590	0.5734815	161.98758
44	0.9259615	0.7155774	0.9382142	182.56055
45	0.9307692	0.7096884	0.8487560	189.77304
46	0.9259615	0.3370642	0.8672816	86.279933
47	0.9326923	0.8891008	0.7838477	172.97709
48	0.9326923	0.8282151	0.5492063	161.94459
49	0.9384615	0.9906684	0.6092368	127.31528
50	0.9346153	0.9157287	0.7229735	173.05630
51	0.9173076	0.2555492	0.6392988	73.351954
52	0.9201923	0.5825071	0.9118242	68.472048
53	0.9375	0.9510886	0.6126288	127.31872
54	0.9317307	0.6521531	0.7030739	182.08383
55	0.9221153	0.4079580	0.6650987	185.62009
56	0.9259615	0.6303729	0.9264831	89.130309
57	0.9201923	0.2816124	0.5074161	66.433107
58	0.9336538	0.9698460	0.5893488	127.28528

59	0.9307692	0.9301465	0.7647613	173.09034
60	0.9355769	0.9766533	0.6474379	127.30506
61	0.9336538	0.9388176	0.7198266	173.00212
62	0.9336538	0.9793363	0.6241921	127.35598
63	0.9346153	0.9740040	0.5707334	127.33802
64	0.9326923	0.9143213	0.6529437	127.31590
65	0.9307692	0.9095362	0.5862235	127.34527
66	0.9326923	0.9088880	0.6675242	173.02942
67	0.9375	1.0	0.6338259	127.25915
68	0.9375	0.9316684	0.6687655	135.14590
69	0.9201923	0.4080337	0.9183365	174.33928
70	0.9182692	0.2995951	0.5852150	76.186663
71	0.9298076	0.9212779	0.6288653	135.12003
72	0.9326923	0.9418948	0.7080657	135.17138
73	0.9336538	0.9456814	0.6470145	135.18682
74	0.9336538	0.8882524	0.6744261	135.16757
75	0.9317307	0.9541097	0.6510758	127.24255
76	0.9278846	0.8853365	0.7105035	69.508873
77	0.925	0.4139186	0.6940545	187.94154
78	0.9375	0.9105299	0.7213094	170.62363
79	0.9375	0.8902845	0.6776409	170.63007
80	0.9326923	0.8816284	0.7138015	170.66495
81	0.9336538	0.9410709	0.6804891	170.62594
82	0.9326923	0.8847563	0.7004630	170.58436
83	0.9375	0.8855908	0.6244287	170.64516
84	0.9298076	0.8707619	0.7871580	88.745203
85	0.9336538	0.8924449	0.6257402	170.59686
86	0.9375	0.9072274	0.8556981	109.17034
87	0.925	0.4247786	0.5512875	69.828142
88	0.9269230	0.7709616	0.8725825	114.54438
89	0.9346153	0.8404783	0.6463685	170.63855
90	0.9288461	0.9074582	0.8991854	109.14423
91	0.9307692	0.9070080	0.8139260	109.19542
92	0.9326923	0.9058638	0.6499647	170.68229
93	0.9394230	0.8567192	0.5792557	170.64821
94	0.9221153	0.3176950	0.6866773	176.77578
95	0.9144230	0.2565987	0.5126297	72.359478
96	0.9269230	0.8706157	0.8726663	109.20431
97	0.9355769	0.9005639	0.5661397	170.65205
98	0.9298076	0.8554108	0.5916655	170.69134
99	0.9365384	0.8551461	0.5652336	170.60702
100	0.9307692	0.9417059	0.8346230	109.14139
101	0.9269230	0.9451873	0.7656391	170.61918
102	0.9298076	0.7282249	0.9229871	57.796764
103	0.9365384	0.8476097	0.5322937	170.64649
104	0.9403846	0.8100535	0.5761032	170.63262
105	0.9365384	0.9317309	0.6048393	120.45487
106	0.9394230	0.8853724	0.6363840	120.45816
107	0.9336538	0.9063313	0.6435644	120.41657
108	0.9182692	0.2612563	0.5932251	114.01620
109	0.9317307	0.9485007	0.8908084	113.20225
110	0.9211538	0.3733999	0.9675236	119.13716
111	0.9317307	0.5844767	0.5124451	120.07627
112	0.9423076	0.8738374	0.5908579	120.46451
113	0.9307692	0.5107830	0.5125949	165.57168
114	0.9346153	0.8854319	0.6056153	120.50118
115	0.9384615	0.8471510	0.6055809	120.43735
116	0.9336538	0.9746408	0.6956624	78.183659
117	0.9365384	0.7856878	0.5365483	170.61748
118	0.9375	0.8876968	0.5688193	120.43184

119	0.9259615	0.5937673	0.9508008	84.725958
120	0.9375	0.8427813	0.5592418	120.46954
121	0.9288461	0.8345298	0.6203961	120.48047
122	0.9355769	0.8988468	0.5275904	161.90945
123	0.9394230	0.9629359	0.5	161.91198
124	0.9365384	0.9544494	0.5016475	161.86310
125	0.9365384	0.8856560	0.7433393	136.39092
126	0.9317307	0.7885012	0.5971745	170.59251
127	0.9336538	0.9736450	0.5473421	161.90130
128	0.9336538	0.9510679	0.5882091	148.66645
129	0.9278846	0.7136174	0.7235378	155.11041
130	0.9355769	0.7840572	0.5566674	170.66868
131	0.9346153	0.8542157	0.6913925	136.38542
132	0.9278846	0.4539530	0.9217381	138.31839
133	0.9278846	0.9253395	0.6986707	136.40384
134	0.9211538	0.4678325	0.8646961	128.62370
135	0.9307692	0.8965739	0.5527225	120.48005
136	0.9269230	0.8322025	0.7576479	136.37600
137	0.9259615	0.9196275	0.7929580	136.39721
138	0.9269230	0.4989763	0.8111177	140.32385
139	0.9336538	0.8333727	0.5572512	120.41591
140	0.9384615	0.8899848	0.7180993	136.33565
141	0.9182692	0.2315756	0.6099393	71.371583
142	0.9336538	0.9829470	0.5	161.95846
143	0.9298076	0.8709625	0.6738693	136.31275
144	0.9346153	0.8864813	0.6058865	120.44151
145	0.9365384	0.9143915	0.7500473	136.31374
146	0.9346153	0.9213682	0.6590679	120.47539
147	0.9259615	0.6054023	0.6797137	194.99382
148	0.9326923	0.8679940	0.7470407	136.30009
149	0.9384615	0.9385039	0.7073193	136.32039
150	0.9336538	0.9284739	0.5	161.93399
151	0.9365384	0.8706443	0.6810756	120.44549
152	0.9384615	0.9130342	0.7188188	120.41960
153	0.9346153	0.8804915	0.6978989	156.12021
154	0.9336538	0.8949741	0.7433747	120.45702
155	0.9307692	0.9230276	0.7054794	135.10592
156	0.9298076	0.8488930	0.7928040	54.874167
157	0.9298076	0.8764788	0.7141156	120.38878
158	0.9278846	0.3941112	0.5698776	132.46783
159	0.9326923	0.9569826	0.7075747	120.42755
160	0.9336538	0.8995575	0.7543454	156.09397
161	0.9346153	0.5540881	0.6271845	122.15822
162	0.9326923	0.8963368	0.5	161.85720
163	0.9298076	0.7939161	0.6071646	170.65522
164	0.9326923	0.8456004	0.7135129	156.06801
165	0.9288461	0.9408295	0.7169725	136.27415
166	0.9336538	0.9306249	0.7622109	120.40672
167	0.9240384	0.4854680	0.6400905	82.569637
168	0.9365384	0.9248395	0.7312494	136.35037
169	0.9173076	0.3351300	0.6749914	141.86328
170	0.9192307	0.2737943	0.5611296	57.946167
171	0.9307692	0.5563989	0.7596537	155.68536
172	0.9317307	1.0	0.5	161.89089
173	0.9307692	0.9882788	0.6025762	120.44393
174	0.9375	0.8199222	0.5515015	170.63717
175	0.9307692	0.7625737	0.5	170.65935
176	0.9173076	0.2719539	0.6683962	156.00052
177	0.9336538	0.9077686	0.5292934	120.38922
178	0.9259615	0.5227802	0.7902010	183.54037

179	0.9403846	0.8352376	0.5	170.58868
180	0.9336538	0.8025343	0.8021426	110.03976
181	0.9326923	0.8110539	0.5157534	170.55951
182	0.9336538	0.8752050	0.5	170.60133
183	0.9221153	0.5397614	0.8594655	195.58129
184	0.9278846	0.5720934	0.6892828	122.15993
185	0.9336538	0.9841798	0.6921208	136.33454
186	0.9375	0.9245357	0.5584488	161.86045
187	0.9288461	0.8707445	0.8256212	108.75059
188	0.9278846	0.8464171	0.7482537	156.14330
189	0.9211538	0.3279100	0.5320825	117.89227
190	0.9317307	0.8332913	0.5864721	81.618356
191	0.9375	0.9006938	0.5892361	161.89362
192	0.9278846	0.5516447	0.5312486	147.59070
193	0.9365384	0.8752380	0.5778948	161.85202
194	0.9384615	0.9193087	0.6129478	161.84976
195	0.9403846	0.9572399	0.5108456	82.263699
196	0.9355769	0.9582652	0.5000012	82.303454
197	0.9326923	0.9586603	0.5520375	82.266518
198	0.9326923	0.9205956	0.5	82.252579
199	0.9346153	0.9187230	0.5842163	161.80893
200	0.9336538	0.9480195	0.6191595	161.88778
201	0.9365384	0.8782264	0.6354820	161.86550
202	0.9230769	0.3909193	0.6800225	53.318069
203	0.9365384	0.8585121	0.5770060	120.44978
204	0.9375	0.8908996	0.6385466	161.81460
205	0.9269230	0.3510162	0.9278837	53.332804
206	0.9298076	0.4380478	0.6620820	115.97417
207	0.9326923	0.8451707	0.6181471	161.81821
208	0.9346153	0.9926968	0.5	82.255156
209	0.9240384	0.3653295	0.7460038	119.65271
210	0.9317307	0.6654116	0.7081515	143.62265
211	0.9317307	0.7538600	0.8731096	162.63279
212	0.9288461	0.5817870	0.5079344	120.08358
213	0.9326923	0.9223750	0.6639424	161.84058
214	0.9144230	0.2675182	0.9200603	158.37898
215	0.9317307	0.8696829	0.8941192	110.09248
216	0.9298076	0.8782531	0.8797283	136.42172
217	0.9394230	0.9095871	0.6408513	161.76470
218	0.9201923	0.3626892	0.9146673	50.158434
219	0.9317307	0.8917793	0.9952115	109.70695
220	0.9336538	0.8450897	0.6104139	161.81269

We can now display the best hyperparameters that have been encountered by Bayesian optimization.

```
In [54]: params = rf_bo.max["params"]
params["n_estimators"] = int(params["n_estimators"])
print(params)
```

```
{'max_samples': np.float64(0.8505863849080447), 'max_features':
np.float64(0.5110046627265787), 'n_estimators': 161}
```

We can now fix these hyperparameters on our model as shown below.

```
In [55]: rf_v1 = RandomForestClassifier(
    max_samples=params["max_samples"],
    max_features=params["max_features"],
    n_estimators=params["n_estimators"],
)
```

We now use our training dataset to train the above model having the best selected hyperparameters.

```
In [56]: rf_v1.fit(X_train, y_train)
```

```
Out[56]: ▼ RandomForestClassifier
RandomForestClassifier(max_features=np.float64(0.5110046627265787),
    max_samples=np.float64(0.8505863849080447),
    n_estimators=161)
```

From the model we have trained above, we use that to predict on the holdout dataset and evaluate the model performance using the classification report.

```
In [57]: preds = rf_v1.predict(X_test)
```

```
In [58]: print(classification_report(preds, y_test))
```

	precision	recall	f1-score	support
0.0	0.96	0.91	0.93	118
1.0	0.93	0.96	0.94	142
accuracy			0.94	260
macro avg	0.94	0.94	0.94	260
weighted avg	0.94	0.94	0.94	260

The classification report above shows that the model correctly predicts the signals with a high precision. We can print out the values of precision score, recall score, and f1 score to better shed light on the classification report.

```
In [59]: print("Precision:{}".format(precision_score(preds, y_test)))
print("Recall:{}".format(recall_score(preds, y_test)))
print("F1 Score:{}".format(f1_score(preds, y_test)))
```

```
Precision:0.9256756756756757
Recall:0.9647887323943662
F1 Score:0.9448275862068966
```

As alluded, we get high performance on the classification report. Can we improve the model performance? Let's implement the stratified `kFold` validation below.


```
In [60]: cv = StratifiedKFold(n_splits=5, random_state=123, shuffle=True)
scores = []

for (train, test), i in zip(cv.split(X_train, y_train), range(5)):
    rf_v1.fit(X.iloc[train], y.iloc[train])
    preds = rf_v1.predict(X_train.iloc[test])
    accuracy = accuracy_score(preds, y_train.iloc[test])
    scores.append(accuracy)

df_val = pd.DataFrame(scores, columns=["Accuracy Test"])
print(
    "KFold validation mean accuracy on test set : {}".format(
        df_val["Accuracy Test"].mean()
    )
)
```

KFold validation mean accuracy on test set : 0.9365384615384615

The accuracy on the test dataset shows a comparable result with what we have seen so far.

7. Backtesting

We now want to see how our trading strategy would perform on historical data, and this is possible with the help of backtesting. Backtesting allows us to simulate how we would have performed in the past without using any financial capital.

```

In [61]: # Create column for Strategy Returns by multiplying the daily returns by
         # the position that was held at close
         # of business the previous day
         backtestdata = pd.DataFrame(index=X_validation.index)
         validation_prediction = rf_v1.predict(X_validation)
         backtestdata["signal_pred"] = validation_prediction
         backtestdata["signal_actual"] = y_validation
         backtestdata["Market Returns"] = X_validation["Close"].pct_change()
         backtestdata["Actual Returns"] = backtestdata["Market Returns"] *
         backtestdata[
             "signal_actual"
         ].shift(1)
         backtestdata["Strategy Returns"] = backtestdata["Market Returns"] *
         backtestdata[
             "signal_pred"
         ].shift(1)
         # backtestdata=backtestdata.reset_index()
         backtestdata.head()

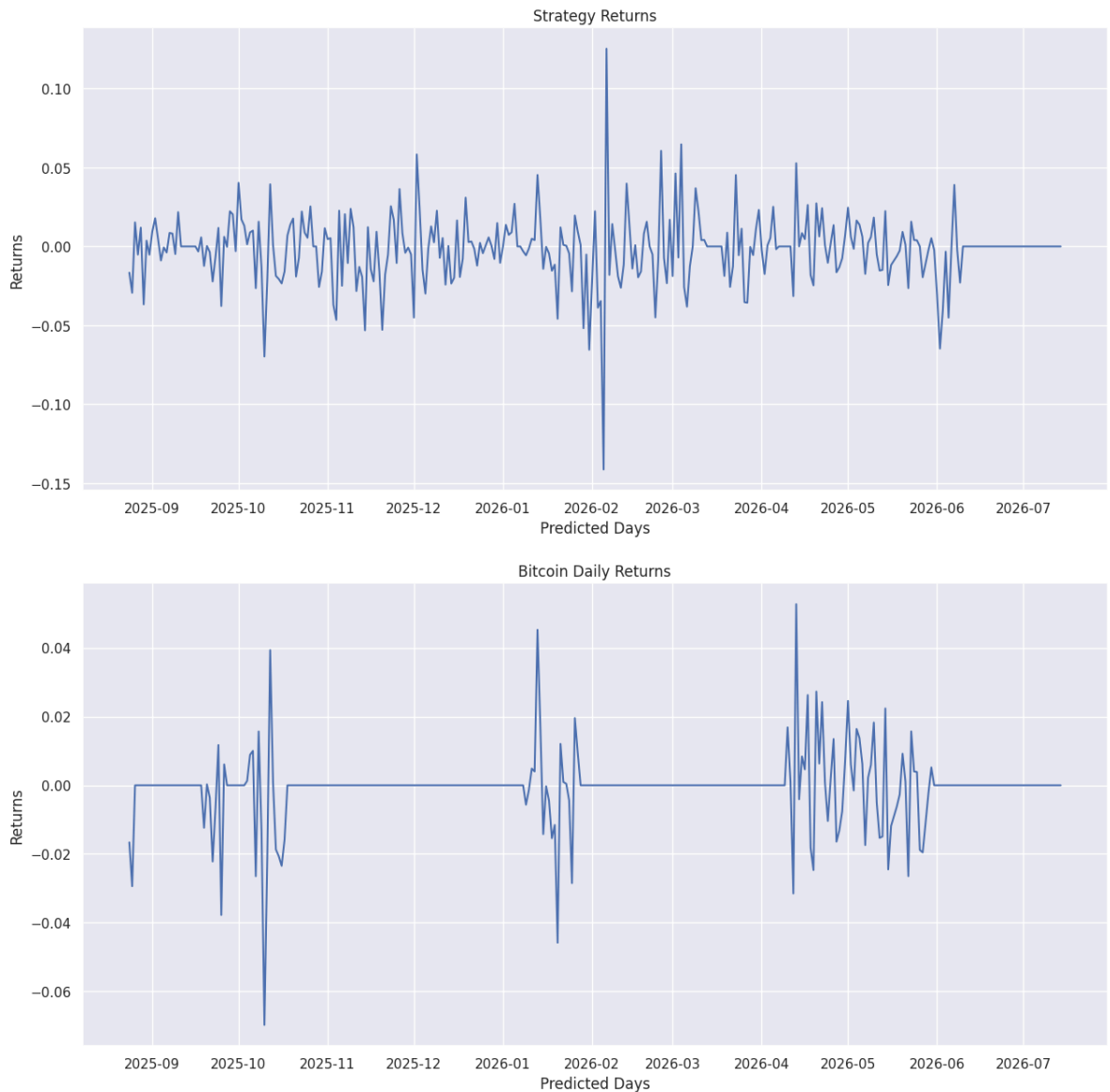
         fig, axs = plt.subplots(2, figsize=(15, 15))
         fig.suptitle("Vertically stacked subplots")
         axs[0].plot(backtestdata["Strategy Returns"])
         axs[0].set_title("Strategy Returns")
         axs[1].plot(backtestdata["Actual Returns"])
         axs[1].set_title("Bitcoin Daily Returns")

         for ax in axs.flat:
             ax.set(xlabel="Predicted Days", ylabel="Returns")

         # `backtestdata`[['Strategy Returns','Actual Returns']].plot()
         # `plt.title`('Cumulative Strategy Returns vs Cumulative Actual Returns')
         plt.suptitle(
             "Fig. 10: Strategy and Bitcoin Daily Returns Separate Graphs",
             fontweight="bold",
             horizontalalignment="right",
         )
         plt.show()

```

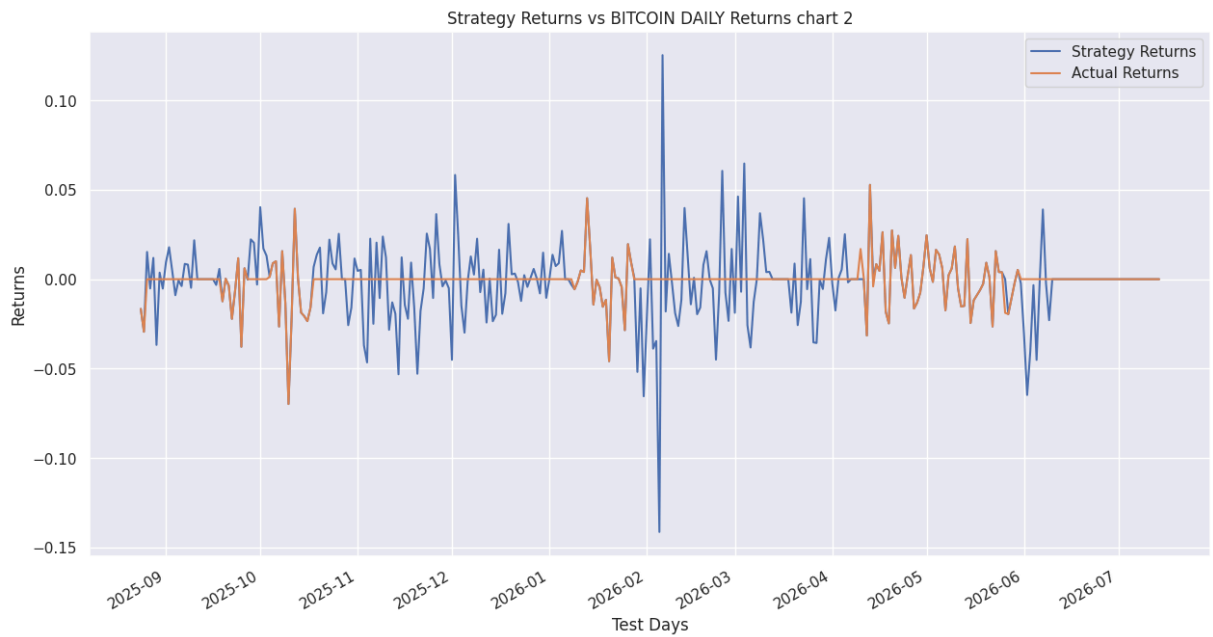
Fig. 10: Strategy and Bitcoin Daily Returns Separate Graphs



In the above diagrams, we visualize the strategy and Bitcoin daily returns in two separate figures. We put the above figures on the same figure below.

```
In [62]: backtestdata[["Strategy Returns", "Actual Returns"]].plot(figsize=(15, 8))
plt.title("Strategy Returns vs BITCOIN DAILY Returns chart 2")
plt.suptitle(
    "Fig. 11: Strategy and Bitcoin Daily Returns",
    fontweight="bold",
    horizontalalignment="right",
)
plt.xlabel("Test Days")
plt.ylabel("Returns")
plt.show()
```

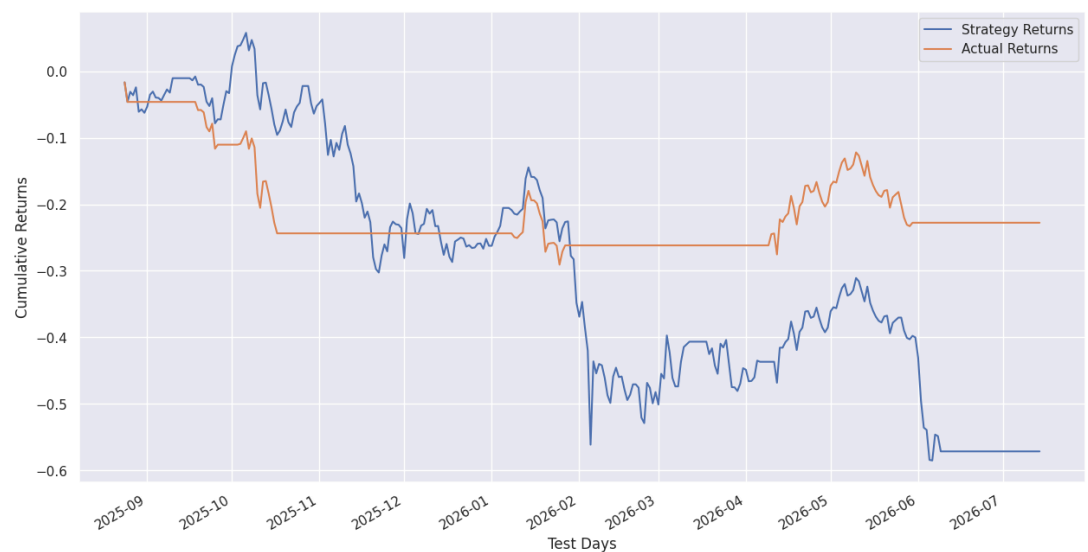
Fig. 11: Strategy and Bitcoin Daily Returns



We can clearly see that the strategy and Bitcoin daily return are identical on most test days with the exception of very few days. We can now plot the cumulative returns for both strategy returns and the Bitcoin returns to compare how we expect to perform in the near future if we applied our trading model.

```
In [63]: backtestdata[["Strategy Returns", "Actual  
Returns"]].cumsum().plot(figsize=(15, 8))  
plt.xlabel("Test Days")  
plt.ylabel("Cumulative Returns")  
plt.suptitle(  
    "Fig. 12: Cumulative Strategy Returns vs Cumulative BITCOIN Returns",  
    fontweight="bold",  
    horizontalalignment="right",  
)  
plt.show()
```

Fig. 12: Cumulative Strategy Returns vs Cumulative BITCOIN Returns



7. Conclusion

In this lesson, we have learned how to apply Bayesian optimization and grid search on our model to improve performance. We also saw how to use baseline models as a reference to the models we will use to predict the buy or sell signals. We anticipate that more could be done to better improve the model, and we ask the students to explore more.

References

1. Snoek, Jasper, et al. "Practical Bayesian Optimization of Machine Learning Algorithms." *Advances in Neural Information Processing Systems*, vol. 25, 2012.
2. Shalini, Talwar, Shah Pranav, and Shah Utkarsh. "Picking buy-sell signals: A practitioner's perspective on key technical indicators for selected Indian firms." *Studies in Business & Economics* 14.3 (2019).
3. Bergstra, James, et al. "Algorithms for Hyper-parameter Optimization." *Advances in Neural Information Processing Systems*, vol. 24, 2011.
<https://proceedings.neurips.cc/paper/2011/file/86e8f7ab32cfd12577bc2619bc635690-Paper.pdf>

Copyright 2025 WorldQuant University. This content is licensed solely for personal use. Redistribution or publication of this material is strictly prohibited.